

Received 19 February 2025, accepted 12 March 2025, date of publication 17 March 2025, date of current version 28 March 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3552225

 SURVEY

Data-Driven Motion Planning: A Survey on Deep Neural Networks, Reinforcement Learning, and Large Language Model Approaches

GABRIEL PEIXOTO DE CARVALHO^{1,2}, TETSUYA SAWANOBORI²,
AND TAKATO HORII¹, (Member, IEEE)

¹Graduate School of Engineering Science, Osaka University, Osaka 560-8531, Japan

²Connected Robotics Inc., Tokyo University of Agriculture and Technology Koganei Campus, Tokyo 184-0012, Japan

Corresponding author: Gabriel Peixoto De Carvalho (g.peixoto@rlg.sys.es.osaka-u.ac.jp)

This work was supported by the Japan Science and Technology Agency (JST) Moonshot Research and Development under Grant JPMJMS2011.

ABSTRACT Motion planning is a fundamental challenge in robotics, involving the creation of trajectories from start to goal states while meeting constraints like collision avoidance and joint limits. Its complexity increases with the number of robot joints. Several traditional approaches tackle this problem, such as sampling motion planning, grid-based methods, potential fields, and optimization techniques. Recent advancements in deep neural networks, reinforcement learning, and large language models enable new possibilities for solving motion planning problems by improving sampling efficiency, optimizing control policies, and enabling task planning through natural language prompts. This survey comprehensively reviews these novel approaches, providing background knowledge, analyzing key contributions, and identifying common patterns, limitations, and research gaps. Our work is the first to integrate all three major data-driven approaches, discussing their applications and future research directions.

INDEX TERMS Robotics, deep learning, survey, large language models, neural networks, reinforcement learning, motion planning, task and motion planning.

I. INTRODUCTION

Motion planning is a PSPACE-hard problem [1], [2], [3] and numerous approaches have been proposed to address it under varying constraints. The complexity is influenced by the robot's configuration, such as a car navigating a surface versus a six-joint arm moving in 3D. Essential for robots to interact with their environment, motion planning is crucial for applications from factory assembly [4], [5] to human-robot interaction [6], [7]. The motion planning problem has different subproblem formulations, such as path planning, constrained motion planning, kinodynamic motion planning, and TAMP [8], [9], [10]. Each subproblem considers a specific set of constraints.

The associate editor coordinating the review of this manuscript and approving it for publication was Yiming Tang¹.

A. PATH PLANNING

Path planning is the simplest formulation because it only considers the kinematic aspect of motion planning, that is, collision avoidance and joint limits [1], [11], [12].

B. KINODYNAMIC MOTION PLANNING

Kinodynamic motion planning considers kinematic and dynamic constraints, that is, collision avoidance, joint limits, velocity, and acceleration constraints [2], [12].

C. CONSTRAINED MOTION PLANNING

Constrained motion planning involves the kinematic limitations beyond collision avoidance, for example, using a 6-degree of freedom (DOF) robot to draw on a flat surface or opening a door or drawer [13].

D. TASK AND MOTION PLANNING

Task and motion planning extends traditional motion planning by incorporating task planning, which integrates

multiple motion planning solutions into a cohesive sequence of actions. For example, instead of just planning the motions to grab a bottle and throw it in the garbage can, task and motion planning coordinates these steps into a unified plan that considers both the high-level actions and the necessary motions to accomplish the goal [8], [9], [10]. Therefore, solutions focus on a specific subproblem depending on the use case.

Different approaches to solving motion planning exist, such as sampling-based motion planning (SBMP), grid-based motion planning, planning as optimization, potential fields, and differential equations.

E. SAMPLING-BASED MOTION PLANNING

SBMPs are the most successful class of motion planning-solving algorithms [1], [11], [12]. They use random sampling to dynamically construct a graph that eventually connects the starting to the target states. These methods succeed in real-world scenarios because they scale well to high-dimensional scenarios [1], [11], [12]. The sampling-based algorithms are probabilistic complete, that is, as the number of samples increases, the probability of finding a solution also increases.

However, the performance and speed of the sampling-based approaches depend on the environment's complexity and the problem's conditions. For instance, in environments with narrow passages or cluttered obstacles, the probability of sampling valid configurations decreases, leading to longer computation times. Similarly, in high-dimensional spaces, such as those involving robotic arms with many degrees of freedom, due to the high dimensionality of the sampling space, sampling valid configurations might take longer. Furthermore, collision check complexity increases as the environment becomes more cluttered, because each robot link must be checked against each object of the environment at each step. Finally, problem-specific conditions, such as dynamic obstacles, non-holonomic constraints, or the need for optimality in the solution path, can further challenge the efficiency of SBMP algorithms. For example, in autonomous driving scenarios, the presence of moving vehicles and pedestrians requires real-time adaptability, while in industrial robotics, precise motion constraints must be satisfied to avoid collisions. These factors highlight the importance of tailoring SBMP algorithms to the specific characteristics of the environment and problem at hand.

F. GRID-BASED MOTION PLANNING

Grid-based methods discretize the environment and apply graph search methods to find the path from start to goal. This approach requires memory to store the discretized environment and does not scale well to high-dimensional scenarios, where processing and memory requirements increase exponentially [11], [14]. The grid-based approaches are resolution complete, that is, the planner finds a solution if one exists within the discretization resolution.

G. ARTIFICIAL POTENTIAL FIELDS MOTION PLANNING

The artificial potential fields (APF) approach models motion planning using physics concepts, assigning a high potential to obstacles and a low potential to the goal [11], [14]. This creates a field where the agent is "attracted" to the goal and "repelled" by obstacles, enabling real-time motion planning, even in dynamic environments. However, the robot may get stuck in local minima depending on the obstacle layout [11], [14].

SBMPs, grid-based, and artificial potential field planners are path planners in their original form. However, it is possible to extend these formulations to consider more constraints by adding additional steps after finding an initial solution, and this is where one might use planning as optimization [11], [14]. Optimization and differential equation methods rely on an initial solution as a starting point [11], [14]. From there, they iteratively refine the solution using an optimization function that incorporates various constraints. However, these methods tend to be significantly slower than other planning approaches and often require manually crafted optimization functions, making them less efficient and more time-consuming [11], [14].

H. AI-BASED METHODS

In the past decade, applying deep neural networks to approximate functions from data has become common in different research fields, including motion planning [1], [15]. The progress of deep neural networks for image processing and natural language recognition inspires its application in robotics. The works of Qureshi et al. [1], [7] and Ichter et al. [15], [16] are the initial approaches using these neural networks. Both papers replace a sampling-based motion planner's sampling component with a neural network that learns a biased sampling distribution from past solutions. By learning sample distributions conditioned to the environment information, this framework achieves solutions more than ten times faster than traditional methods, improving the sampling efficiency considerably and consequently reducing the number of collision-checking operations.

Another approach, more aligned with control theory, is to model motion planning, or any of its subproblems, as a reinforcement learning problem. In this framework, discrete or continuous reinforcement learning algorithms control the robot's motion, using feedback from the environment to guide decision-making and improve performance over time [17], [18].

The most recent approach is to use large language models (LLMs) for task planning component in TAMP or fine-tune these models with trajectories, control inputs, and sensor data for direct motion planning and task planning using human natural language prompts [19], [20]. While LLMs demonstrate strong capabilities for high-level task planning without requiring additional training, they are not well-suited for low-level planning tasks. Current methods that combine heuristics with neural networks to accelerate

planning execution remain the most effective in terms of both accuracy and speed. However, since LLMs are still in the early stages of development, it is possible that future developments could make them more suitable for low-level control tasks.

Our survey presents a review of methods in the intersection of data-driven artificial intelligence (AI) and motion planning, including neural networks, reinforcement learning, and large language models. It presents all the background knowledge necessary to understand the works we discuss in this present study. Additionally, we execute a deep analysis of each work and explain the main contributions and strong and weak points relative to other works in the context of motion planning problems. We explore a wide range of potential applications, from robotic arm manipulation [1] to self-driving cars [21], as both are fundamentally motion planning problems. Our survey serves as a starting point for researchers aiming to grasp the current state-of-the-art in motion planning using data-driven, neural-based techniques.

Previous surveys in the field focus heavily on one specific approach [22], [23] or have a broad but shallow focus on a wider range of approaches [24], which are valid. However, our survey presents all the necessary background knowledge for the reader to develop a deeper understanding of a wide spectrum of works in the machine learning-driven motion planning space. Additionally, we emphasize the interconnections between different approaches and draw parallels to identify common patterns in existing research, helping to illustrate how researchers apply advances in machine learning and deep learning to motion planning problems. A key contribution of our survey is the inclusion of motion planning using large language models (LLMs), which, to the best of our knowledge, has not been covered in previous reviews. By incorporating this emerging area, we aim to address gaps in prior surveys and present a more thorough and critical discussion of the research landscape.

As conclusive remarks, we also discuss future directions for the field, highlighting emerging trends and key challenges in motion planning with data-driven approaches. We focus on potential advancements in neural networks, reinforcement learning, and LLMs, as well as the need for more efficient, scalable methods for real-time applications.

We organize the remaining sections of this paper as follows: Section II presents the motion planning problem, its subproblems, applications, and metrics, and an in-depth analysis of traditional algorithms. Section III discusses previous surveys on this topic and highlights our survey contributions in contrast to others. Section IV presents the theoretical background of machine learning, from neural networks to large language models. Section V reviews the current state of the art in neural motion planning, including reinforcement learning and LLMs. Section VI presents discussions and remarks about the existing research and future trends.

II. MOTION PLANNING PROBLEM

In this section, we follow the taxonomy and definitions of Lynch et al. [11]. The most basic definition of motion planning is the problem of finding a robot motion from a start state to a goal state that satisfies a set of constraints (e.g., collision avoidance, joint limits, torque limits, and other kinematic or dynamic constraints). Additionally, we recommend other references beyond the main ones used in our survey, such as the book by Howie et al. [14], the course notes by Tedrake [25] and the SBMP survey by Orthey et al. [12].

To facilitate the understanding of the various concepts in this section, Figure 1 presents a concept map illustrating the relationships of each concept and provides some examples.

A. C-SPACE

The configuration space (C-space or C) is a fundamental concept for motion planning, as it defines the space within which motion planners operate. The C-space is the space that contains all the possible robot configurations. It is divided into two regions: C_{free} , the free space, and C_{obs} , the obstacle space. The free space contains configurations the robot may assume without collision and exceeding joint limits (respecting other constraints). The obstacle space contains all the configurations resulting in collision or disrespecting other constraints (e.g., joint limits). Finally, $C = C_{free} \cup C_{obs}$. The C-space of a robot arm with n joints are represented as a list of n joint positions $q = (\theta_1, \theta_2, \dots, \theta_n)$ with $C \subset \mathbb{R}^n$.

B. MOTION PLANNING TAXONOMY

We categorize the motion planning problem into groups of methods that address specific scenarios within the broader research field. These methods take into account factors such as the constraints that the solution must satisfy, computational speed, optimality, and the precision of the final solution. From this initial categorization, we construct a taxonomy of motion-planning methods based on the key aspects they consider when formulating solutions.

Complete motion planning involves finding a path from the starting state to the goal state, along with determining the control inputs, such as velocity and acceleration. However, in many cases, simple path planning suffices. In other words, path planning is a subproblem within the broader motion planning framework. Path planning involves the geometric task of identifying a collision-free path from a start to a goal configuration, without considering kinematic and dynamic constraints beyond collision avoidance and joint limits. Most approaches in the literature fall into this category, where researchers focus on finding a trajectory in C-space and delegate kinematic and dynamic optimization to the robot controller [11].

On the other hand, when adhering to kinetic and dynamic constraints is part of the problem, i.e., a robot erasing a blackboard, where the motion in 6D (translation and rotation in the x , y , z axis) is constrained to 3D space (translation in the x , y axis and rotation in the z axis), there is a subproblem of

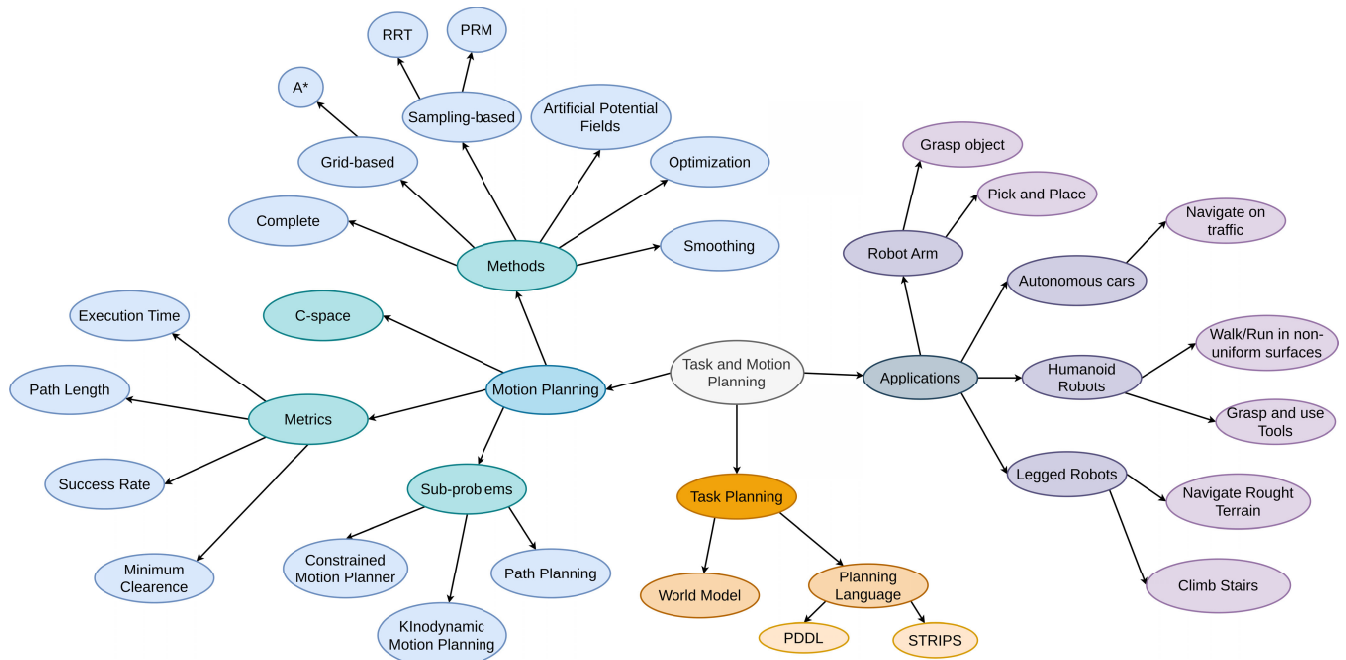


FIGURE 1. A concept map illustrating the relationships between TAMP and its core components. The map branches into three main areas: motion planning, task planning, and applications. While motion and task planning are foundational components of TAMP, the applications branch shows use cases common to all three planning domains. Terminal nodes provide concrete examples of each concept.

constrained motion planning. Another way of understanding this relates to the number of control inputs we provide to the robot. When working with robot arms, the number of control inputs m equals the number of joints n , thus $m = n$. However, there are systems where the number of control inputs is smaller than the degrees of freedom, making the motion kinematically constrained, which is the case for car robots, where $n = 3$ (position x , y and orientation θ of the chassis in the plane) but has $m = 2$ (forward and backward motion and steering angle).

Furthermore, the most common validity conditions a robot arm motion is subject to are: joint limits and collision. Most motion planning algorithms consider at least these two validity conditions [1], [11].

Regarding computational speed, there are two types of planners: online and offline. Online planners are required when we assume that there is a possibility that the environment may change at any time. Thus, the planner must find a solution quickly for a new environment every time, whereas a slower offline planner is enough in the opposite case.

Regarding optimality, for some scenarios, even reaching a solution is already very complex, and reaching an optimal solution is intractable [26]. A satisfactory planner finds a trajectory from the starting state to a goal state without executing any optimization (e.g., time or length). In contrast, an optimal planner optimizes the first solution path toward a condition, for example, time, torque, or path length.

Finally, regarding exactness, the planner finds a path from start to goal where the goal state is sufficiently close to the

desired goal state, or it needs to find a trajectory that arrives exactly in the goal state.

C. MOTION PLANNING PROPERTIES

All motion planning algorithms share two categories of properties: query and completeness. The query property relates to how the planner constructs a representation of the environment and whether we expect the environment to change. Completeness is the most crucial property of a motion planner [1]. These planning algorithms ultimately control real-world systems, such as cars and robot arms. Therefore, the solutions should be exact (or within a known small interval) and adhere to some notion of completeness which provides a notion of how much “trustworthy” that algorithm is.

Regarding the query property, If the planning environment does not change over time (or we do not expect it to change), one may want to spend some time building a representation of C_{free} . After that, subsequent queries use this representation to find solutions more efficiently: the multi-query planners [11].

On the other hand, if we expect the environment to change during the robot operation (e.g., obstacles appear or change place), one may prefer to use a single query planner that solves the problem every time. This is to avoid building a representation of C_{free} due to its complexity [11].

A planner is said to be complete if it is guaranteed to find a solution in finite time if one exists and to report failure if there is no feasible plan. This notion of completeness makes motion planning intractable to complex real-world problems.

Thus the methods focus on weaker notions of completeness: resolution and probabilistic completeness.

A planner is considered resolution complete if it is guaranteed to find a solution if one exists at the resolution of a discretized representation of the problem. Grid-based methods are resolution complete, while sampling-based methods are probabilistic complete. Probabilistic completeness refers to the property where the probability of finding a solution increases over time. Therefore, if a solution exists, it is guaranteed to be found eventually [11].

D. FORMAL DEFINITIONS

We establish the general formal definitions for motion planning and constrained motion planning problems, which assist in understanding subsequent algorithms and diagrams. For more eager readers, we recommend Lavelle [26] and Lynch [11] books for general motion planning problem definition and the survey by Kingston et al. [13] that covers sampling-based constrained motion planning. In this subsection, we formalize motion planning and constrained motion planning problems enough to help the explanation of subsequent methods. For more details, consult the suggested references.

1) MOTION PLANNING

At the beginning of this section, we define the *C-Space* as C , which is the space of all robot configurations. We also define the free space and the obstacle spaces as C_{free} and C_{obs} , then $C = C_{free} \cup C_{obs}$. In this context, we formalize the motion planning problem. Let $C_{free} \subseteq C$, which is the set of all configurations where the robot does not collide with any obstacles or with itself. Given a start configuration $q_{start} \in C_{free}$ and a goal configuration $Q_{goal} \in C_{free}$. The motion planning problem is to find a continuous path $\tau : [0, 1] \rightarrow C_{free}$ that connects $q_{start} = \tau(0)$ and $\tau(1) \in Q_{goal}$. Motion planning is a PSPACE-HARD problem [13], [26]. Figure 2 illustrates a simple 2D setting with C_{free} and C_{obs} .

2) CONSTRAINED MOTION PLANNING

The most common occurrence is motion planning with kinematic constraints [27], [28]. For simplicity, we formalize the motion planning with kinematic constraints. Since the *C-space* contains all robot kinematic configurations. We define constrained motion planning in this context. However, we consider constrained motion planning for both kinematic and dynamic constraints.

To formalize constrained motion planning, we need to define two key concepts: constraint function and constraint manifold. A constraint function describes the constraint $F : C \rightarrow \mathbb{R}^k$ such that $F(q) = 0$ when q satisfies the constraints. k is the number of equality constraints, and F must be continuous and differentiable. The constraint function defines a m -dimensional constrained configuration space within the *C-space* (C), which we call constraint manifold $X : X = \{q \in C \mid F(q) = 0\}$. The constraint manifold is the space that

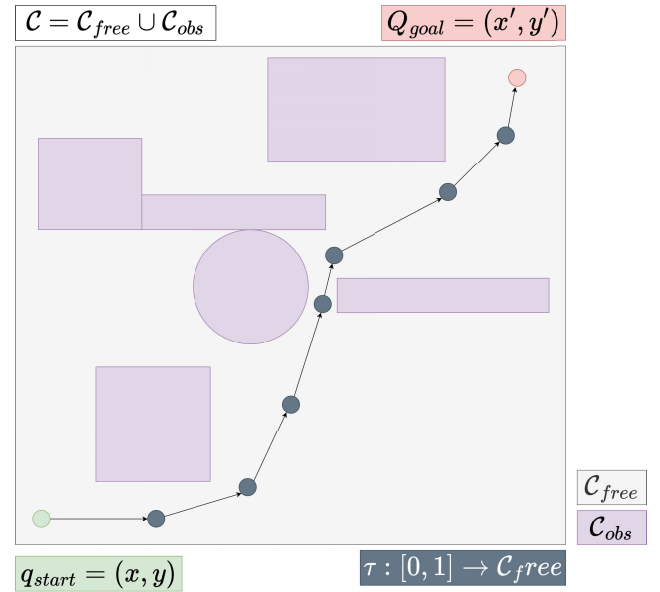


FIGURE 2. 2D problem setting for motion planning showing the *C-Space* and example trajectory τ from q_{start} to Q_{goal} .

contains all configurations that satisfy the constraint [13], analogous to the *C-Space* (C).

Constrained motion planning requires that function F , and C , X manifolds are continuous and differentiable. In this context, the constrained motion planning definition is similar to standard motion planning, where the objective is to find a continuous path $X_{free} = X \cap C_{free}$ that connects a given start q_{start} and $q \in Q_{goal}$ [13]. Figure 3 illustrates the constrained motion planning setting.

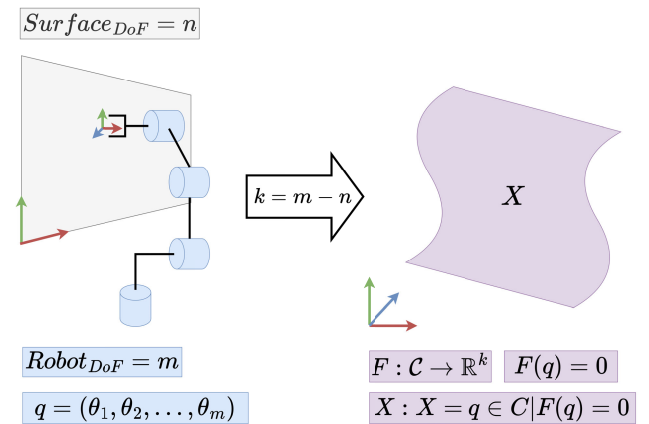


FIGURE 3. A constrained setting where a robot with m degrees of freedom is constrained to move on a n dimensional surface. This is one example of kinematic constraints, but it can include other dynamic constraints such as speed and acceleration.

E. KINODYNAMIC MOTION PLANNING

In the simplest terms, kinodynamic motion planning is a subproblem within the motion planning problem where the robot is subject to kinematic and dynamic constraints. Thus,

given a robot system, kinodynamic motion planning aims to find the shortest trajectory from a starting position and velocity to a goal position and velocity while avoiding obstacles and obeying velocity and acceleration constraints. The works usually express these constraints in terms of differential equations. Thus, we refer to this problem as planning under differential constraints [2].

Analogously to the motion planning problem, there are no exact solutions for the kinodynamic planning problem, and it is an NP-hard problem [3]. Therefore, all solutions are approximations that compute solutions close to the optimal. The core task of the kinodynamic motion planning algorithm is the combinatorial search. Formally, we define kinodynamic motion planning as follows:

Let $\mathcal{X} \subseteq \mathbb{R}^n$ and $\mathcal{U} \subseteq \mathbb{R}^m$ be the state space and the control space, respectively, and the dynamics of the robot are: $\dot{x}(t) = f(x(t), u(t))$, $x(t) \in \mathcal{X}$, $u(t) \in \mathcal{U}$. The configuration and workspace relate to the robot state space. Therefore, given an initial state x_{init} and a goal state $\mathcal{X}_{goal}$ in the obstacle-free region $\mathcal{X}_{free} = \mathcal{X} \setminus \mathcal{X}_{obs}$ method seeks state and control trajectories $x : [0, T] \rightarrow \mathcal{X}$, $u : [0, T] \rightarrow \mathcal{U}$ that are dynamically feasible and satisfy equation above, are collision-free ($x(t) \in \mathcal{X}_{free}$ for all $t \in [0, T]$) and satisfy boundary conditions $x(0) = x_{init}$, $x(T) \in \mathcal{X}_{goal}$ [2].

This type of problem appears in applications such as mobile robots, wheeled vehicles, fixed-wing aircraft, and multi-joint robots that must balance or swing to achieve the goal. Regarding the solution methods, the initial approaches used dynamic programming search over a state transition graph in a position-velocity grid. Currently, SMBP methods are the default solution for this problem because they find feasible trajectories quickly [2].

F. TASK AND MOTION PLANNING

Task and motion planning has two main components: task planning and motion planning. Since we already define motion planning in II-D1, we primarily focus on task planning in this subsection. Given a starting state and a goal task, the task planner must find a sequence of actions (plan) to achieve the goal using an internal world model. Whereas motion planning actions are exclusive motions, task planning is one level higher, and the actions are more abstract, such as grabbing an object or moving to a location [9], [10].

To achieve task planning, we need a world model and planning language. The world model primarily describes the world's state, including locations, robots, and objects. The planning language describes the planning domain. The first planning language is the Stanford Research Institute Problem Solver (STRIPS) [29]. The most popular planning language is the Planning Domain Definition Language (PDDL) [30].

The planning languages describe the entire robot action space and possible problems. Robots in different environments and with different goals may take the same actions in this space. Therefore, PDDL defines a domain and a task-specific problem. Domains describe how the world works,

and the task-specific problem defines the parameters for the task planning, such as agents, objects, locations, states, and goals [30].

Similarly to motion planning, task planning is a search problem and PSPACE-Complete. The task planner must search the action space for feasible actions to reach the goal. That is why the planning language must describe the domain and problem precisely to avoid ambiguities similar to programming languages. Similarly to motion planning, there are heuristic solvers for task planning; one example is the Fast-forward (FF) planning system [31].

Task planning allows robots to reason about the world and devise a plan to achieve a goal. However, task planning handles the high-level reasoning of the task, so it does not consider kinematic, dynamic, or other feasibility checks on the domain and problem definitions. Therefore, there is TAMP, which considers task planning, kinematic, dynamic, and other feasibilities to achieve the goal [9], [10].

Task and motion planning bridge the gap between task planning symbolic nature and the real-world interaction of motion planners. The key idea is that before making the plan or during its creation, it may use motion planning to check the feasibility of the plan's actions. Therefore, motion planning concretizes the task planning symbolic plan in the real world [9], [10].

Since motion and task planning may have large search spaces, finding all possibilities becomes untractable. Therefore, several approaches to handling this search exist, such as hierarchy [8], [29] and sampling [32]. Exploring classic TAMP approaches is outside of the scope of this paper. The critical ideas of TAMP are enough to understand the neural approaches that solve TAMP problems. We provide references for surveys that explore TAMP in-depth [9], [10].

G. MOTION PLANNING METHODS

There are different classes of methods that tackle motion planning problems. Depending on the problem representation, the type of motion planning method focuses on solving different subproblems with different properties. This subsection briefly describes the main classes of motion planning methods in the literature: complete, grid-based, sampling-based, artificial potential fields, optimization, and smoothing.

Complete methods aim at finding exact representations of the geometry and topology of $\mathcal{C}_{free}$ ensuring completeness. These methods quickly become intractable for problems except for the simplest low-dimensional motion planning problems.

Grid methods discretize the $\mathcal{C}_{free}$ into a grid and use graph search algorithms (e.g., Dijkstra [33], A* [11]) to find a plan from the starting state to the goal state. This method guarantees resolution completeness depending on the size of the grid. However, the memory requirements of this category of method grow exponentially with the number of dimensions in the space, limiting its usage for low dimensional problems.

Sampling-based motion planning methods rely on a function to draw samples of the C -space. Another method checks if this sample is valid (according to the set of conditions) and connects it to the previous valid sample using a local planner. The algorithm repeats the sampling and validity check multiple times to build a graph or tree that connects the starting state to the goal state. These methods guarantee probabilistic completeness [11], [12], [26].

The artificial potential field methods derive from the theory of potential energy fields [11], [14], where a body naturally moves from a region of high potential energy to a region of low potential energy. It attributes the lowest potential to the goal state and the highest potential to the obstacles. By applying force to the robot, which equals the negative gradient of the potential, it flows naturally to the goal. The gradient computation is quick, which makes this method fast, and the planner can react to changes in real-time. However, there is a possibility that the robot gets stuck in the local minima of the artificial potential field and does not reach the goal [11], [14].

One may express the motion planning problem as a non-linear optimization problem. However, the optimization methods usually require a good initial guess of the solution, which makes them unsuitable for most use cases [11], [14].

Smoothing methods focus on improving existing solutions for shorter and less jerky paths. This method class is usually a post-processing step after an initial solution is already available [11], [14].

The sampling motion planner methods are modular, and scalable (specifically for high-dimensional robot arm planning) [7], [11], [26]. Due to its modularity, a great portion of neural planning approaches try to extend sampling motion planners [1], [7], [27], [28], [34], [35], [36], [37]. Therefore, we focus on presenting a more complete definition of sampling motion planners and their components as a base for understanding the neural planning approaches.

H. SAMPLING-BASED MOTION PLANNING

Sampling-based motion planners are among the most successful types of planning algorithms due to their scalability for high-dimensional problems and their ability to produce satisfactory plans within a reasonable time frame [1], [11], [12], [26]. These planners use several key functions: one to sample the C -space, typically through random uniform sampling, one to verify the validity of the sampled point by checking it against a set of conditions for a valid state, one to find the nearest node to the newly sampled point, and a local planner to connect the new valid sample to the existing graph. There are two primary approaches to sampling-based motion planning: rapidly-exploring random trees (RRTs) [38] and probabilistic roadmaps (PRMs) [39], which are designed to solve single-query and multi-query problems, respectively.

We focus on RRT [38] and PRM [39] because these algorithms (and variants [40], [41], [42]) are the simplest sampling-based motion planning algorithms and are the base

Algorithm 1 RRT Algorithm

```

1:  $T.InitTree(x_{start})$  ▷  $T$  is the search tree
2: while  $T.size() < T.maxSize$  do
3:    $x_{samp} \leftarrow \text{sample from } \mathcal{X}$ 
4:    $x_{nearest} \leftarrow T.NearestNode(x_{samp})$ 
5:    $x_{new} \leftarrow LocalPlanner(x_{samp}, x_{nearest})$ 
6:   if  $IsValid(x_{nearest}, x_{new})$  then ▷ Constraint checker
7:      $T.AddNode(x_{nearest}, x_{new})$  ▷ Add node to tree  $T$ 
8:     if  $x_{new}$  is in  $\mathcal{X}_{goal}$  then
9:       return  $SUCCESS, RecoverPath(x_{start}, x_{new})$ 
10:    end if
11:  end if
12: end while
13: return FAILURE

```

of many neural planning approaches [1], [7], [15], [16], [27], [28], [34], [35], [36], [37].

First, we present the RRT [38] on Algorithm 1. Lines 3, 4, 5, and 6 are the essential lines of the Algorithm 1, and we go through each one to build a general notion of the RRT algorithm, which gets reused on other variants [40], [41], [42].

- **Line 3 executes the sampler:** Usually, the algorithm draws random samples from a random distribution which is straightforward for Euclidean C -spaces and n -joint robot C -spaces but for other types of systems, it is not straightforward.
- **Line 4 finds the nearest node in the tree:** This step depends on a definition of distance, which depends on the C -space definition and type of robot. The distance may be a simple Euclidean distance in a robot without motion constraints in $\mathbb{R}^2$ or $\mathbb{R}^3$, but for a car-like robot, the definition of distance also depends on the controls of the robot since a car-like robot cannot move sideways.
- **Line 5 executes the local planner:** It aims to find a trajectory from $x_{nearest}$ to x_{new} , which also depends on the type of robot and controls available, for simple scenarios (most of the cases) a straight line planner suffice, but for robots under constrained motion, this is not straightforward.
- **Line 6 executes the validity checker:** Here, we define this more generically as a sampled state validity checker. Usually, this method only checks for collision and joint limits. However, this line is usually not defined in most motion planning libraries, so the user must define the conditions for the sampled state to be valid or not.

RRT has several variants, so let us explore two of them, which some neural planners reuse: RRT* [42] and BRRT [26]. We choose these algorithms because they introduce simple modifications to the original RRT algorithm so we demonstrate how to extend and improve the original RRT algorithm.

The paths that RRT finds are not optimal due to the random sampling step. To improve upon this, RRT* [42] is a variation

Algorithm 2 PRM Algorithm

```

1:  $R_m \leftarrow \text{Roadmap}()$  ▷ Defines Roadmap
2: for  $i \leftarrow 0$  to  $N$  do
3:    $q_i \leftarrow \text{sample from } C_{\text{free}}$ 
4:    $R_m.\text{AddNode}(q_i)$ 
5: end for
6: for  $i \leftarrow 0$  to  $N$  do
7:    $\mathcal{N}(q_i) \leftarrow k$  closest neighbours of  $q_i$ 
8:   for each  $q \in \mathcal{N}(q_i)$  do
9:     if  $\text{IsCollisionFree}(q, q_i) \wedge \neg R_m.\text{EdgeExists}(q, q_i)$ 
       then
10:       $R_m.\text{AddEdge}(q, q_i)$ 
11:    end if
12:  end for
13: end for
14: return  $R_m$ 

```

of RRT that ensures asymptotic optimality, that is, as the number of samples increases, the solution converges to the optimal path. It achieves this by rewiring the tree every time a new node is added to optimize the cost of reaching the neighbor nodes around the newly added node to find the most cost-effective parent node.

BRRT [26] is the acronym for bidirectional RRT, which creates two search trees: one starting at the starting state and another starting at the goal state. During execution, the algorithm expands both trees in alternation and tries to merge the trees. This algorithm guarantees an exact solution, and in most environments, the trees find each other much faster than a single tree may find the goal state.

The PRM [39] method is similar to RRT [38] but focuses on building a roadmap representation of C_{free} before solving any problem. In using PRM, we assume that the environment is static, i.e., we do not expect the obstacle set to change over time. With the roadmap information, it is straightforward to connect a new start and goal node to the roadmap and search for the shortest path. We present the basic algorithm below. Algorithm 2 summarizes the PRM Roadmap (R_m) construction process.

This approach's limitations are similar to those in the grid-based approach. It requires an exponentially larger amount of memory to store the roadmap for higher dimensional problems. Additionally, this approach applies better to static environments.

I. CONSTRAINED SAMPLING-BASED MOTION PLANNING

Due to their modularity, sampling-based motion planners are good at handling the constrained motion planning problem. It is possible to integrate constraint functions without altering the core mechanics. Constrained sampling-based motion planners build upon the definitions of sampling-based motion planners. However, the constraint function adds more complexity to the problem because the motion planner must satisfy additional constraints to validate a sampled state. Furthermore, the constraint function defines a constraint

manifold that is not known *a priori*. Since the constraint manifold is a manifold within the C -space, the probability of a sample in C -space falling in the constraint manifold is zero [27], [28].

There are strategies to handle constraints in the sampling-based motion planning framework: relaxation, projection, tangent space, atlas, and reparametrization.

The relaxation method relaxes the constraint function equality from $F(q) = 0$ to $\|F(q)\| < \varepsilon$ [13]. With this, standard sampling-based motion planners may be able to handle the constraints.

The projection method defines a project operator that projects a sample in general C -space into the closest point of the constraint manifold [13], [27], [28].

Tangent space and Atlas are related. The tangent space [43] is generated from a known satisfying configuration by finding the basis for the constraint function's nullspace derivative. The motion planner may generate feasible configurations and motions in this tangent space. The AtlasRRT method [13], [44] combines the tangent spaces into a piece-wise linear approximation of the constraint manifold. The motion planner sample directly from this constraint manifold approximation.

Finally, the reparametrization method aims to find a new parametrization of the robot configuration allowing direct sampling and motion in this reparametrized space. The configurations in the reparametrized space may be mapped back to the original space.

J. APPLICATIONS

While the definition of motion planning is simple—finding a trajectory from a starting configuration to a goal configuration within a set of constraints—the problem becomes increasingly complex due to the wide range of applications and unique requirements in robotics. Each application often involves distinct constraints, embodiments, and customizations, making motion planning both a foundational yet highly adaptable challenge across various robotics domains.

The simplest application is to execute movements with a robotic arm [1]. More complex applications involve controlling the tool center point (TCP) to grasp and transport objects, such as in pick-and-place and reorganization tasks [1], [27], [28]. The robotic arm consists of multiple joints and a TCP or end effector [11]. The robotic arm moves in 3D space, if we consider the TCP motion or move in the joint space, if we consider the robot joint angle motion, both are valid for motion planning problem states [11].

Another type of application is vehicles, which move in 2D space freely or are constrained by the kinematics of their wheels. Autonomous vehicles also fall within this category, as they must follow high-level traffic rules while using the motion planning algorithm for controlling the low-level motions of the car [11].

Similarly, there are flying drone robots that move freely in the air, such as quadcopters, or more constrained motions,

such as winged airplanes. Also, there are four-legged and humanoid robots, similar to the robotic arm, where there is a set of joints and want to plan a trajectory of motions. The applications are diverse, but a motion planning algorithm performs the underlying low-level control. Thus, in this survey, we consider works with any application as long as the approach relates to solving the motion planning problem: sampling, collision checking, trajectory optimization, and many others.

K. METRICS

The most common metrics to evaluate motion planning algorithms are planning time, path length/cost, and success rate [1], [15]. In the motion planning context, time refers to the time the planner takes to find a solution. Path length or path cost refers to the size of the path in terms of the waypoint or distance the robot needs to move. The success rate refers to the ratio between the test motion planning problems and the number of problems the planner could solve. With these three metrics, it is possible to provide a satisfactory judgment on the performance of a planner.

Additionally, some works that consider different metrics, such as minimum clearance, and the minimum distance the trajectory passes from an environmental obstacle [45], for the works focusing on collision checking. Other works may have specific metrics that make comparing difficult, but the standard metrics are prevalent.

III. RELATED WORKS

In this section, we review recent surveys covering advancements in neural motion planning to justify and highlight this review's contributions compared to the others.

A survey paper aims to classify and review the existing research corpus in an area of knowledge [46]. Additionally, surveys are a common tool for beginners in any field to understand the current state of the art and even the history of research in that area [46]. Therefore, we compare existing surveys based on how they present information and the depth of their analysis, recognizing that both shallow and deep approaches have value depending on the scope and focus of each work.

The work of Wang et al. [22] analyzes data-driven methods for motion planning. The authors define a taxonomy based on the SBMP pipeline in four distinct steps: pre-processing, prediction, execution, and collision-checking. They also consider end-to-end algorithms that can completely replace the steps in an SBMP. Their survey introduces basic concepts such as the motion planning problem and the reinforcement learning (RL) theory and reviews the recent works in the field according to their taxonomy. Their taxonomy does not cover the full range of approaches to solving motion planning, instead, it focuses on SBMP. Additionally, they present the recent works in the field but do not draw comparisons between works and present deeper discussions based on presented works.

In comparison, we introduce fundamental concepts in our survey and improve on previous survey shortcomings. We also conduct a deeper analysis of each work and make connections between them.

Noroozi et al. [24] take a different approach, where they introduce existing motion planning methods and analyze real-world applications for robot motion planning, from self-driving cars to humanoid robots. Additionally, the authors perform a statistical analysis of the number of publications in this area throughout the years. The statistical analysis shows an increasing trend in the publication of neural-based approaches. This work focuses on integrating robots in real-world scenarios and discusses this topic. The motion planning algorithm review supports the analysis and discussion about real-world integration.

In contrast, we do not aim to make such analyses or predictions on the real-world integration of robots in our survey. Instead, we provide a panorama of current methods and explain why the current literature moves toward specific trends, such as deep learning.

Zhou et al. [47] review papers in two major categories: traditional algorithms and ML-based algorithms where the traditional algorithm group includes graph search, sampling-based and interpolating curve planning, and the ML-based includes supervised learning and RL approaches. They provide a detailed analysis of both categories, explaining each method thoroughly. Additionally, they explore different autonomous robot architectures, such as sense-plan-act and reactive frameworks. To compare methods, they use both category-specific and general analytical criteria, evaluating factors like path length and speed. Their work focuses on the fundamental principles of motion planning approaches and their application contexts.

Conversely, our survey focuses on specific approaches using the base methods and introduces each fundamental approach to understand each work better.

The work by Macmahon et al. [23] is similar to the work by Wang et al. [22], but they focus only on the integration of ML-based algorithms to the SBMP framework. Since this work has a narrow focus on SBMP, they go deeper into the existing approaches in this planning framework. The taxonomy they adopted is related to how the approaches integrate ML in the SBMP framework, such as learning sampling, learning collision checking, and learning steering. The authors explore all possible modifications to the SBMP framework to include ML-based algorithms and what are their implications. They compare the works regarding path quality, computational efficiency, and ease of use. Finally, they discuss the limitations of existing approaches and future directions for research.

The Survey by Macmahon et al. [23] focuses on a specific framework to solve the motion planning problem and perform a detailed analysis of the literature in this scope. On the other hand, our survey has a broader scope, not only in terms of the types of motion planning problems we consider but also in terms of possible solution methods.

We investigate other survey works in the literature beyond the ones we discuss here. Despite including motion planning, these works focused on a single application class, such as autonomous vehicles [48], [49] or mobile robots [50], [51]. We decided not to discuss these because the scope of our survey is broader in comparison.

Considering the variety of approaches to solving the motion planning problem we present in Section II, the literature lacks a survey that performs a broader review that includes different approaches. Thus, our survey aims to cover a more comprehensive range of AI-based approaches to solving the motion planning problem. Furthermore, given the diversity of possible approaches, to understand each survey work, the reader must have background knowledge of each topic, and existing surveys do not cover this knowledge, which is essential to contextualizing and understanding the importance of each surveyed work contribution.

Therefore, our survey provides the necessary background knowledge for the reader to understand the works we present and discuss. Regarding application scenarios, our survey has no precise scope for the type of application, which might be manipulation or locomotion. Finally, regarding the motion planning problem, we review works ranging from path planning to task and motion planning. Still, we emphasize the path planning works because they are the majority.

IV. THEORETICAL BACKGROUND

The goal of this section is to provide the definitions and background understanding of the base methods the learn-based motion planning methods use at their core so the reader can focus on the novelty of the approach. We do not aim to fully explain all the topics in detail and with mathematical definitions, instead, we aim to provide an introductory explanation of each topic and provide references and recommendations for readers interested in going deeper into a topic. Additionally, in Figure 4 we present a concept map of this section drawing relationships between topics and adding examples.

A. FULLY CONNECTED NEURAL NETWORKS

Neural networks are not a recent topic, research in artificial neural networks comes from the 50s in the seminal paper [52]. However, at that time, the computational resources and data were insufficient to train a neural network effectively. Therefore, applications favored other Machine Learning (ML) methods to the detriment of neural networks.

In the past decades, due to an increase in computational resources and data availability, researchers started using neural networks again to solve problems that heuristical methods could not solve [53]. Understanding the architecture is key to grasping how neural networks learn and why they excel at recognizing patterns compared to other methods. The literature defines neural networks as a direct acyclic graph with an input layer for signals, one or multiple hidden layers (layers between input and output) that make

linear combinations, and an output layer that outputs another signal [54].

With more data and computational resources, we could scale the number of hidden layers to have more parameters to tune and consequently learn linear and non-linear patterns. Deep learning comes from neural networks with several hidden layers, i.e., deep neural networks. The theory behind how neural networks effectively learn is still a research topic [54]. However, the intuition is simple: the hidden layers process weighted combinations of the inputs to generate the output. The training process of the neural network consists of tuning weights and other parameters for each single neuron unit (each neural network layer contains several neurons). A simplified training step is as follows [54]:

- 1) Forward pass: Input the samples to the network and calculate the outputs;
- 2) Loss computation: Use the loss function to compute the error between the actual and expected outputs;
- 3) Backward pass (backpropagation) and weight update: Compute gradients and update the weights;

During training, the algorithm repeats steps 1 to 3 until the error is below a certain threshold.

Neural networks might be interpreted as universal approximators, that is, they can approximate any function given that the data correctly represents it. This approximation occurs because each neuron represents a simple mathematical function, and when the neurons get combined on each layer, they might represent more complex mathematical functions. Therefore, within the space that the data covers, neural networks can generate accurate approximations, but as we go farther from this space, the approximations become less accurate [54].

B. RECURRENT NEURAL NETWORKS

Neural networks can learn functions for regression and classification problems for single outputs and sequences. Another type of neural network that processes sequences is called recurrent neural networks (RNNs). RNNs are neural network architectures that process data sequences by unrolling them in time. Each element enters the network at a time step, and then the sequence is processed one element at a time. The limitation of RNNs is that they only have the context of the current time step and the previous time step, losing the long-term context. The long-short-term memory (LSTM) neuron attempts to overcome traditional RNN limitations. LSTM is a new type of neuron that learns how much it should learn and when to forget the context [54].

C. CONVOLUTIONAL NEURAL NETWORKS

Another approach that revolutionized computer vision applications is the Convolutional Neural Networks (CNN). This type of architecture emulates the visual cortex, which hierarchically constructs the meaning of images by combining low-level features (edges, corners) into more complex concepts (faces, shapes). CNNs enabled researchers to achieve

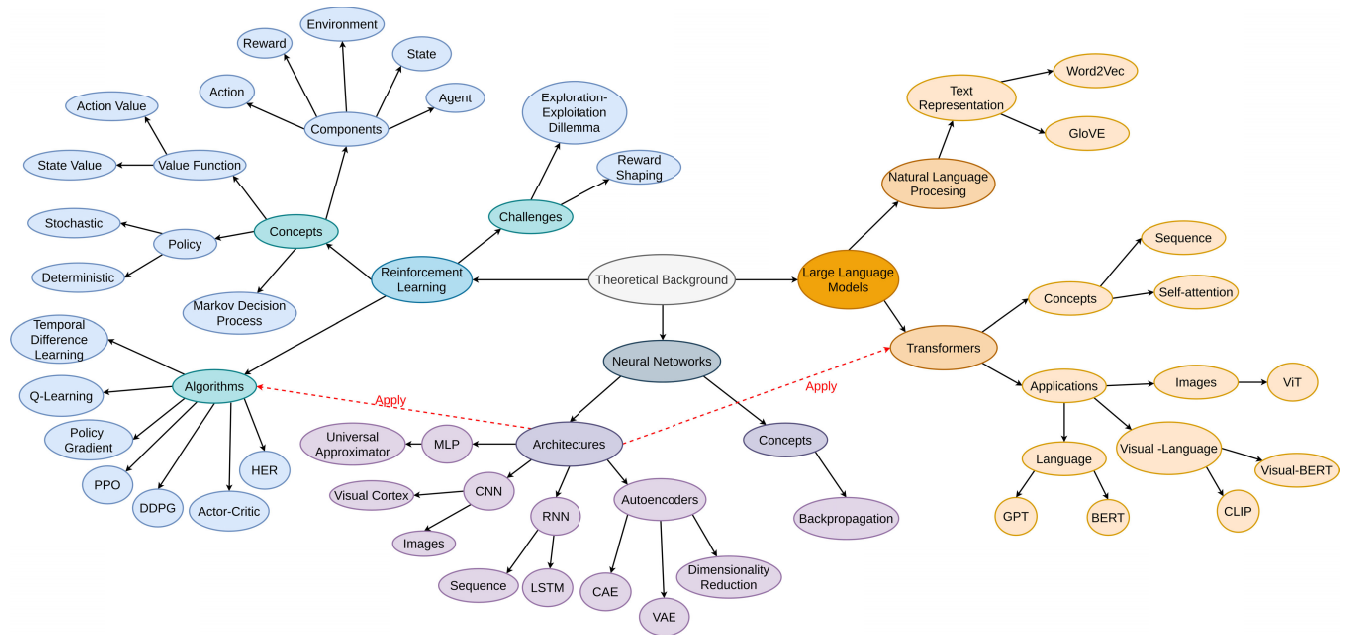


FIGURE 4. A concept map depicting the interconnections between large language models, neural networks, and reinforcement learning. While the map is divided into these three main areas, it highlights the role of neural network architectures in both language models and reinforcement learning through additional connecting relationships in red.

unprecedented accuracy rates in computer vision applications such as image classification, object detection, and object segmentation [55], [56]. The core idea of convolutional neural networks is that instead of learning a single weight for each neuron, the neurons are convolution kernels. During training, the elements of these convolutional kernels change. Each kernel specializes in extracting one type of feature from the image, such as edges and corners. Adding more layers, we combine these basic concepts into more concrete ones, such as faces, shapes, and objects. There are many techniques in convolutional neural networks beyond the convolution operation. Notable mentions are AlexNet [55], ResNet [56], Xception [57], and EfficientNet [58].

D. AUTOENCODERS

Auto-encoders (AE) are neural networks that aim to learn an identity function (f_θ) capable of reconstructing input using supervised learning [59]. The main goal of an AE is to represent the input in a lower dimensional space (using g_ϕ), which we call latent space (Z). The process is similar to PCA [60]. The key idea is that by bottle-necking the input using neural networks, it extracts the most significant features in a smaller dimensional space, and by generating the same input, it calculates an error and updates the weights. Different approaches extend this idea for 2D image data with convolutional auto-encoders (CAE) to improve the representation capabilities of the latent space using a probability distribution with the Variational-auto-encoder (VAE) [61]. The AE is a crucial concept for early motion planning methods to generate representations of the environment and

use these representations to plan a trajectory [1], [15]. Figure 5 illustrates an autoencoder architecture.

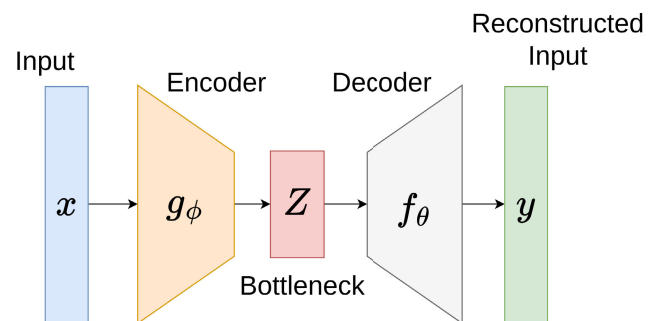


FIGURE 5. Simple Autoencoder diagram. Here, the Z is the latent space, a lower-dimensional representation of the input x obtained using the function g_ϕ , and the reconstructed output y is obtained using the identity function f_θ .

E. TRANSFORMERS

To understand transformers, we must introduce the concept of attention in the context of machine learning. Attention is a mechanism that was developed to tackle an existing problem in neural machine translation (NMT), where existing models had problems remembering long sequences, the same goal as LSTM.

In this context, attention helps the model to create a vector of importance weights on the input, which connects the context vector (has global information on the input sequence) with the original input sequence, making it possible to remember the entire input sequence. Attention

measures the relationship between elements in the sequence. Thus, it encourages the model to learn more effectively by understanding where to focus in the sequence, which contrasts to RNN and LSTM where all the sequence is processed in the same manner [62].

Another important concept is self-attention. Self-attention means applying the attention mechanism in element pair in the input sequence, which represents such sequences in terms of their relationships and context. The foundation of the transformer model is the self-attention mechanism [63].

Transformer models enable sequence processing without recurrent units. The core component of the transformer architecture is the multi-head self-attention block, which applies self-attention to different parts of the input sequence and concatenates all the representations as output. The multi-head self-attention allows the model to create an ensemble of representations with different contexts and allows a richer input representation. The encoder and decoder use the same multi-head self-attention to generate representations and retrieve from representations, respectively. Since transformers process the input in parallel, they do not have the sequence information. Therefore, the transformer architecture has a positional encoding to preserve the input sequence information [63].

The transformer architecture was first applied to solve a problem in NMT, but other researchers applied the same idea to different areas after its creation. The Vision transformer (ViT) [64] is a notable extension of the transformer idea. The ViT Method divides the image into fixed-size patches, and similarly to word processing, it embeds each patch. It combines it with positional encoding to feed to a transformer encoder. In this context, the self-attention allows the model to focus on different parts of the image and capture complex spatial relationships. This approach is present in image classification, object detection, and object segmentation tasks [64].

F. DIFFUSION

Diffusion models are similar to AE in that they encode the input data into a latent space. The difference between diffusion models and AE (and other generative models) is that it defines a Markov chain of diffusion steps, i.e., adding noise to the input, and the reconstruction part learns the inverse transform to recover the data. The advantage of using diffusion compared to previous approaches is its representation capability, given that the latent space is the same size as the input, and there is no compression. However, generating samples using diffusion is slower than other models because it requires several steps [65], [66].

G. REINFORCEMENT LEARNING

In this section, we follow the taxonomy and definitions of Sutton and Barto book [67]. Reinforcement Learning (RL) is a machine learning framework that models the learning problem as a problem of learning a behavior based on stimuli

obtained from the external environment, which mimics how humans learn new concepts, using trial and error [67]. The agent interacts with an environment and upon interaction it observes the result, which might be positive or negative, based on the result, the agent adjusts its actions to maximize the positive results.

A formal definition follows: the agent acts in an environment. The result of the agent actions in the environment is defined by a model, which the agent may or may not know. Every time the agent takes an action ($a \in \mathcal{A}$) it moves from one state to another, and the agent stays in one state at the time ($s \in \mathcal{S}$). When the agent performs an action a it moves from one state s to another state, which state is the next is governed by a transition probability function ($\mathcal{P}$). Additionally, after an action, the environment returns a reward ($r \in \mathcal{R}$) [67].

The agent uses the model to perceive the world, gaining information about the immediate result of its action, specifically the next possible states. If the agent knows the model and has perfect information on the environment, one might apply dynamic programming to find the optimal solution, or the agent learns the model by interacting with the environment. This approach is called **model-based reinforcement learning**. Conversely, in situations where the agent lacks information about the environment, it must operate without relying on a pre-existing model. This approach is referred to as **model-free reinforcement learning**. In this approach, the agent learns optimal behavior by trial and error, interacting directly with the environment and using feedback from its actions to improve performance over time.

Some problems are suitable for model-based methods, where constructing or understanding a complete model of the environment is feasible. However, in many real-world scenarios, the environment is too complex or unpredictable for a comprehensive model to be created or known, necessitating a model-free approach. Each method has its strengths and is chosen based on the nature of the problem and the available information about the environment.

When the agent interacts with the environment, the goal is to learn the best actions to be executed in each state to optimize the rewards given by the environment. The structure that provides the guidance on the agents actions is called the **policy** ($\pi(s)$). In this context, each state has a value function ($V(s)$) that models the expected rewards the agent gets if it starts in the current state and uses the current policy to guide its actions. The value function measures how good the state is and the agent tries to learn the value function and policy. Therefore, there are two new approaches: **on-policy** and **off-policy**.

On-policy learning refers to methods where the agent learns from actions sampled from its current policy. The agent must follow the same policy for both collecting experiences and updating the policy. Any changes to the policy require new samples to be collected using the updated policy.

Off-policy learning refers to methods where the agent can learn from actions sampled from a different policy (behavior

policy) than the one being learned (target policy). This allows the agent to learn from stored experiences or from actions generated by any policy, including random policies or older versions of the current policy. In the following subsections, we provide a formal definition for Model, Policy, and Value Function.

1) MODEL

The model is the agent's representation of the environment. With the model, the agent knows how the environment would react to its actions. The model has two parts: the probability function and the reward function. The probability function describes the transition step. The transition step occurs when the agent is in a state s and then performs action a to go to the next state s' and obtain reward r , which is described by the tuple (s, a, s', r) [67]. This probability function states the probability of transition from state s to state s' by performing action a and getting reward r . Consequently, the reward function predicts the next reward triggered by taking an action a in a state s .

2) POLICY

The policy is a function that describes the agent behavior, which might be deterministic or stochastic. The deterministic policy is defined as $\pi(s) = a$, and the stochastic policy is defined as $\pi(a|s) = \mathbb{P}_\pi[\mathcal{A} = a|S = s]$.

3) VALUE FUNCTION

The value function $V(s)$ predicts how good a state is, which means how much future reward the agent might obtain starting from that state s taking action a under policy $\pi(s)$ [67]. The future reward is a total sum of discounted future rewards, and it is known as return G_t . The discount part of the future rewards comes from the idea that future rewards have greater uncertainty, future rewards do not provide any immediate benefits to the agent and it is also mathematically convenient in the sense that we do not need to track future steps forever. In this context, the discount factor is defined as $\gamma \in [0, 1]$.

With the return G_t we define the **state-value** $V_\pi(s) = \mathbb{E}_\pi[G_t|S_t = s]$ and **action-value** $Q_\pi(s, a) = \mathbb{E}_\pi[G_t|S_t = s, A_t = a]$. The state-value of a state s is the expected return when we are at this state in time $S_t = s$. Similarly, the action-value of an action is the expected return if we are at state s in time $(S_t = s)$ and execute action a ($A_t = a$).

We might also use the policy $\pi(a|s)$ and action-value $Q_\pi(s, a)$ to obtain the state value $V_\pi = \sum_{a \in \mathcal{A}} Q_\pi(s, a)\pi(a|s)$. Another important parameter is the **advantage** function $A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s)$, which encodes the difference between state-value and action-value [67].

The optimal value function and policy are the ones that achieve the maximum return [67].

$$V_*(s) = \max_{\pi} V_\pi(s), Q_*(s, a) = \max_{\pi} Q_\pi(s, a) \quad (1)$$

$$\pi_* = \arg \max_{\pi} V_\pi(s), \pi_* = \arg \max_{\pi} Q_\pi(s, a) \quad (2)$$

Additionally $Q_{\pi_*(s,a)} = Q_*(s, a)$ and $V_{\pi_*(s)} = V_*(s)$.

H. MARKOV DECISION PROCESS

The reinforcement learning problems are usually framed as a **Markov Decision Process** (MDP) [67]. The states in a MDP have the Markov property, which states that the future transitions only depend on current states and do not depend on the sequence of past states (history) [67]. The MDP is described by a tuple of five elements $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ which are described as:

- $\mathcal{S}$ is the set of possible states;
- $\mathcal{A}$ is the set of possible actions;
- $\mathcal{P}$ is the transition function;
- $\mathcal{R}$ is the reward function;
- γ is the discounting factor to obtain the return.

I. BELLMAN EQUATIONS

The Bellman equations are a set of equations that decompose the state-value $V(s) = \mathbb{E}[R_{t+1} + \gamma V(S_{t+1})|S_t = s]$ and action-value $Q(s, a) = \mathbb{E}[R_{t+1} + \mathbb{E}_{a \sim \pi} Q(S_{t+1}, a)|S_t = s, A_t = a]$ functions into the next reward and the sum of discounted future rewards by using the definition of the return function G_t . This set equations are a central part of the reinforcement learning agent training process [67].

1) BELLMAN EXPECTATION EQUATIONS

The bellman equations describe a recursive update process. These equations might be further expanded to be based on both state-value $V(s)$ and action-value $Q(s, a)$ equations by following policy π [67].

$$\begin{aligned} V_\pi(s) &= \sum_{a \in \mathcal{A}} \pi(a|s) Q_\pi(s, a) \\ Q_\pi(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_\pi(s') \\ V_\pi(s) &= \sum_{a \in \mathcal{A}} \pi(a|s) (R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_\pi(s')) \\ Q_\pi(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') Q_\pi(s', a') \quad (3) \end{aligned}$$

2) BELLMAN OPTIMALITY EQUATIONS

This set of equations are analogous to the expectation equations when one is interested in the optimal values and policy [67].

$$\begin{aligned} V_*(s) &= \max_{a \in \mathcal{A}} Q_*(s, a) \\ Q_*(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_*(s') \\ V_* &= \max_{a \in \mathcal{A}} (R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_*(s')) \\ Q_*(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \max_{a' \in \mathcal{A}} Q_*(s', a') \quad (4) \end{aligned}$$

If $P_{ss'}^a$ and $R(s, a)$ are known one can solve the MDP by directly applying the Bellman equations. However, in most

Algorithm 3 Q-Learning Algorithm

```

1: Initialize  $t = 0$ 
2: State  $\leftarrow S_0$ 
3:  $t \leftarrow 0$ 
4: for  $\text{dot} < \text{max\_steps}$ 
5:    $A_t \leftarrow \arg \max_{a \in A} A(S_t, a)$   $\triangleright$  action with  $\epsilon$ -greedy
     strategy
6:   Reward  $\leftarrow R_{t+1}$   $\triangleright$  Get reward for action
7:   State  $\leftarrow S_{t+1}$   $\triangleright$  Move to next state
8:    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) -$ 
      $Q(S_t, A_t))$   $\triangleright$  Update
9:    $t \leftarrow t + 1$ 
10: end for

```

cases that is not the case, so the agent must learn $P_{ss'}^a$ and $R(s, a)$.

J. REINFORCEMENT LEARNING METHODS

There are different ways to approach solving the MDP. The most fundamental ones are Dynamic programming (on-policy), and Monte Carlo (off-policy). For the eager reader, we recommend reading chapters 4 and 5 of [67], which cover dynamic programming and Monte Carlo methods in reinforcement learning, respectively. However, these basic methods cannot solve complex problems. Therefore in this subsection, we focus on exploring methods present in the works we survey in this document.

1) TEMPORAL DIFFERENCE LEARNING (TD)

Temporal difference (TD) learning is a model-free method. The key difference between TD learning and the previous methods is that TD learning learns from incomplete episodes. The means by how the TD learning accomplishes that is through **bootstrapping**. TD learning updates targets based on existing estimates rather than using the actual rewards and complete returns [67].

On TD learning the value function $V(S_t)$ is updated towards an estimated return $R_{t+1} + \gamma V(S_{t+1})$, called **TD Target** [67]. Additionally, it controls how much update the value function using a parameter called learning rate α .

2) Q-LEARNING

Q-Learning is the off-policy TD control. TD control is the method by which agents learn the optimal policy [68]. Below, we detail the steps the Q-Learning process follows in one step:

In simple environments, one can keep all the $Q_*(\cdot)$ for all state-action pairs, similar to a lookup table. However, as the state and action spaces grow larger, this becomes unfeasible in terms of computational memory. Therefore a solution to this limitation is instead of memorizing $Q_*(\cdot)$ for each state-action pair, it approximates using function approximation. Since neural networks are universal function approximators, it is natural to use neural networks to approximate $Q_*(\cdot)$, thus Q value becomes $Q(s, a; \theta)$ where θ is the function

parameter. However, the Q-learning method suffers from instability by combining non-linear function approximation with the **bootstrapping** [68].

The Deep Q-Network (DQN) [69] aims to improve the stability of Q-Learning training by using two new strategies: Experience replay and periodically update targets.

Experience replay consists of storing the episode steps $e_t = (S_t, A_t, R_t, S_{t+1})$ into a replay buffer D_t which contains experience over many steps and episodes. During the Q-learning updates (Algorithm 3 line 8), samples are drawn from this buffer randomly, so the agent reuses past experience when updating its Q-value. This strategy improves data efficiency, smooths over changes in the data distribution, and removes correlations in the observation sequences.

Q is updated towards target values that are periodically updated, this means that the Q-network is frozen as the optimization target every C step (C is another hyperparameter), which overcomes the short-term oscillations and stabilizes the training.

The loss function has a form:

$$\mathcal{L}(\theta) = \mathbb{E}(s, a, r, s') \sim U(D) [(r + \gamma \max_{a'} Q(s', a', \theta^-) - Q(s, a; \theta))^2] \quad (5)$$

where $D(U)$ is a uniform distribution over the replay buffer D_t and the θ^- is the frozen Q-network.

3) POLICY GRADIENT

The TD-learning method aims to learn the state and action values and then select actions accordingly [67]. The Policy gradient methods aim to learn the policy directly with a function $\pi(a|s; \theta)$ parametrized with θ . On Policy gradient methods we aim to maximize a reward function instead of minimizing a loss function. It computes the gradient numerically by perturbing θ by a small amount ε in the k -th dimension and is applicable either to $\mathcal{J}(\theta)$ when it is differentiable or not.

4) ACTOR-CRITIC

The actor-critic algorithm learns both value function and policy [67]. The critic updates the value function parameter w which could be action value $Q(a|s; w)$ or state value $V(s; w)$ (depending on the implementation). The actor updates the policy parameter θ , in the direction suggested by the critic, $\pi(a|s; \theta)$. In summary, the actor learns the policy that governs the actions of the agent and the critic evaluates if that policy is good or not, that is if the action or state values are good or not, and then one updates the other.

5) DEEP-DETERMINISTIC POLICY GRADIENT (DDPG)

Deep-deterministic policy gradient (DDPG) [70] is a combination of Deep policy gradient (DPG) [71] and DQN [69]. The DQN stabilizes the training by using an experience replay and by freezing the network during a fixed number of steps to avoid short-term oscillations. However, the DQN only works in discrete space. The DDPG extends the DQN

to continuous space and incorporates the actor-critic while learning a deterministic policy.

DDPG also creates an exploration policy $\mu'(s) = \mu_\theta(s) + \mathcal{N}$ is created by adding noise $\mathcal{N}$, to improve exploration. Additionally, DDPG does a soft update on the parameters of the actor and critic with $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$, this process is known as conservative policy iteration. In this way, the parameters of the networks change slowly, which contrasts with DQN which changes only during certain periods of time.

6) PROXIMAL POLICY OPTIMIZATION (PPO)

Proximal policy optimization (PPO) [72] is an off-policy algorithm that simplifies the trust region policy optimization (TRPO) [73] by using a clipped surrogate objective while retaining similar performance. TRPO reward $r(\theta)$ relies in the ratio between the two new policies $\pi_\theta(a|s)$, $\pi_{\theta_{\text{old}}}(a|s)$.

However, without limiting the distance between θ_{old} and θ , maximizing the objective function $J^{\text{TRPO}}(\theta)$ would lead to extremely large parameter updates and big policy ratios. PPO forces the $r(\theta)$ to stay within a small interval around 1: $[1 - \epsilon, 1 + \epsilon]$, where ϵ is a hyperparameter.

The function $\text{clip}(r(\theta), 1 - \epsilon, 1 + \epsilon)$ limits the ratio to be within $[1 - \epsilon, 1 + \epsilon]$. The PPO takes the minimum between the original value and the clipped value, making the agent to lose interest in increasing the policy updates to high values to get better rewards.

When using PPO in a network architecture that has shared parameters between actor and critic it is necessary to add (in addition to the clipping function) an error term on the value estimation ($c_1(V_\theta(s) - V_{\text{target}})^2$) and add an entropy term ($c_2 H(s, \pi_\theta(.))$) to encourage sufficient exploration, where c_1 and c_2 are two hyperparameter constants.

7) HINDSIGHT EXPERIENCE REPLAY (HER)

The Hindsight Experience Replay (HER) method aims to improve the sample efficiency on binary and sparse environments [74]. Binary and sparse environments occur when the agent only gets a non-negative reward it wins, which makes it difficult for the agent to explore and learn if there is only valid information if it wins, making the training take a really long time compared to other types of environments and with supervised learning. In this context, HER uses the same memory buffer explored in previous methods such as DDPG algorithm [70], but goes beyond by recalculating the reward using trajectories that could not achieve the goal. In this way, the agent gets valuable information in every episode, independent if it reaches the goal or not because HER changes the goal to be within the failed trajectory when it happens.

K. REINFORCEMENT LEARNING LIMITATIONS

The Reinforcement learning framework has a significant potential when compared to other machine learning frameworks in the sense that it does not require humans to collect data because the agent naturally does it by itself when interacting with the environment. Additionally, since the

agent interacts with the environment freely and its actions are not deterministic (in complex scenarios), we can observe behaviors that are not initially foreseen and might lead to more optimized behavior. However, RL has its shortcomings, so we discuss two of them: exploration-exploitation dilemma and reward function design [67].

The exploration and exploitation dilemma [67] occurs when an agent faces a new unknown scenario. Without enough exploration, the agent cannot learn the environment sufficiently, and without enough exploitation, the agent cannot complete the optimization of its reward task.

This leads to sample inefficiency, where the agent in reinforcement learning requires considerably more samples than a supervised learning approach. This difference exists because the agent needs to explore, while in supervised learning, the samples are already curated in advance.

In the context of sample efficiency, this problem becomes more prominent in sparse reward cases. In these situations, the agent receives no reward most of the time, only receiving a reward at the end of the episode signifying the success or failure of a task. This means that the agent gets no useful information and thus needs considerably more samples for training.

The ϵ -greedy approach is one way to balance exploration-exploitation. To improve the sample efficiency and address sparse reward shortcomings, authors employ a memory buffer and use the hindsight experience replay (HER) method [67], [74].

Another issue of reinforcement learning approaches is the reward function design (reward shaping). The reward function is what dictates when and how much reward the actor gets for its actions. It encourages the agent to solve a designated task. However, some problems are difficult to describe in reward functions and thus the researcher needs to experiment with different reward functions until it finds the one that fits the problem better. In contrast to simple hyperparameter optimization, the reward function experiments cannot be easily automated. There are approaches that do not use a reward such as imitation learning, but those approaches limit the generalization capabilities of the agent [67].

For readers who would like to go deeper into reinforcement learning basics, we recommend the Sutton and Barto [67] book that we use extensively in this subsection. The book is heavy in mathematical definitions but covers all the background necessary to understand modern reinforcement learning. Additionally, we recommend reading the Hindsight experience replay paper [74] to understand the current state of reinforcement learning.

L. LARGE LANGUAGE MODELS

Large Language Models (LLMs) are the latest Natural Language Processing (NLP) iteration. Therefore, to understand LLMs, we need to introduce NLP. NLP is a research field that aims to process natural language data, which includes text and voice. NLP includes applications such as text translation, sentiment analysis, and text summarization [75].

Early NLP systems encoded the linguistic rules as rule-based methods, which were very limited. In the 1990s, NLP research started using statistical and machine learning methods. Representation in a lower-dimensional space is essential to using these methods. Therefore, one-hot encoding and bag-of-words were popular methods to represent words and text. One-hot encoding represents the presence of words or letters using ones and absence using zeroes. Bag-of-words represents the text by the frequency of its words [75].

The problem with one-hot encoding and bag-of-words is that they generate sparse representations. To solve this issue, researchers proposed methods such as Word2Vec [76] and GloVe [77], which represent the words in dense vectors in continuous vector space. These vectors capture semantic relationships between words. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) Networks use this type of representation to model word sequences. One example is the Seq2Seq model used for machine translation, which allows for sequences of variable size [78].

Transformers and their attention mechanisms represent a major leap in NLP, forming the foundation for modern LLM architectures, we explored transformers architecture in detail in the Subsection IV-E. In Summary, transformers are capable of processing entire sentences simultaneously using self-attention mechanisms, allowing models to weigh and connect relevant words regardless of their position in the text [63].

The key innovation for large language models is combining transformers with extensive text data. The transformer architecture in the LLM models has a very large number of parameters (e.g., billions), and this quantity of parameters and extensive text data allow these models to demonstrate capabilities unseen in previous approaches, such as better machine translation and human-like text generation. Notable LLM models are Bidirectional Encoder representations from transformers (BERT) [79] and Generative Pre-Trained Transformers (GPT) [80]. BERT represents the words using bidirectional embeddings that consider both the left and right context. On the other hand, GPT uses a unidirectional representation, thus generating text by predicting the following word in the sequence. These models demonstrate the capabilities of generating human-like text and is able to perform tasks zero-shot or few-shot with no specific training.

Furthermore, incorporating LLMs and Image models such as CNNs and ViT gives rise to Visual Language Models (VLM), which combine NLP and computer vision fields. A pioneer in this approach is the VisualBERT [81], which extends the BERT architecture to process image and textual features. Another VLM is the Contrastive Language-Image Pre-training (CLIP) [82], which learns image and text relationships by training on a vast dataset of images and their captions. The VLMs are able to execute image captioning, give accurate descriptions of images, zero-shot image classification, and generate new images from text inputs [81], [82], [83].

V. LEARNING-BASED PLANNERS

In Section II, we explored the diversity in traditional motion planning and task and motion planning approaches. In this survey, for simplicity's sake, we do not adopt a rigid taxonomy to classify the works. Instead, we divide the works into larger groups, then explore each work and detail how it compares to the classic methods we explore in Section II. The papers reviewed in this section can be categorized into three main groups: neural planners, reinforcement learning (RL) planners, and large language model (LLM) planners.

Neural planners aim to enhance or replace traditional motion planning methods using data-driven neural networks. For instance, a neural network can substitute the sampling component of a sampling-based motion planner (SBMP) [7] or replace a collision detection module [45]. These approaches often integrate traditional methods with neural networks to improve efficiency or generalization. Tables 1, 2, and 3 summarizes the works on neural planners, detailing the AI methods used, the specific motion planning subproblems addressed, and their applications.

RL planners, on the other hand, leverage the RL framework to solve motion planning problems end-to-end, without relying on heuristics. Unlike neural planners, RL planners are tailored to specific environments or setups but excel in handling dynamics, such as dual-arm manipulation and dynamic obstacles. Tables 4, and 5 provides an overview of RL planner studies, organized similarly to Table 1.

The third and most recent category is **LLM planners**, which directly apply large language models to address motion planning and TAMP. This approach follows two primary directions: (1) using LLMs as task planners in combination with a low-level motion planner or pre-trained RL policies, or (2) fine-tuning LLMs with multimodal data such as images and videos to enable both high-level semantic planning and low-level joint control. Table 6 presents a summary of LLM planner works, using the same structure as Tables 1 and 4.

Inside each group, it is possible to find a variety of approaches and methods. For example, in the context of SBMPs, some works that replace steer function, sampling, and collision detection with data-driven neural networks. Another example is using an LLM as a high-level task planner and an RL policy to execute discrete actions. Another point is the variety of use cases and environments. There is a great variety of test scenarios, embodiment, and environments. It is ineffective to group work without a more precise point in common. Figures 6 and 7 give an example of the taxonomies we adopt in this work and examples of existing approaches.

Finally, Figure 8 presents a concept map for this section containing a summary of the applications, methods, and advantages of each type of planner we discuss in this section.

A. NEURAL PLANNERS

Neural planners are a novel approach to solving motion planning algorithms. This approach leverages the neural networks learning capabilities to learn from an expert

TABLE 1. Summary of neural planner works (Part 1).

Work	Motion Planning Problem	AI Method	Application	Dataset	Advantages	Limitations
[1], [7]	Path planning	Contrastive AE, MLP	2D robot, 3D robot, 7 DOF robot	New Dataset	- High-speed planning; - Effective in high-dimensional spaces; - Probabilistic complete.	- Requires expert demonstration data; - Limited generalization.
[15]	Path planning	AE	2D robot, humanoid robot	New Dataset	Suitable for high-dimensional humanoid robot environments.	- Poor performance in simpler 2D environments compared to baselines; - No standard state representation.
[16]	Path Planning	Conditional VAE	2D robot, 12D robot	New Dataset	- Superior performance in complex, high-dimensional scenarios; - Probabilistic complete.	- Requires expert demonstration data; - Limited generalization to unseen environments; - No standard state representation.
[84]	Path Planning	MLP	2D robot, 3D robot	New Dataset	Improves success rate for PRM-like algorithms, in scenarios with narrow passages.	- Limited to multi-query planning; - Limited to static environments.
[85]	Path Planning	VAE	6 DOF robots	New Dataset	- Learns inverse kinematics effectively; - Handles collision detection in latent space.	- Marginal improvement in complex scenarios; - No completeness guarantees; - Limited obstacle shape.
[86]	Path Planning	PointNet++, GAN	7 DOF robots	New Dataset	- Scalable with large data; - Handles moving obstacles; - Higher accuracy than MP-Net [7].	- Extremely large data requirements; - No completeness guarantees.
[27]	Constrained Motion Planning	CAE, MLP	7 DOF robots, pick and place	New Dataset	- Handles constrained manifolds; - Supports task-specific encodings; - Probabilistic completeness ensured; - Faster execution time than baselines.	- Computational overhead for voxel-based representation; - Requires task-specific tuning; - Limited to known environments.
[37]	Kinodynamic Motion Planning	Transformer, AE, MLP	2D robots	New Dataset	- Effective for 2D non-holonomic scenarios; - Better success rate and speed than ROS navigation stack methods.	Limited to 2D environments.
[87]	Kinodynamic Motion Planning	Transformer	2D robots	New Dataset	- Works for different map sizes; - Handles kinodynamic and non-holonomic constraints well.	- Only works in 2D scenarios; - Focuses only on identifying regions for sampling; - Motion planner handles constraints; - Slight accuracy drop compared to baselines.
[88]	Kinodynamic Motion Planning	AE, GAN	2D robots, 3D robots	New Dataset	- Handles complex kinodynamic constraints; - Reduces computation time with neural discriminator.	- Higher path costs; - Limited evaluation for diverse robots/environments.
[28]	TAMP	AE, GAN	7 DOF robots, pick and place	New Dataset	- Handles constrained motion tasks effectively; - High success rates in real-world scenarios; - Faster execution time than baselines; - Probabilistic complete.	- Requires robust constraint integrators; - Limited to known environments.
[89]	Kinodynamic Motion Planning	MLP	2D robots	New Dataset	- Improves success rates by replacing RRT components; - Lowers path cost in multi-robot scenarios.	Requires expert data.
[35]	Path planning	VQ, Transformer	2D robots, 7 DOF robots, 14 DOF robots	New Dataset	- Accelerates sampling-based motion planning; - Reduces execution time significantly.	- Requires trajectory data to cover the entire planning space; - Limited environmental generalization.

TABLE 2. Summary of neural planner works (Part 2).

Work	Motion Planning Problem	AI Method	Application	Dataset	Advantages	Limitations
[36]	Constrained motion planning	VQ, Transformer	7 DOF robots, pick and place	New Dataset	- Fast sampling for constrained motion planning; - Better generalization to new constraints; - Faster execution times than baselines.	- Limited scalability for unseen environments; - Requires high-quality feature extraction.
[90]	Path Planning	Conditional AE	7 DOF robots, pick and place	New Dataset	- Adaptive sampling strategy; - Handles dynamic environments.	- Limited voxel grid resolution; - Slower than point cloud methods.
[91]	Path Planning	MLP	7 DOF robots	New Dataset	- Uses diffeomorphisms for informed sampling; - Faster execution time and success rate than baselines.	Limited scalability due to occupancy grid representation.
[92]	Path Planning	VAE, PointNet [93], MLP	7 DOF robots	New Dataset	- Handles out-of-distribution scenarios; - Uncertainty-aware sampling.	Improvements are minimal compared to baselines.
[94]	Kinodynamic Motion Planning	MLP	Autonomous underwater vehicle	New Dataset	- Replaces steer function; - Handles vehicle dynamics; - Faster planning times compared to baselines.	- Specialized to specific robot types; - Requires expert data.
[95]	Path Planning	3DCNN, CVAE	Mobile manipulator	New Dataset	- Combines global and local planning; - Works in cluttered environments; - Zero shot sim-to-real transfer; - Higher success rates and faster execution times compared to baselines.	Does not consider very large changes in environments.
[96]	Kinodynamic Motion Planning	AE, MLP	2D robots	New Dataset	- Specialized for kinodynamic planning; - Faster than baselines.	Path cost worse than baselines.
[97]	Path Planning	AE, MLP	2 DOF robots, Dynamic obstacles	New Dataset	- Handles dynamic environments; - Faster planning time.	- Limited to simple environments; - Fixed time interval context.
[98]	Path Planning	Physics-informed neural network	6 DOF robots	New Dataset	- Fast data generation; - Handles complex shapes; - Faster planning times compared to baselines.	- Limited scalability; - Requires retraining for new environments; - Higher path length compared to MPNet [7].
[99]	Path Planning	Physics-informed neural network	6 DOF robots	New Dataset	- Improves upon NTFields in complex scenarios; - Can learn multiple environments on the same model.	Cannot generalize to unseen environments.
[100]	Constrained motion planning	Physics-informed neural network	6 DOF robots	New Dataset	- Fast data generation; - Handles constrained planning; - No need for expert data.	- Limited to specific constraints; - Requires careful design; - Worse performance than baseline (COMPNetX [28]).
[101]	Path Planning	Diffusion	2D robots, 7 DOF robots	New Dataset	- Generates multiple solutions; - Handles complex environments.	- Requires retraining for new environments; - Slow inference.
[102]	Path Planning	Neural Ordinary Differential Equations (NODE)	2D robots, 3D robots, 7 DOF robots	New Dataset	- Models motion as vector fields; - Adapts to perturbations.	- Limited generalization; - Requires retraining for new tasks.

and generalize to similar unseen problems. In the motion planning application, these networks can replace traditional sampling, collision checking, and other components of

the traditional sampling motion planners. However, neural networks are universal approximators, i.e., they do not provide exact solutions, so it is necessary to add

TABLE 3. Summary of neural planner works (Part 3).

Work	Motion Planning Problem	AI Method	Application	Dataset	Advantages	Limitations
[58]	Path Planning	MLP, Point-Net [93]	7 DOF robots	New Dataset	- Combines CBF with RRT; - Handles safety constraints; - Better success rate and path length than baselines.	Slightly longer planning time than RRT.
[103]	Path Planning	MLP	2D robots	New Dataset	- Handles multi-agent planning; - Identifies critical regions.	Limited to static environments.
[104]	Constrained Motion planning	MLP	6 DOF robots	New Dataset	Learns planning function on constraint manifold.	- Limited to local planning; - Requires retraining for new tasks.
[105]	Path Planning	CVAE, RNN	Scara robots	New Dataset	- Outputs high-level motion commands; - Reduces motion latency.	- High Latency due to command parsing; - Does not support dynamic obstacles.
[106]	Path Planning	MLP	8 DOF robots, collision checking	New Dataset	- Models sensor uncertainties; - Reduces collision risk.	- High planning time; - Limited to specific scenarios.
[107]	Constrained Motion planning	Neural Fields	2D robots	New Dataset	- Handles thin and U-shaped obstacles; - Smooth trajectories.	- Slow execution time; - Longer path lengths than baselines.
[108]	Path Planning	MLP, HSMM	7 DOF robots, pick and place	New Dataset	- Incorporates motion primitives; - Avoids obstacles.	Limited to a specific tasks.
[109]	Path Planning	Transformer	Autonomous vehicle	New Dataset	Combines scene prediction and motion planning.	- Slow optimization; - Unfeasible for real-time applications; - Assumes perfect environment information.
[110]	Path Planning	VAE	7 DOF robots, Collision checking, dynamic obstacles	New Dataset	- Finds multiple solutions; - Faster than baselines.	- Requires retraining; - Not reusable for other problems.
[45]	path planning	MLP	7, 11, 14, 30 DOF robots, Collision checking	New Dataset	- GPU-accelerated collision checking; - Handles different robot embodiments.	- Collision check is not 100% accurate; - Limited generalization to new environments; - Does not support dynamic obstacles; - High data requirements.
[111]	Path Planning	MLP	7 DOF robots, Collision checking, dynamic obstacles	New Dataset	- Fast collision checking; - Handles dynamic environments.	- No environment representation; - Retrain for each new environment.
[112]	Path Planning	CVAE	2D robots, Collision checking, Dynamic obstacles	New Dataset	- Handles dynamic obstacles; - Smooth path deviation.	- Assumes precise obstacle information; - Limited to mobile robots.

methods to guarantee completeness and exact solutions where necessary.

One of the seminal works for neural planning is Qureshi et al. motion planning networks (MPNet) [1], [7]. The MPNet architecture has two main components: the contrastive autoencoder (contrastive-AE) and the sampler network. Figure 9 presents a simplified diagram of the architecture, where Enc_{obs} is the environment encoder and the $Net_{planner}$ is the planning network. The Enc_{obs} learns a latent representation (Z) of the environment as a point cloud (x_{obs}). The sampler network ($Net_{planner}$) accepts the latent representation of the environment and the start (x_t) and goal (x_T) states and outputs

the next state (x_{t+1}). Therefore, by executing this process interactively, one expects to reach the goal state or sufficiently close to it.

However, the neural network methods are not exact. They define support methods to satisfy completeness and make the method exact: bidirectional planning, replanning, lazy states contraction, steering, and validity check. Bidirectional planning makes the network reach exactly the goal state from the start state since there are two trees: one from start to goal and another from goal to start. Replanning fixes the path if there are any discontinuities after the trees are connected, i.e., if the planning cycle is complete, but the path

TABLE 4. Summary of RL planner works (Part 1).

Work	Motion Planning Problem	AI Method	Application	Dataset	Advantages	Limitations
[113]	Path planning	SAC, Point-Net	Collision checking	New Dataset	• Learns collision avoidance effectively; • Integrates PointNet for spatial understanding.	• Dependent on the rigid body geometry.
[114]	Path Planning	PPO	Residual RL	New Dataset	• Demonstrates end-to-end learning capability; • Handles dynamic environments.	• Limited scalability to unseen scenarios.
[115]	Path Planning	PPO	Teleoperation	New Dataset	• Enhances teleoperation tasks with RL; • Robust under noisy conditions.	• Requires high-quality teleoperation data; • Limited to specific robot setup; • Worse performance than baselines.
[5]	Path Planning	DDPG	Manipulation	New Dataset	• Efficient episode memory management; • Improved training speed and reward.	• Fixed starting point; • Limited to a specific robot joint configuration; • No obstacles in the environment.
[17]	Path Planning	DDPG	Dual-arm, manipulation	New Dataset	• Solves dual-arm coordination problems; • End-to-end learning for complex setups; • Better results than baselines.	• Fixed start and goal; • Task specific solution; • Limited generalization.
[18]	Path Planning	PPO (Curriculum learning)	Manipulation	New dataset	• Curriculum learning improves performance.	• Limited to specific tasks; • Limited to specific robot and environment configuration.
[116]	Path Planning	PRO	Teleoperation, manipulation	New Dataset	• Efficient for teleoperation with low-latency response.	• Limited performance in highly dynamic environments; • Model specific to task setting.
[117]	Path Planning	PPO	Multi-agent, manipulation, pick and place	New Dataset	• Faster training and higher rewards than baselines.	• Limited robustness to unseen agents; • Simple experiment setting.
[4]	Path Planning	PPO, Actor-critic	Manipulation	New dataset	• Virtual sensors; • Better performance than baselines.	• Only works in simulation assumes perfect sensors; • Uses a high number of sensors.
[118]	Path Planning	DDPG	Dual-arm, manipulation, pick and place	New dataset	• Handles dual-arm coordination; • Real-world application; • Multi-agent; • Good sim-to-real transfer.	• Limited to a specific task.
[119]	Path Planning	SAC	Dual-arm	New dataset	• Handles dual-arm collision avoidance; • Progressive training.	• Limited to a specific task.
[120]	Path Planning	SAC, HER	Dual-arm	New Dataset	• Learns robust dual-arm coordination; • Integrates hindsight experience replay for better training; • Better metrics than baselines.	• Fixed obstacles.
[121]	Path Planning	SAC, HER	Dual-arm, dynamic obstacles	New dataset	• Handles dynamic obstacles; • Better than baselines.	• Worse results than static obstacles.
[122]	Path Planning	SAC, HER, LSTM	Dual-arm, dynamic obstacles	New Dataset	• Handles dynamic obstacles effectively; • Integrates temporal learning with LSTM.	• Sensitive to obstacle dynamics changes; • Limited long horizon obstacle motion prediction.
[123]	Path Planning	Q-Learning	Manipulation	New dataset	• Handles obstacle avoidance; • Assisted learning speeds up training.	• Does not consider robot kinematics.

TABLE 5. Summary of RL planner works (Part 2).

Work	Motion Planning Problem	AI Method	Application	Dataset	Advantages	Limitations
[124]	Path Planning	β -VAE, TD3	Manipulation, pick and place	New dataset	- General-purpose skills; - Fine-tuned β-VAE improves representation; - Ablation study with different methods.	Limited to a specific task.
[125]	Path Planning	GNN, Neural Fields, DAG-GER	Manipulation, Dynamic obstacles	New Dataset	- Combines GNN for spatial reasoning; - Handles dynamic environments.	- Worse success rate and path length than baselines in complex scenarios; - Handles only periodic dynamic and known obstacles.
[126]	Path Planning	SAC, ResNet, LSTM	Object navigation, locomotion	Gibson Environments	- Excels in object navigation tasks; - Handles diverse locomotion challenges; - Interesting emergent behaviors; - Better than baselines.	Limited to 2D scenarios.
[127]	Path Planning	SAC, DDPG	Object navigation, locomotion	New dataset	- Handles dynamic environments; - Faster convergence; - Ablation study.	Limited to simulation.
[128]	Path Planning	Q-Learning, PPO	No application	New dataset	Combines RL with deterministic planning.	- Limited to 2D planning; - Not scalable to high DOF robots and complex environments; - Faster but worse performances than baselines.
[129]	Kinodynamic Motion Planning	Q-Learning	Autonomous vehicle	New Dataset	- Effective in vehicle motion planning; - Simplifies problem representation; - Handles uncertainty.	Limited to 2D scenarios.
[130]	Kinodynamic Motion Planning	SAC	Autonomous vehicle	New dataset	- Handles dynamic obstacles; - Better cost and completion time than baselines.	Limited to 2D environments.
[6]	Path Planning	Q-Learning	Manipulation, robotic arm	New dataset	- Learns from human demonstrations; - Adaptive planning.	- Limited generalization; - Library grows exponentially.
[47]	Kinodynamic Motion Planning	SAC, LSTM	Indoor service robots, dynamic obstacles	New dataset	- Handles dynamic environments; - Improved data replay.	- Limited to 2D environments; - Assumes perfect information of obstacles.
[131]	Kinodynamic Motion Planning	SAC	Redundant space robots, dynamic obstacles	New dataset	- Handles null-space motion; - Generalizes to multiple obstacles; - Better performance than baselines.	- Limited to 2D plane; - Assumes perfect sensing.
[132]	Path Planning	SAC, Point-Net [93]	Manipulation, robotic arm, dynamic obstacles	New dataset	- Combines APF with RL; - Handles dynamic obstacles; - sim-to-real transfer.	Lower success rate than baselines.
[133]	Path Planning	SAC, DDPG, BC, SILP, HER	Manipulation, collision detection	New dataset	- Improves data efficiency; - Reduces training time; - sim-to-real transfer.	Limited to specific tasks.
[134]	Path Planning	Hierarchical RL, Decision Transformers	2D, 6 DOF manipulator, humanoid robots	New Dataset	- Combines hierarchical RL for better task decomposition; - Works across various robots; - Better success rate than baselines.	Limited scalability.

is not continuous. The lazy state contraction is a method for smoothing the path by removing redundant and unnecessary states. The steering function tries to connect two states using

a discretized straight line until it finds an unfeasible state. For the context of our survey, the validity check consists of checking for collisions and joint limits. These supporting

TABLE 6. Summary of LLM planner works.

Work	Motion Planning Problem	AI Method	Application	Dataset	Advantages	Limitations
[135]	Task Planning	GPT-3	Manipulation, pick and place	New Dataset	• Leverages GPT-3 for task-level reasoning; • Effective in handling semantic-level tasks; • Closed-loop feedback.	• Limited low-level control capabilities; • Requires low-level planner.
[136]	Task and Motion Planning	Llama-7B, Llama-13B, GPT-3.5	Self-driving car	New Dataset	• Supports complex vehicle navigation tasks; • Utilizes multimodal data for planning.	• Requires low-level planner.
[137]	Task Planning	GPT-3	Self-driving car	nuPlan	• Combination of rule-based and LLM; • Improves rule-based algorithm output.	• Limited to specific rule-based scenarios; • High dependency on prompt engineering.
[138]	Task Planning	Not disclosed	Manipulation, pick and place	New Dataset	• Integrate high-level planning with low-level skill library.	• Requires pre-trained low-level skill.
[139]	Task Planning	GPT-4-turbo-preview	Quadruped robot	New Dataset	• Use three LLMs for task planning, parameters, and code generation.	• Requires pre-trained low-level skill.
[140]	Task Planning	InstructGPT, CLIPort	Manipulation, pick and place, locomotion	New Dataset	• Integrates vision-language models for multimodal tasks.	• Requires pre-trained low-level skill; • Limited generalization.
[141]	Task Planning	GPT-3, LaMDA, FLAN	Manipulation, pick and place, locomotion	New Dataset	• Combines LLMs with grounded action execution.	• Requires pre-trained low-level skill; • Limited generalization.
[21]	Task and Motion Planning	LiaVA	Self-driving car	New Dataset	• Integrates vision and language for planning; • Supports detailed task decomposition.	• Limited to simulation.
[19]	Task and Motion Planning	RT-1	Manipulation, pick and place, locomotion	New Dataset	• Combines task and motion planning seamlessly; • Demonstrates strong real-world performance.	• Restricted to pre-defined datasets; • Limited generalization to unseen environments; • Very high data requirements for fine-tuning.
[20]	Task and Motion Planning	PALM-E, PALI-X, ViT	Manipulation, pick and place, locomotion	New Dataset	• Enhances multimodal task understanding; • Integrates well with various input modalities.	• High computational cost; • Dependency on specialized hardware.
[142]	Task and Motion Planning	RT-1, PALM-E, PALI-X, ViT	Manipulation, pick and place, locomotion	Several existing datasets	• Demonstrates some generalization between embodiments and tasks in the dataset; • Better performance than baselines.	• Limited scalability beyond supported scenarios; • Requires extensive training data.
[143]	Task and Motion Planning	SayCan, CLIP, ViLD	Manipulation, pick and place, locomotion	New Dataset	• Improves SayCan [141] object location capabilities.	• Still limited by SayCan [141] low-level skill set.
[144]	Task and Motion Planning	PALM-E	Manipulation, pick and place, locomotion	New Dataset	• Handles multimodal inputs; • Better performance than baselines.	• Computationally intensive; • Limited generalization to unseen modalities.

functions make the algorithm probabilistic complete, and exact.

The Algorithm 4 contains a summary of the MPNet algorithm. The authors present different algorithm definitions for MPNet, yet in this section, we present the simplest version. The BNP term stands for bidirectional neural planner and applies the $Net_{planner}$ part of Figure 9 for two sets of trajectories, one starting at the initial state and one starting

in the goal state. This makes the solution exact because the trees meet in the middle. Additionally, the MPNet has a *LazyStatesContraction* algorithm that discretizes and optimizes the path. Finally, suppose the initial solution has discontinuities of unfeasible states. In that case, the *Replan* method tries to fix this by planning using BNP or any motion planner (e.g., RRT*) only in the problematic regions of the trajectory. The authors claim that their method surpasses by

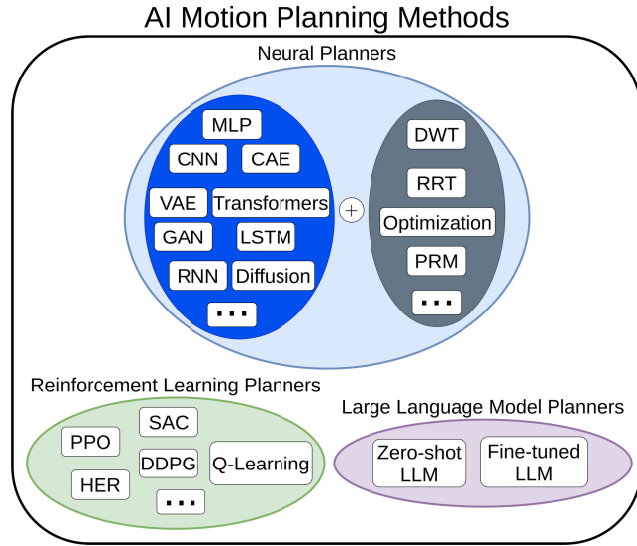


FIGURE 6. Survey taxonomy.

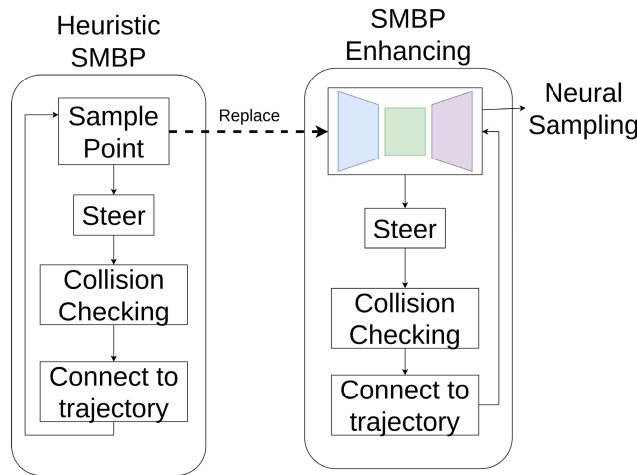


FIGURE 7. Example of approach that replaces the sampling block of an SMBP with a neural network trained on successful planning solutions and the knowledge of the environment.

far the traditional motion planning methods. For instance, the authors compare the MPNet approach with RRT* [42], Informed RRT* [145], and BIT* [146]. The results show that if the complexity of the problem increases (number of dimensions), the MPNet advantage grows and reaches a mean of 41 times speedup, reducing the planning time from 15.58 seconds to 0.38 seconds.

However, despite MPNet being a simple and practical approach, it still has its limitations: data and representation. Training data is still a bottleneck since the MPNet needs an expert to provide paths for training. It still needs a considerable amount of data and only generalize to environments within the training set patterns. Representation is another bottleneck that might be causing the limited generalization and high data requirement. When training the

autoencoder, the input is a flattened point cloud, causing it to lose all its geometric relationships.

Algorithm 4 MPNet(x_{init}, x_T, x_{obs})

```

1:  $Z \leftarrow \text{Enet}(x_{obs})$  ▷ Encoder
2:  $\sigma \leftarrow \text{BNP}(x_i, x_T, Z)$  ▷ Planner
3:  $\sigma \leftarrow \text{LazyStatesContraction}(\sigma)$  ▷ Optimize path
4:  $\text{MaxAttempts} \leftarrow N$  ▷ e.g., 10
5:  $\text{Attempts} \leftarrow 0$ 
6: if  $\text{IsFeasible}(\sigma)$  then
7:   return  $\sigma$ 
8: else
9:   while  $\text{Attempts} \leq \text{MaxAttempts}$  do
10:    for  $i \leftarrow 0$  to  $N_r$  do
11:       $\sigma \leftarrow \text{Replan}(\sigma, Z)$ 
12:       $\sigma \leftarrow \text{LazyStatesContraction}(\sigma)$ 
13:      if  $\text{IsFeasible}(\sigma)$  then
14:        return  $\sigma$ 
15:      end if
16:    end for
17:     $\text{Attempts} \leftarrow \text{Attempts} + 1$ 
18:  end while
19:  return  $\emptyset$ 
20: end if

```

Ichter et al. [15] also present a similar approach for neural motion planning using autoencoders (AE). Their approach uses three networks: state encoder/decoder (ϕ), dynamics network (ψ), and collision checking network (ζ). The state encoder/decoder encodes the state into a latent vector which might be anything from a vector of joints to an image. The dynamics network uses the latent vector from the state encoder to predict the next state. The collision checking network verifies whether the predicted next state latent vector is valid. Finally, the state decoder recovers the trajectory from the latent space.

Figure 10 presents the network architecture. During training, the authors train the autoencoder. Still, during inference, the encoder only encodes the initial states. The method samples subsequent states from the latent space (Z_i) using a best first (BF) selection function to find the lowest cost tree node in the vicinity of the sample. After this, the dynamics network generates the next state, and the collision check network validates it.

They execute experiments in two types of environments: 2D and humanoid robots (50 dimensions). In the 2D environment experiments, the authors compared their method with the Fast Marching trees (FMT*) [147], and RRT-BestNear [148] algorithms and their method had a worse performance than both. In contrast, the previous algorithms could not handle the humanoid robot environment, whereas the proposed method could handle it with no problem.

They do not define a standard state (environment + robot state) representation, so it becomes another hyperparameter to tune. In contrast, the MPNet defines the state as a point cloud and a vector of joint angles or Cartesian

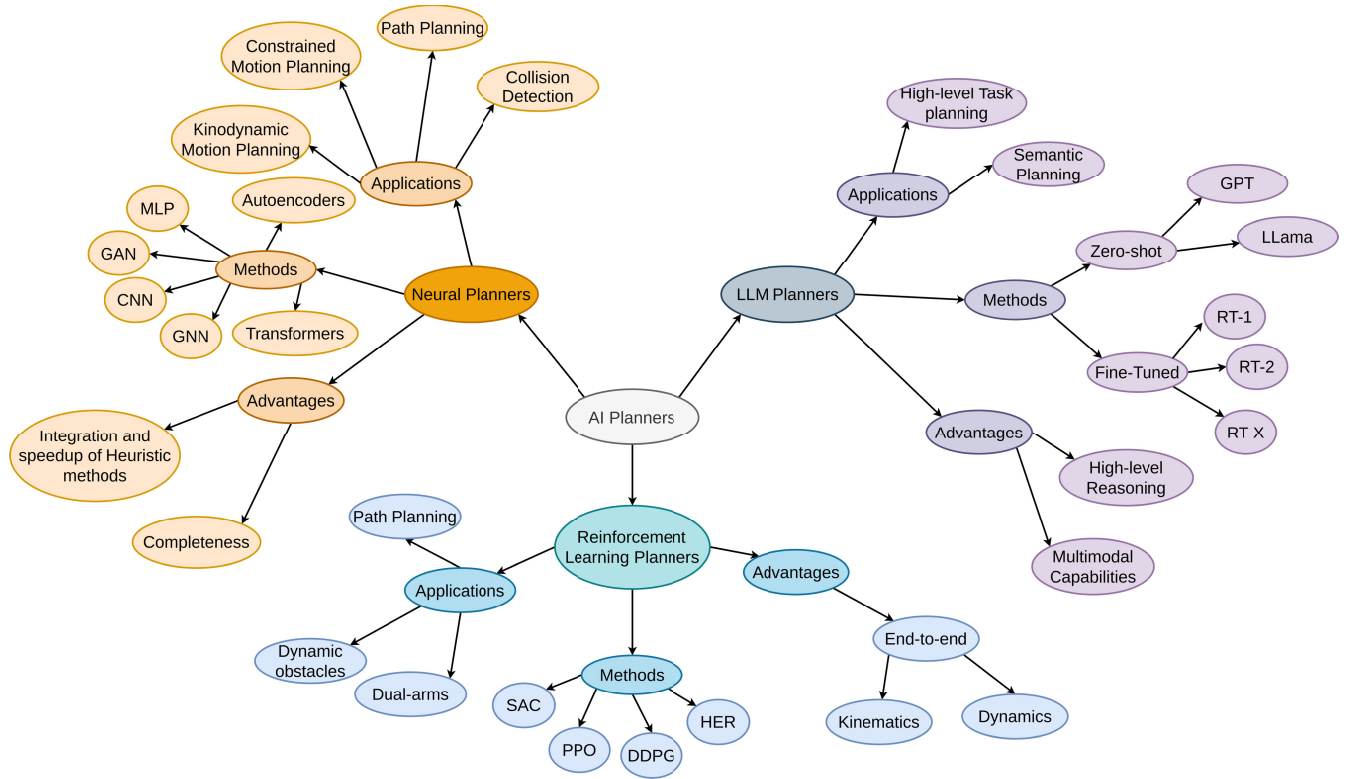


FIGURE 8. A concept map summarizing three categories of AI planners: neural planners, reinforcement learning (RL) planners, and large language model (LLM) planners. For each planner type, the map outlines its key applications, methods, and advantages, providing a structured overview of the concepts covered in this section.

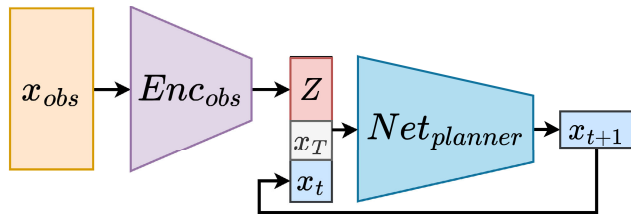


FIGURE 9. MPNet architecture diagram [1], [7].

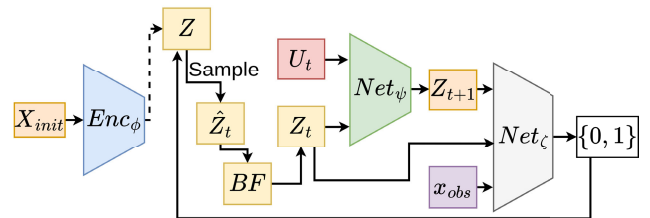


FIGURE 10. L-SBMP [15] diagram.

positions, which is easier for the reader to understand and reduces the system's complexity. Additionally, the selection of experiment scenarios seems poor because the 2D environment performs worse than the baselines, and the baselines cannot be applied to the humanoid environment, making the comparison invalid. One highlight is that their method for collision checking using the latent space is accurate, with 95% accuracy for the humanoid robot case.

An incremental update on the work by Ichter et al. [16] uses a conditional variational autoencoder (conditional VAE) to learn a latent distribution of the motion planning problem. The model has an encoder and decoder. The encoder input is the condition and the robot states (e.g., expert demonstrations). The condition might be the start state, goal state, and environment information (obstacles), i.e., $x_{init}, \mathcal{X}_{free}, \mathcal{X}_{goal}$. The input of the decoder is the condition

and the latent space. After training, the network generates samples as seeds for a classic motion planning algorithm, FMT* [147], in this case.

Figure 11 illustrates a simplified version of the architecture during training and inference. This architecture replaces the sampler module of sampling motion planning and leaves the rest of the tasks for the motion planner to realize. This approach assures probabilistic completeness and exactness. They compare their method to the original BIT* [146] and FMT* [147] and prove that it performs similarly in the 2D environment. This method shows its advantages in more complex scenarios, where the authors experiment with a 12D environment showing that this approach is significantly better than BIT* [146] and FMT* [147].

The authors provide a much better experiment setup in this work [16] than in [15] and extensively test their approach.

It consistently shows a lower cost and higher success rate than traditional sampling schemes in all test scenarios.

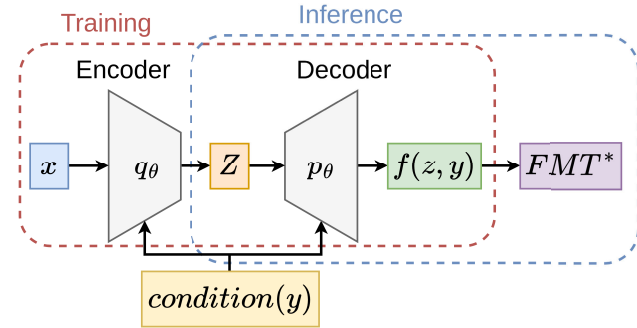


FIGURE 11. Ichter et al. [16] approach diagram.

Another work from Ichter et al. [84] propose a novel method to identify critical regions in the PRM algorithm. In this context, critical regions are small regions where the path must converge to reach the goal, that is, a hole in a wall.

The algorithm works by sampling states (same as usually done for RPM and other sampling-based motion planning) and then computes the criticality of the states based on the concept of betweenness centrality in graph theory. After that, the algorithm selects critical samples and non-critical samples, which are samples near the critical region and samples far from critical regions, respectively. At this step, the method connects the critical samples to all non-critical samples, making them a hub for the connections in the environment. Additionally, it connects the non-critical samples among themselves based on a distance radius. Finally, for planning, it connects the initial and goal states to this graph and finds the shortest path.

In summary, they use the centrality data and environment representation to train a neural network to identify the critical states based on environmental information. In experiments, the authors evaluate their method in simulation (2D and 3D) and real-world scenarios, improving the success rate considerably from the uniform PRM algorithm.

Hung et al. [85] present a similar approach. In their work, the VAE learns the robot's inverse kinematics, and they also add another network to predict collision using the latent space as input. The inference process differs from previous approaches, using the activation-maximization method in the latent space to generate samples that create a trajectory from the start state to the goal state. Figure 12 presents a diagram of their approach. The activation maximization (AM) process consists of backpropagating the loss between x_{t+1} and X_{goal} (Euclidean distance) but instead of updating the weights, they use the AM method to update the latent vector to draw another sample. They combine the AM method in the decoder (Dec_{vae}) and collision detection (Net_{cc}) networks to update the latent vector.

They use a single-arm with multiple cylindrical obstacles as a testbed for the algorithm. They compare their approach with classical methods such as FMT* [147], BIT* [146],

and CHOMP [149], among others. They also compare the methods using three metrics: success rate, time, and path length. The results did not favor their method, which is marginally better than other methods in more complex situations, but in more simple scenarios, it has comparable performance.

The approach by Hung et al. [85] is fully data-driven. There are no supporting functions like MPNet [1], [7]. Therefore, since they do not have any supporting methods to enforce completeness, i.e., this method is not complete. Additionally, the network only has the joint angles and the shape of the obstacle, i.e., the cylinder's height, radius, and position. The training information is a series of points in space, not a trajectory.

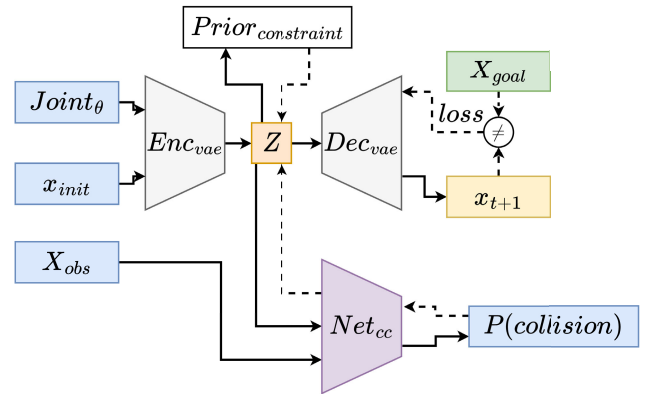


FIGURE 12. Chia-man Hung et al. [85] approach diagram. The dotted lines mean the gradient operation and the full lines mean the forward operation in the networks.

Fishman et al. [86] propose an update on the MPNet [1], [7] framework, called MPiNet. The authors add a configuration encoder and replace the environment encoder (contrastive AE) of MPNet with the PointNet++ [150] architecture. PointNet++ claims to preserve the geometric relationship of the points while creating a compressed representation of the point clouds. It is a well-known method for point cloud data segmentation and classification. Additionally, the authors segment the point cloud to highlight the robot, obstacles, and goal position. The authors combine configuration encoder and environment encoder latent spaces to input into a decoder that outputs the next position. Figure 13 illustrates the MPiNet architecture.

This approach is very similar to the MPNet [7], and the authors use MPNet as a comparison in their tests. The authors also compare other methods such as Adaptive Informed Trees (AIT*) [151], Geometric Fabrics [152], and STORM [153]. The results favor their approach. The authors also prove that their method scales with data better than MPNet. Additionally, the authors show that the robot can deal with moving obstacles depending on the speed of the obstacle.

They produced considerable data, over 3.2 million planning problems, and over 500k environments. This amount

of data is unfeasible for application in real-world scenarios. The computational resources and time required to train such models are probably orders of magnitude greater than those used to train the MPNet. For small datasets, MPNet is still better. Another point is that the authors do not have supporting methods to enforce completeness, that is, they cannot prove that this method is complete.

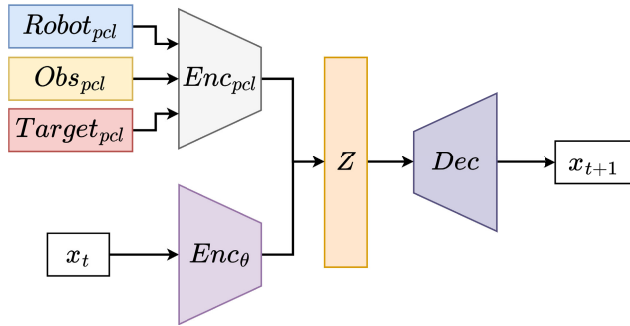


FIGURE 13. MPiNet diagram [86].

In the constrained motion planning application, Qureshi et al. [27] extend the MPNet architecture to deal with planning in constraint manifolds and create the CoMPNet. The CoMPNet extends the basic structure of the MPNet: Environment encoder, sampling network, and support method for completeness. They also add new methods that adhere to the constrained motion planning task. The additional module is the task encoder, which encodes a task phrase (for example, open the cabinet) to a latent representation similar to the environment encoder. They change the environment representation from point cloud to voxels to apply CNNs to encode the environment. The sampling network is similar to the MPNet.

Figure 14 illustrates the network component. The most significant change lies in the supporting methods. In this new formulation, the authors add new methods: *TransverseManifold()* (Algorithm 5 lines 6, 8), and *NearestNode()* (Algorithm 5 lines 5,7). *NearestNode()* projects the outputs of the sampling network to the constraint manifold. *TransverseManifold()* is similar to the previous *SteerTo()*, which tries to connect the current node to the goal.

They compare their methods with the current state-of-the-art methods: CBiRRT [40], Atlas-RRT [44], and TB-RRT [43], in terms of execution time. The results favor CoMPNet, where execution time is inferior to all other methods in all testing scenarios. The authors also execute an ablation study to evaluate the importance of using the projection, task encoder, and one-hot encoding. In this ablation study result, the impact of using one-hot encoding is minimal ($\approx 1\%$ success rate), but the addition of task encoding and projection increases considerably the success rate ($+5 \sim 13\%$), whereas the execution time difference is negligible.

Johnson et al. [37] extend the MPNet to support non-holonomic robots. The architecture has three components:

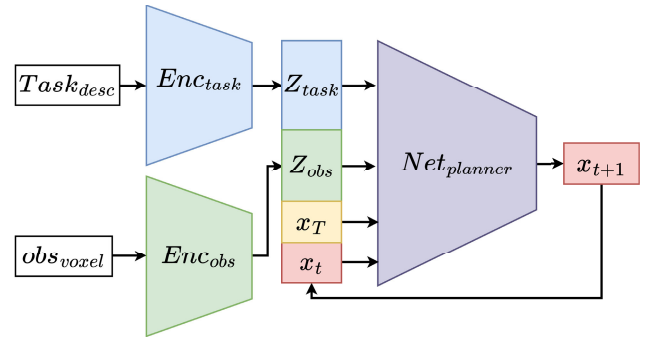


FIGURE 14. CoMPNet diagram [27].

Algorithm 5 CoMPNet($Z_{task}, Z_{obs}, x_{init}, x_{goal}$)

```

1:  $T_a \leftarrow x_{init}; x_t \leftarrow x_{init}$ 
2:  $T_b \leftarrow x_{goal}; x_T \leftarrow x_{goal}$ 
3: for  $i \leftarrow 0$  to  $n$  do
4:    $x_{t+1}^a \leftarrow \text{NeuralConfig}(Z_{task}, Z_{obs}, x_t, x_T)$ 
5:    $x_{near}^a \leftarrow \text{NearestNode}(x_{t+1}^a, T_a)$ 
6:    $x_{new}^a \leftarrow \text{TransverseManifold}(x_{t+1}^a, x_{near}^a, T_a)$ 
7:    $x_{near}^b \leftarrow \text{NearestNode}(x_{new}^a, T_b)$ 
8:    $x_{new}^b \leftarrow \text{TransverseManifold}(x_{new}^a, x_{near}^b, T_b)$ 
9:   if  $\text{Reached}(x_{new}^a, x_{new}^b)$  then
10:    return  $\text{Extractpath}(T_a, x_{new}^a, x_{new}^b, T_b)$ 
11:   else
12:      $x_t \leftarrow x_{new}^a; x_T \leftarrow x_{new}^b$ 
13:      $\text{Swap}(T_a, T_b)$  ▷ Bidirectional plan
14:      $\text{Swap}(x_t, x_T)$  ▷ Bidirectional plan
15:   end if
16: end for

```

transformer (T), encoder (Enc_{cmap}), and planner network ($Net_{planner}$). Figure 15 illustrates the architecture. The transformer (T) centers the cost map egocentrically to the robot. At the same time, the encoder (Enc_{cmap}) network converts it to a latent vector (Z). Finally, the planning network ($Net_{planner}$), like MPNet, outputs the next control input for the robot, given the current position, the goal position, and the encoded latent vector. The authors also add safeguard methods to enforce completeness in their method, similar to MPNet. They experiment in simulated and real environments and compare the results to the available method in the ROS navigation stack: Dynamic window Approach (DWA) [154], Time-elastic Band (TEB) [155], and Anytime RRT* (where RRT* [42] samples in the windows instead of the author's neural sampler). The results show that the MPNet approach is considerably faster and has a better success rate than other approaches.

The work by Johnson et al. [37] has an explicit limitation: it only works with 2D data. In contrast, the original MPNet works with any robot configuration. The choice of using vision transformers also corroborates with this specialized MPNet architecture.

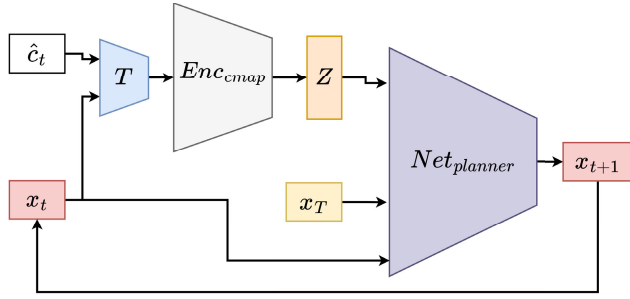


FIGURE 15. Jacob J. Johnson et al. [37] approach diagram. It is very similar to the MPNet [7] but focused on 2D planning. $\hat{c}_t$ is the padded cost map, the transformer (T) block centers $\hat{c}_t$ with respect to the robot position x_t , the rest of the blocks are similar to MPNet [7].

Johnson et al. [87] continue the specialization in 2D motion planning on a different approach focusing even further on 2D data. They propose a motion planning framework using transformers, which they name motion planning transformers. The method uses transformers (Net_{FCN} , Enc_{trans}), attention, and region proposal network (Net_{RPN}) (faster-RCNN) [156] to predict a map region (m_{RP}) likely to contain the best path from start to end. Any sampling motion planner can obtain a collision-free and complete path with that region. The highlights of this method are that it works with different map sizes without retraining and works for kinodynamic and non-holonomic constraints planning.

Figure 16 illustrates the architecture. The inputs of the network are a collision map containing the start and goal positions (m_e) encoded in an additional layer of the collision map image (m_c). The output is a region proposal map (m_{RP}). The authors obtain the region proposal map from paths created by sampling motion planners and consider an area around the path as anchor points for the region proposal network to improve accuracy.

They experiment with two types of scenarios and two types of robots. Compare with RRT* [42] and Informed-RRT* [145] methods. The scenarios are a simple random collision map and the maze scenario, and the robots are a point robot and a Dubins car model. The results show that while motion planning transformers lose accuracy ($\sim -2\%$), it improves speed several folds (from 5 seconds to 0.20 seconds).

Johnson et al. [87] claim that their method works under kinodynamic and non-holonomic constraints, which works, but their method is the transformer network. The network does not know this information. The network only predicts the best region for planning, the part of the method that works under these constraints is the motion planner, which is not the paper's contribution.

Li et al. [88] formulate a new neural architecture for solving the kinodynamic motion planning problem. This problem contrasts with standard motion planning, which only considers the collision-avoidance condition, whereas, in kinodynamic motion planning, the constraints are kinematic (collision-avoidance) and dynamic (velocity, acceler-

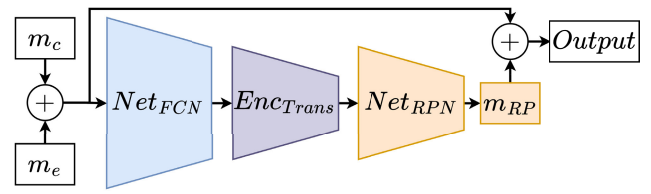


FIGURE 16. Motion planning transformers (MPT) [87] diagram.

ation). This characteristic makes the problem PSPACE-hard and computationally much more complex. They propose a generator-discriminator approach, where the generator (Net_G) predicts the subsequent states and the discriminator (Net_D) filters these states to obtain the best one according to a specific loss function. Finally, model predictive control (MPC) steers the robot. The environment encoder network (Enc_{obs}) is similar to the CoMPNet [27] approach and uses a representation of the environment in voxels (v_{obs}).

Figure 17 illustrates the architecture. They define two types of models: *MPC-MPNetPath*, and *MPC-MPNetTree*. The difference between them is how it conducts the exploration (grows the tree). The *MPC-MPNetPath* generates and adds nodes sequentially, expanding the tree one node at a time in a linear fashion. In contrast, the *MPC-MPNetTree* processes multiple samples simultaneously, allowing the tree to grow with several nodes in parallel during each iteration. Another difference between *MPC-MPNetPath*, and *MPC-MPNetTree* lies in the inputs of the generator (Net_G) where in the *MPC-MPNetPath* they come from a single nearest node in the tree and *MPC-MPNetTree* they come from a batch of nearest nodes. Consequently, the output reflects the difference by growing multiple nodes simultaneously in *MPC-MPNetTree*.

They compare their methods with sparse-RRT (SST) [148] in the following scenarios: Acrobot, Cart-pole, Car, and Quadrotor. The execution time experiments favor the MPC-MPNet, but the path cost seems worse or equivalent to SST. The authors also executed an ablation study to evaluate the impact of the neural discriminator in their approach, concluding that discriminator use reduces the computation time.

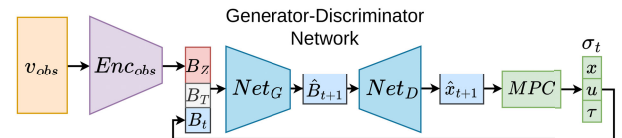


FIGURE 17. MPC-MPNet diagram [88].

Qureshi et al. [28] introduce the CoMPNetX as a direct extension of the CoMPNet [27]. This approach builds upon [27], [37], [88]. In this approach, the authors further extend the task encoder and create a task planner ($NTP2$) module that breaks larger tasks into smaller sub-tasks interactively until it reaches the most basic task and creates a sequence of operations from this. The environment encoder

(Enc_{obs}) is the same as CoMPNet [27]. In CoMPNetX, the authors replace the sampling network with an adversarial architecture (Net_G, Net_D) where the generator (Net_G) predicts the next state and the discriminator Net_D predicts its distance to the constraint manifold (d_M). This method relies on a sampling motion planner with a constraint integrator (SMP_{CI}) to connect the samples the CoMPNetX finds and guarantee probabilistic completeness. The authors use RRTConnect [157] with constraint adherence (Projection, Atlas [44], and Tangent Bundle [43]), but any other motion planner with similar characteristics would suffice.

Figure 18 illustrates this architecture. In Algorithm 6, we define the CoMPNetX algorithm connecting to Figure 18. In this algorithm, the v refers to the threshold for projection into the manifold M . In the Algorithm 7, we define an application of the CoMPNetX similar to CoMPNet [27] Algorithm 5 and MPNet [7] Algorithm 4 where the method performs bidirectional planning. In contrast to previous approaches, the algorithm executes a mixed sampling using the neural planner and a sampling motion planner (SMP) to obtain better coverage of the environment (Algorithm 7 lines 10 and 12).

They compare RRTConnect with or without the CoMPNetX sampler. The result shows that the success rate is comparable. At the same time, execution time is considerably shorter for the CoMPNetX sampler, a trend we observe in all works using neural networks as samplers. The neural network becomes a biased sampler that samples more effectively. Since CoMPNetX uses a sampling motion planner as a base, it ensures probabilistic completeness.

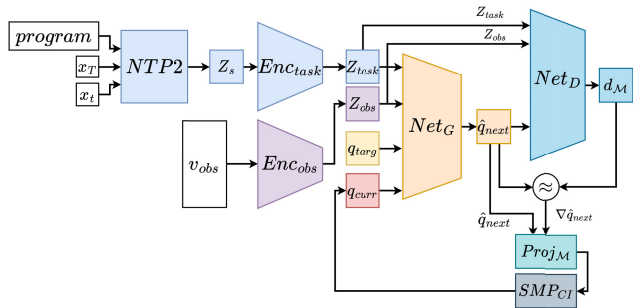


FIGURE 18. CoMPNetX diagram [28].

Algorithm 6 CoMPNetX [28] ($Z_s, v, q_{curr}, q_{targ}$)

```

1:  $Z_{task} \leftarrow Enc_{task}(Z_s)$ 
2:  $Z_{obs} \leftarrow Enc_{obs}(v_{obs})$ 
3:  $\hat{q}_{next} \leftarrow Net_G(Z_{task}, Z_{obs}, q_{curr}, q_{targ})$ 
4:  $d_M \leftarrow Net_D(\hat{q}_{next}, Z_{task}, Z_{obs})$ 
5: if  $d_M > v$  then  $\triangleright Proj_M$ 
6:    $\hat{q}_{next} \leftarrow \hat{q}_{next} - \gamma \nabla_{\hat{q}_{next}} D_\theta(\hat{q}_{next}, Z_{task}, Z_{obs})$ 
7: end if

```

Xie et al. [89] develop a data-driven approach for multiple robot motion kinodynamic planning. The authors

Algorithm 7 Bidirectional CoMPNetX

```

1:  $t \leftarrow 1; p_0 \leftarrow InputProgram$ 
2: while not ProgramEnd do
3:    $x_t, v_t \leftarrow GetObservation()$ 
4:    $p_t, Z_s, ProgramEnd \leftarrow NTP2(x_t, x_T, p_{t-1})$ 
5:    $q_{init}, q_{goal} \leftarrow GetConfigs(p_t, Z_s)$ 
6:    $\mathcal{T}_a, \mathcal{T}_b \leftarrow InitializeBiSMP(q_{init}, q_{goal})$ 
7:    $q_{targ}^a, q_{targ}^b \leftarrow q_{init}, q_{goal}$ 
8:   for  $i \leftarrow 0$  to  $N_{max}$  do
9:     if  $i < N_{ismp}$  then
10:       $q_{next}^a \leftarrow CoMPNetX(Z_s, v_t, q_{curr}^a, q_{targ}^b)$ 
11:    else
12:       $q_{targ}^a \leftarrow TraditionalSMP(\mathcal{T}_a, \mathcal{T}_b)$ 
13:    end if
14:    if  $q_{next}^a, PathFound \leftarrow BiSMP(q_{next}^a, \mathcal{T}_a, \mathcal{T}_b)$ 
15:      if PathFound then
16:         $\sigma_t \leftarrow ExtractPath(\mathcal{T}_a, \mathcal{T}_b)$ 
17:      end if
18:       $q_{curr}^a \leftarrow q_{next}^a$ 
19:      Swap( $\mathcal{T}_a, \mathcal{T}_b$ )
20:      Swap( $q_{curr}, q_{targ}$ )
21:    end for
22:    if  $\sigma_t$  is not empty then
23:      ExecutePlan( $\sigma_t$ )
24:    else
25:      return Failure or AskExpert
26:    end if
27:     $t \leftarrow t + 1$ 
28: end while
29: return  $\emptyset$ 

```

have two networks, steer and cost-to-go, and combine both in a centralized RRT planner to plan trajectories of up to 16 robots. This method considers each obstacle and each robot position for obstacle avoidance constraints. The authors replace the original RRT steer function for their proposed data-driven network and the distance function, which considers local observations and temporal costs.

The authors consider success rate, path length, and path cost for evaluation. They also executed an ablation study using both networks, only the steer network and only the cost network, and compared it with the original RRT algorithm. The results show that using both heuristics increases the success rate, and the path cost decreases considerably. The path length remains similar.

This work takes a different approach than MPNet [11], [7] and derivative works because it does not train a network to replace the sampler module but instead learns a new steer function and cost-to-go approximations from data. The authors compare replacing the steer function and the cost-to-go functions separately. However, there are still approaches that replace the collision check and the sampler with neural networks, so it would be interesting to compare replacing each component of the SBMP with a learned network and assess its impact on the overall path quality and speed.

Johnson et al. [35] introduce the VQ-MPT method that accelerates sampling-based motion planners (SMP) by narrowing the sampling space. This method follows the same framework set by MPNet. The fundamental concept of this paper is vector quantization (VQ), a generative model (similar to VAE) that represents the data in latent space as a collection of learnable vectors named dictionaries. The approach has two stages: quantization and prediction.

In the quantization stage, the network learns how to represent the planning stage with a set of distributions using an encoder-decoder architecture. At the quantization stage, they do not include information on the environment, i.e., point clouds, but only the robot trajectory.

The prediction stage predicts the sampling dictionary from the sensor data and start and goal information. The prediction stage has two models: embedding and a final prediction model. The embedding model embeds the environment information (point cloud features), start, and goal, and the final prediction model selects a subset of sampling dictionaries from the quantization model output. The output of the model replaces the sampling distribution of the RRT [38] algorithm. The authors train the models end-to-end using the RRT* [42] output.

This work is another further improvement in learning sampling distributions for SBMP methods. It takes a different approach, where the model that creates the sampling dictionaries learns exclusively from trajectory data. The prediction stage incorporates the environment information to choose the best sampling dictionary. However, this leads to the logical conclusion that trajectory data is needed to cover most, if not all, of the planning space. Otherwise, it cannot represent the entire space in the sampling dictionaries. Additionally, it would be valuable to execute ablation studies using environmental information to generate the sampling dictionaries and their impact on the method performance.

In subsequent work, Johnson et al. [36] extend the VQ-MPT idea to handle the constrained motion planning problem. This work focuses on joint configuration and not on dynamics or kinematics constraints. The two-stage method is similar to the unconstrained version of VQ-MPT; the encoder stage is the same. The difference is in the sampler part of the architecture, where it maps distributions to the constraint manifold. The authors propose a novel optimization-based projection using the gradient descent method and backpropagation.

The evaluation considers existing heuristic methods for constrained motion planning (CBiRRT, Atlas-RRT, TB-RRT) and the neural MPNet with projection operator [27], [28]. The results show that their method is two times faster than existing methods while maintaining high success rates and low execution time and path length. In the previous work, they did not mention which feature extraction method they use for point clouds, but this time, they mention PointNet++ [150] for point cloud feature extraction.

Gaebelt et al. [90] implement a neural network-based sampler for a sampling-based motion planning algorithm.

The authors use a conditional variational auto-encoder (CVAE) architecture for the sampling network. The model inputs are the starting and goal position with a progress variable (from 0 to 1) that controls where to sample across the path, i.e., if the progress variable is 1, it samples close to the goal state. The conditional variable is the compressed scene representation, obtained by passing a voxel grid to a 3D-CNN to obtain a vector of size 10. They use the RRT* to generate training samples for the CVAE. The authors combine this network sample with uniform sampling for optimality guarantees in a random manner, and they can also control the rate of use of each planner. Additionally, since the limited voxel grid resolution, the authors define an adaptive sampling algorithm that initially relies on the network sampler. If the network sampler is insufficient to get a path, it returns to a uniform sampler.

For experiments, the authors consider four variants of the RRT algorithm: RRT, RRT-Connect, RRT*, and RRT#, and compare uniform sampling with their neural sampling strategy in terms of path cost and planning time. On average, their method performs better than the uniform sampler for the initial solutions. If given more time to work, the cost improvement increases. This strategy is similar to MPNet, but including the adaptive sampling and fallback strategy makes it slightly different. However, the results report shows no significant advantage in terms of sampling time compared to the initial solutions if we compare them to MPNet. Still, the approach adapts to dynamic environments, according to the authors. The bottleneck is the representation and overall network architectures that use voxel and 3D CNN, which is slower than raw point cloud representation and samples one sample at a time instead of batch sampling.

Tin et al. [91] present a novel neural sampling strategy for SBMPs. This novel architecture uses diffeomorphisms to transform sampling distributions with knowledge of the obstacles to sample fewer samples in collision areas. This process is akin to normalizing flows. This approach combines the obstacle information with the path cost to learn diffeomorphisms to reshape a base distribution in a more informative sampling distribution. First, they represent the environment in occupancy grids and use a neural network to learn a representation of this environment in terms of probability of occupation so they can generate a continuous representation. Then, they use this to create the diffeomorphisms and apply them to morph the distribution. They employ GPU to parallelize the occupancy grid representation and the distribution morphing to provide the sample distribution for the SBMP algorithm.

The authors compared their sampling method with RRT*, RRT-connect*, PRM* and STOMP for experiments. The results indicate a considerable decrease in planning time and an increase in success rate when using the proposed informed sampling method. This work is similar to the MPNet framework but leverages a different approach to reach the same goal. The biggest bottleneck in this work is the occupancy grid representation, which hinders scalability.

Therefore, improving this aspect is a natural direction for this approach.

Eunhoo et al. [92] extend the MPNet approach to tackle out-of-distribution scenarios. The approach has two networks: a feature extraction network and an uncertainty-aware sampling network. The feature extraction network consists of a VAE [61] with PointNet [93] as its encoder, in which they model the latent space as a multimodal distribution parametrized by normalizing flow to enhance the distribution expressive power. The uncertainty-aware sampling network is a deep neural network that predicts a sampling distribution. With this distribution, the algorithm calculates the uncertainty and adjusts the sampling process accordingly. The authors integrate their approach as the sampling block for the RRT-connect algorithm.

The authors evaluate their approach in simulation and real-world scenarios and compare it with the heuristic Point-Guided-RRT (PG-RRT) method. Their method achieves similar path costs but slightly higher success rates and lower planning times while achieving similar results in unseen environments. This work is interesting because it recognizes environmental uncertainty. This information is essential and might be used to improve an existing model by collecting more data in uncertain environments or to execute more careful planning in those cases (the article approach does that). However, in terms of performance, this approach is not significantly better than a heuristic approach, which makes it less desirable due to its complexity and data collection requirements.

Chow et al. [94] develop a neural network to replace the steer function of a sampling-based motion planner (SBMP) that considers vehicle dynamics. This idea is similar to other works that replace the sampling function of an SBMP with a neural network [7], [16]. However, instead, they replace the steer function, which is more complex in a kinodynamic constraint scenario. This scenario happens mostly on vehicle-like robots, where the closed-loop planning simulates the vehicle dynamics during planning. This scenario is uncommon for robotic arms, where the constraints are others.

Their approach is named Parallel Closed-Loop RRT* (PCL-RRT*). In this approach, the neural network approximates a computationally heavy controller. The training dataset is the recorded response of an expert controller, given random start and goal states. Additionally, the authors make two assumptions: relative state and maximum controller response. They model the network input as a relative difference between the current and goal states, reducing complexity. The method bounds the network inputs by the distance that generates the controller maximum thrust. The PCL-RRT* adds two new operations to the original RRT*: batch operations and neural network steer. They modify the algorithm to execute the RRT* operations in batches and thus executed in parallel, so initially, the network samples a batch of possible expansion nodes, then inference with steer function (neural network steering), the lowest cost expansions

are added to a list, which is checked against collision and then rewired. This process repeats until the end of the allotted planning time.

This approach considerably improves the controller trajectory evaluation compared to model predictive control (MPC) methods, from 73 min to 3 seconds. Furthermore, the approach improves the number of generated and executed plans compared to the original RRT, RRT*, and SST methods.

Lu et al. [95] tackle the problem of mobile manipulator motion planning in cluttered environments (house interior). This work combines the SBMP with neural planning (NP), using SBMP for global planning and NP for local planning. For SBMP, the authors use RRT and RRT*. For the local planner, the authors experiment with two approaches: discriminative (3DCNN) and generative (CVAE), which learn from data collected using PRM*. The authors represent the environment using occupancy grids. Both local planners encode a sampling distribution.

They compare their approach with several state-of-art methods (e.g., IRRT*, NEXT, FIRE*, BIT*) and prove that higher success rate and lower path length and planning time. Overall, the discriminative approach is slightly better than the generative. Additionally, the authors state that given the similarity between house interiors, their approach trained in simulation could be used zero-shot in real-world environments. Their approach follows the foundational framework of the MPNet. However, considering the specific use case of in-house navigation and manipulation, where the overall structure of the rooms remains the same. Small changes such as moving furniture or chairs might impact heuristical planning accuracy, but a more generalizable neural planner can overcome these issues. Hence, applying a heuristic approach for global planning with a more precise neural approach for local planning makes sense.

Palomares et al. [96] introduce the Control Inference Network (CINet), another variant of the MPNet framework for kinodynamic planning problems. The CINet uses the SST* as a planning algorithm and replaces its sampling method/steer methods. The network, analogous to the MPNet, takes the current position, goal position, and environment features and outputs the controls and their application times. It differs from previous variations of MPNet, which are biased samplers, but the general logic is the same. The environment encoding network is the same as the MPNet, a CAE.

In the experiments, the authors execute an ablation study by fixing the starting position to an area and increasing the number of training samples, which indicates that the more training samples, the better the result, and the larger the starting area, the worse the success rate. Next, the authors compare the SST and SST* algorithms; their solution has considerably lower time and higher success rates in most environments; however, SST and SST* still have lower path costs in most scenarios.

The work by Wullt et al. [97] takes the MPNet (and other neural motion planners) as a base. However, they intend to support dynamic environments. They use the PointNet as a

base for environmental representation and extend it. Since they intend to support dynamic environments, they stack four consecutive point cloud measurements as input. Additionally, they sample the point cloud points at the object surface. The authors concatenate the start and goal to each point of each point cloud to generate the local features and then combine them to generate global features (using max-pooling). The method concatenates the global features, forming a latent space. Then, another network gets this space as input and outputs the velocity. To create the dataset, they employ space-time RRT (ST-RRT). ST-RRT is a bidirectional planner that can plan in dynamic environments and requires complete knowledge of future movement. The authors use a planner similar to ST-RRT, where sampling occurs in time intervals. Additionally, the planner considers only states at the current time interval for connecting the points.

In the evaluation, they execute an ablation study of their representation learning scheme (StackedPointNet) compared to the ReRRT-C and simply using Pointnet. Although using the point net encoding results in shorter paths, StackedPointNet has a better overall success rate. In this work, the authors use the ST-RRT as a planner and the obstacle representation to predict the transition velocity between time steps, not the next time step.

Ni et al. [98] formulate a novel method using physics-informed neural networks (PINN) for motion planning. The method consists of solving the eikonal equation to generate a time field. The Eikonal equation is a first-order non-linear partial differential equation that finds the shortest travel time between two points under the speed constraint. This method has advantages in terms of speed and data requirements. The authors can generate the dataset within minutes, compared to other data-driven approaches, which take hours. Another advantage is overall speed and success rate. This method generates trajectories faster and more accurately than previous methods.

This method architecture comprises three networks: a robot-centric workspace encoder, a C-space encoder, and a time field generator. The workspace encoder encodes a robot-centric workspace using 3DCNN. The C-space encoder encodes the robot C-space using a resnet-style network, and finally, another resnet-style network generates the factorized time field. One can compute the speed with the original Eikonal equation with the factorized time-field information. In summary, the robot moves faster in free space regions and slower in obstacle regions. The authors also introduce a non-linear min-max symmetric operator to combine the workspace and C-space features as input for the time-field generator. This operator addresses the symmetry property of the eikonal equation.

In the experiments, they compare their methods in two steps: they compare the eikonal equation solution architecture and the motion planning solver methods. The first set of experiments aims to evaluate how the architecture and the symmetric operator compare to existing networks to solve

eikonal-equations, and the results show that the proposed architecture can represent complex shapes better than previous ones for obstacle avoidance while planning. For Motion planning comparative results, the authors compare against classic and data-driven methods, i.e., RRT* and MPNet, and the results show that their method has the lowest solution time but still has a higher path length than data-driven methods. The disadvantage of this method is its specialization, which cannot adapt to new environments. The method computes the time fields for each environment. Thus, a new environment requires training a new network. Therefore, this method is unable to respond to sudden environmental changes.

NTFields has limitations in scaling robot complexity and environment complexity. The reason for that limitation lies in the eikonal equation learning. Ni et al. [99] improve existing work by proposing a novel formulation for the eikonal equation and a novel training method. The novel formulation uses a viscosity term that provides a differentiable and unique approximation of the equation solution because the eikonal equation is not differentiable in every point of the space, and the gradient is not unique in these non-smooth points in space, causing training to be sub-optimal because it uses the gradient for training.

The novel training method consists of a progressive speed scheduling where they gradually progress from a max speed (obstacle-free) to low speed (around obstacles) in the time fields, avoiding the local minima issue in NTFields. The neural architecture uses a Fourier-based C-space encoding using a resnet-style multi-layer perceptron. Next, the authors apply the same symmetric non-linear operator to combine the features and use a resnet-style network as a time field generator. The authors proposed a new objective function to improve the original NTFields L1 norm.

They compare their approach with NTFields, IEF3D, FMM, RRT*, LazyPRM*, and RRT-Connect for experiments. The comparison metrics are time, length, safe margin (distance to obstacle), and success rate. In simple scenarios, their performance is comparable to the original NTFields, but as the complexity increases, their approach success rate increases compared to NTFields. Additionally, NTFields requires training a model for each environment. In contrast, their method does not have this limitation, i.e., the model can learn multiple environments simultaneously, whereas the original NTFields require a network per environment. The critical limitation of this method also lies in the environment description. The network can learn multiple environments simultaneously but cannot generalize to unseen environments.

Ni et al. [100] further extend the NTFields methods to handle constrained motion planning (CMP) scenarios. The neural network learns how to solve the Eikonal equation on the constraints manifold and work without expert data. The authors modify the standard Eikonal equation using the NTFields approach. The authors also compute the ground-truth speed to use in the isotropic loss function,

compare it to the predicted speed by the Eikonal equation, and thus optimize the model weights during training.

For comparison, they use the methods CBiRRT, P-NTFields, and CoMPNetX to evaluate their method against CMP state of the art. The authors consider planning time, path length, distance to constrained manifold (margin), and success rate. The authors execute experiments in simulation using a real robot (door opening and object manipulation). Their method proves to have better performance in time and path length in all scenarios. Still, it performs worse than the method and CoMPNetX for simulation and object manipulation scenarios, respectively. This method's main advantage is that it requires a very short time for data generation (seconds) to output good results. However, it requires a careful design of equations and constraints that might affect the model accuracy and generalization if not well designed.

Carvalho et al. [101] implement a diffusion model to learn the optimal sampling distribution conditioned by start and goal, thus combining sampling and optimization, called Motion planning diffusion (MPD). The authors use successful solutions to the motion planning problem to train the diffusion model. During inference, the authors sample a batch of trajectories and proceed to the optimization step. The method computes the diffusion prior and the gradients of costs for each trajectory step and moves the trajectories to low-cost regions using the diffusion models. The costs are related to motion planning constraints and path costs: self-collision, collision, and distance to the goal.

For the experiments, the authors compare their method with CVAE, RRT-Connect combined with the Gaussian process motion planning framework in 2D, 3D, and 7D environments. The results show that their method has inferior solution times compared to all other methods while having comparable success rates. However, when adding new random obstacles, the method success rate decreases due to not using environmental information during training. Additionally, the diffusion model generates more possible solutions than CVAE. Their work is similar to the MPNet in learning an optimal sampling strategy for motion planning problems. However, the formulation of the approach in this work does not consider the environmental information, which leads to an issue: the model might not have knowledge of the entire environment if the trajectories used during training do not cover the environment uniformly. This model needs retraining on each new environment, given that the model has no environment information and has not even been successful in planning solutions. Still, diffusion is a good fit for the sampling-based motion planning context, given the random sampling and capabilities of encoding sampling distributions in the diffusion process.

Nawaz et al. [102] propose a novel approach (extension of dynamic system framework) for learning from demonstration (LfD) motion planning by learning Neural Ordinary Differential Equations (NODE) to learn the dynamics of periodic

motions and augment its error dynamics (enforce stability) with Control Lyapunov and control barrier functions. This approach models the motion plan as a vector field instead of waypoints for improved adaptation to unseen environments.

The authors use the LASA [158] dataset, which contains 2D handwritten data, and a 7 DOF robot with periodic motions for evaluation. They compare their approach with other dynamical systems (DS) approaches (SEDS, LVP-DS). The primary metrics are dynamic time warping (DTW), training, and execution time. The results show that their method has a smaller DTW measure (measures the distance between sequences), a smaller training time, and a considerably smaller execution time. Using ODE to model the motion requires training on each problem again. It can adapt to slight perturbations but cannot generalize.

Yu et al. [58] address the motion planning problem by combining control barrier functions (CBF) and sampling-based motion planners (SMBP). The authors combine the CBF real-time collision avoidance control with the long-horizon planning capabilities of RRT. The proposed method is called CBF-induced neural controller (CBF-INC) and has two types of input: signed distance and point clouds. The authors train a neural network to encode the safety constraints of the CBF. They collect offline data using heuristic controllers to solve motion planning problems with random start and goal positions for environmental coverage. For the point cloud variant of the method, the authors employ point-net to extract feature vectors for the obstacles. They use CBF-INC as a steer function for the RRT, i.e., it expands the tree to the following motion point and discards unsafe actions.

They evaluate their approach in 4 DOF and 7 DOF robots. They compare CBF-INC with RL, Imitation learning, and original RRT. Their method has a slightly longer planning time than the original RRT but the best success rate and path length of all methods. Their approach is similar to MPNet, but instead of training a sampler, it tries to improve the collision checking speed. Combining the biased sampler and this steering function might lead to some good improvements.

Arias et al. [103] investigate the problem of multi-agent motion planning with dynamic environments. The method tries to identify dynamic regions and construct a sparse probabilistic roadmap to handle this case. The goal is to improve sampling efficiency and reduce overall planning time. This problem suffers from the same issues as the single-agent motion planning problem, such that with increasing robot complexity and environment complexity, the sampling and collision checking become intractable.

The authors train a network to identify critical regions in the static environment where inter-agent collision might happen, e.g., doorways. One of the key ideas of this work is the measure of betweenness centrality in graph theory, which measures the visitation frequency of a vertex in the graph. However, this measure is not enough to achieve the goals, so the authors extend it by adding information called avoidance critically that consists of identifying how long the

agent waits because waiting is a sign of avoiding collision. Regarding the model, they focus on a binary classification problem of identifying critical and non-critical regions. The input of the model is an occupancy grid, and the model identifies whether there is a critical region in the center of the occupancy grid or not. The rest of the method is straightforward: generate a discrete occupancy grid, then identify and connect the critical regions, sample C-space uniformly, and connect the non-critical regions. This process repeats for every agent.

They test both in simulation and real-world environments and compare their results to the state of the art in multi-agent planning. The results demonstrate that the proposed method generates more efficient roadmaps with fewer nodes.

Kicki et al. [104] introduce a novel method to solve constrained motion planning problems called constrained neural motion planning with B-splines (CNP-B), which directly learns a planning function on the constraint manifold from data using B-splines to represent the learned path efficiently. The boundary points (start, goal) are computed analytically and fed to the network, which predicts the control points of the B-spline. Additionally, the authors define the trajectory as a combination of configuration and rate of change in time. To train the network, the authors employ plans created using CiBRR.

For experiments, the authors consider air hockey and moving a heavy box without changing orientation. They consider several state-of-the-art methods as a comparison method: CBiRRT, MPC-MPNet, TrajOpt, and SST. Their approach achieved a better success rate and planning time while achieving comparable motion time and vertical error in the block move task, considerably lower planning time, and comparable results in other categories. The authors also execute an ablation study reducing the number of dataset samples in a train, network size, object mass, and spline control points.

As for limitations, their approach focuses on local planning and does not consider obstacle avoidance. Also, since it is an approximator, the network might output unfeasible paths, which, according to the authors, can be quickly evaluated and fixed using heuristic methods. Additionally, each scenario must have a different network, i.e., the networks do not generalize to different scenarios. However, they can adapt to environmental conditions like object weight and air hockey height.

Hou et al. [105] develop a data-driven approach to learn high-level motion commands and robot dynamics from collision-free data. In this work, the network does not output direct robot positions. Instead, it outputs command types (e.g., point-to-point, linear, circle) and command parameters (e.g., motion velocity or desired state). Additionally, since it considers a discretized dynamic environment (with known dynamics), for each step, there is a representation of the environment. Their architecture has three levels: representation, high-level control policy, and high-level robot dynamics. The

authors employ a CVAE network to encode a representation of the environment (in voxel format) on each time step, and an RNN-style network aggregates these step representations into a single vector. The high-level control policy network has two branches that predict which discrete control command to use and which control command parameters. Finally, the dynamics network also has two branches, gets input from the previous level, and predicts the execution time of the command and interpolation. After this, the method checks the output for collision, and the robot moves to the next time step. The advantage of this approach is that it matches precisely the motion commands that the robot is expecting at the correct time. Since the network generates generic motion commands common to many robot controllers, a simple synthetic converter can make these commands into any controller-specific language.

They define two experiment scenarios: dual SCARA (dynamic collision) and machine tending (6D). They use simulation with Improved RRT* to collect training data. For experiments, the authors compare their approach with the original RRT and Improved RRT* methods regarding execution time. The proposed method has an error between the predicted and expected motion of 5% for the SCARA and less than 2% for the machine tending application. In terms of execution time, it is comparable to RRT but faster than improved RRT* (up to 10 times). Their approach differs from usual neural planners because it tries to bridge the prediction with the actual low-level control and reduce the motion latency and, consequently, the collision probability in dynamic settings. The framework is similar to MPNet, where the networks learn a biased sampling distribution. However, it is not sampling points but outputting commands and parameters here.

Quintero-peña et al. [106] introduce a novel neural network architecture to model the sensor uncertainties for motion planning under uncertainties. The motion planning problem under uncertainty aims to find a path where the risk of collision is lower than a given threshold (assuming there are uncertainties in sensor measurements). This work uses signed distance functions (SDFs) to model the distance of the robot links to a point in space as a Gaussian random variable, which models the uncertainty of the sensor measurement. Next, the method uses this information to solve a chance-constrained IK (formulated as a chance-constrained optimization problem) problem where the initial candidate path comes from any motion planning algorithm. Then, it checks each waypoint using the chance-constrained IK for the collision probability threshold.

The authors use a benchmark tool (tabletop manipulation tasks) to evaluate their method with a simple strategy of inflating the measurement points to maintain larger clearance and minimize collisions. They used RRT-Connect as their motion planning algorithm. The results show that their method has a lower-end effector displacement, path length, and considerably lower path risk. Their work shows

promising results encoding the uncertain, which is also done by [92] because when working on real-world scenarios, the uncertainty of measurement and even obstacle motion might negatively affect the solution. However, given that they model the chance-constrained IK as an optimization problem, planning time is magnitudes higher than that of RRT-Connect.

Mikhail et al. [107] approach uses the NeRF [159] concept. This work proposes improving a CHOMP-like algorithm by using neural fields for collision loss function and Lagrange multipliers to enforce the non-holonomic constraints. The proposed method is called neural field optimal motion planner (NFOMP). The CHOMP algorithm uses the signed distance function as the default, which causes problems with long and thin walls and U-shaped obstacles, which might cause the trajectory to get stuck on a local extremum. The authors combat this by using the obstacle neural fields (ONF), which take Gaussian position embedding obtained from Fourier Feature networks as input. The authors collect data along the trajectory and randomly around the environment to build this collision model. Trajectory optimization loss combines collision, path length, smoothing (Laplacian regularization), and constraint (Lagrange multipliers) losses.

They compare their method with existing heuristic-based planners such as RRT, RRT*, and BIT*, among others. They consider standard metrics of success rate, path length, and execution time but also motion smoothness metrics such as cups (number of stops, turns, and abrupt changes of direction), maximum and normalized curves, and the smoothness of the path (Angle-over-length). The results show that NFOMP can handle obstacle geometries problematic to CHOMP (thin and U-shaped obstacles). Compared to the other methods, their approach is superior in path smoothness metrics. However, it falls short in execution time (a weak point of optimization methods) and path length.

Ahsley et al. [108] formulate a novel learning from demonstration (LfD) approach that incorporates the dynamics of demonstrations while avoiding obstacles using motion primitives. The authors combine a hidden semi-Markov model (HSMM) with neural networks to encode and select motion primitive sequences. These encoded primitives and their transition probabilities become a discrete sample space where a random tree search algorithm finds the path. In summary, the HSMM learns the state transition probabilities between motion primitives, and the neural network encodes those primitives in a latent space to increase generalization capabilities. The authors train the neural network using the conditional neural movement primitives framework. The authors combine the motion primitive score (from the neural network) with the transition probability (from the HSMM) to form a probability mass function that guides the sampler (particle filter). The authors validate their approach in a bar-tending task scenario (grab cups and bottles), and the approach could execute this task successfully.

Huang et al. [109] propose a novel framework for autonomous driving (AV), combining scene prediction and motion planning. The key concept is that the network forecasts the future trajectories of the objects in the scene and outputs them to a differentiable motion planner (optimizer) that gives feedback during training to the trajectory forecast network, which in turn might affect the forecasting network result. In this work, the motion planner is a differentiable non-linear optimizer, and the gradient from this planner can also be used in backpropagation to optimize the trajectory forecasting network. The goal is to connect trajectory prediction to motion planning and have them work together to improve the overall trajectory of autonomous vehicles. They assume perfect scene information tracking. The optimizer cost function considers path length (time), traffic rules (staying in the middle of the lane and following lane direction), and safety (distance from other agents).

The authors compare their work with several state-of-the-art approaches such as imitation learning, imitation learning with trajectory forecast, conservative Q-learning, and intelligent driver mode. The authors execute experiments in closed-loop scenarios and open-loop scenarios and execute ablation studies to evaluate if their framework is the correct choice. Their method performed better than the other baseline methods in terms of success rate and accuracy in all experiments. However, since their method uses optimization for motion planning, it is slow and unfeasible for real-time applications.

Osa [110] focuses on learning the solution manifold, where multiple solutions lie through a model representing this manifold in a latent space. Finding multiple solutions is necessary because motion planning as an optimization problem is non-convex, that is, there might be multiple local optima. Usually, motion planning has multiple solutions to the same problem. The author proposes combining a generative model with optimization called Learning the Solution Manifold in Optimization (LSMO). The author uses VAE to map the samples to latent space and then uses the optimization method to find the best samples from the latent space. Any optimization problem might integrate this framework, but the authors propose a motion planning algorithm based on LSMO as a use case. The motion planning algorithm extends LSMO using CHOMP for projection into the constraint solution space.

They experimented with simple function optimization and robotics applications (in both simulation and real scenarios) for evaluation. The author compared its approach with CHOMP, STOMP, and their previous approach, finding that the method generates trajectories considerably faster than all the other baseline methods but requires training, which requires several minutes. Still, training is a clear limitation of this method, given that the other methods do not require training, and this method approach is not reusable for solving other problems.

Collision checking is an integral part of any motion planning pipeline [7]. According to [7], the collision-checking operation takes most of the time in sampling-based motion planning because it checks every sample for validity before connecting to the path tree. Therefore, we must discuss and present some collision-checking works.

Chew et al. [45] present an approach for collision checking. Their work focuses on collision detection using neural networks. This paper has three main contributions:

- 1) A network estimates the minimum clearance (distance robot obstacle) given a robot pose and workspace pose (ClearanceNet);
- 2) A GPU-accelerated RRT algorithm, which is an improvement on the FastronRRT (CN-RRT);
- 3) Definition of extended configuration space.

One of the critical points in the work by Chew et al. [45] is the extended configuration space. In this space, the authors try to model dynamic objects. Some workspace components are static, but others might be dynamic while solving the problem. Therefore, they use this concept to train a neural network to predict the minimum clearance between the robot and the obstacle (workspace or self-collision). Additionally, since they have a neural network, they can batch inference thousands of samples at a single inference pass in parallel, speeding up the collision checking process. Finally, unconnected path portions are connected using the traditional RRT, retaining the completeness of the classical method.

In summary, [45] is very similar to MPNet, which integrates itself into an existing sampling-based motion planner and replaces one component: collision checking. The authors compare their method with two other collision-checking methods, integrating Gilbert-Johnson-Keerthi (GJK) and Fastron into RRT. Their method is far superior in terms of success rate, collision check operations, compute time, and path length.

Koptev et al. [111] introduce a novel data-driven approach for computing distance-to-collision in the joint space for robotic manipulators. Their approach allows for reactive control and dynamic environments. The main goal is to learn a neural representation that encodes the minimal distance between a point in the robot workspace and a robot configuration (each robot link). They generate the dataset in simulation by moving the robot using random sampling, selecting random points, and calculating the distance to each joint. The feature vector is the robot joint angles, joint angles sine, joint angles cosine, and the evaluation point. The network architecture is a standard fully connected layer network. The minimum RMSE is approximately 1 cm, and the STD is 0.81 cm. The network takes less than 1 ms to process 10000 sample batch.

To demonstrate their approach, the authors apply this method to reactive motion planning methods: QP-IK and sampling-based MPC. The results show that their method achieves faster planning times and smaller clearances than baseline methods. Their work proposes a novel

collision-checking approach for motion planning. Depending on the complexity of the environment, collision checking is usually the most expensive operation in motion planning. This approach has promising results and speed. However, the tests only cover simple environments such as spheres and human arms made by spheres. Therefore, it is unclear how scalable this approach is for complex scenarios and working with data such as point clouds.

Yang et al. [112] formulate a CVAE-based collision avoidance method for mobile robots that navigate dynamic environments. The basic idea is that the CVAE network generates a temporary target distribution to deviate the mobile robot path to avoid the moving obstacles despite deviating from the original path/ This strategy is more natural and guarantees that the robot continues moving smoothly to the destination. Human experts create the training data for the temporary targets.

In the experiments, the authors compare their approach with the risk-RRT method, showing that their method is more efficient and has a higher success rate. Their work focuses on a particular scenario for mobile robots. This approach might also benefit human-robot collaboration scenarios applied to 6 DOF robots. One downside is that it assumes information about the obstacles to be precise (position and velocity), which is usually noisy in the real-world.

B. REINFORCEMENT LEARNING PLANNERS

The work by Strudel et al. [113] focuses on obstacle representation and motion planning in this context. It uses RL (SAC) for motion planning and PointNet for obstacle representation. The novelty lies in how they represent the obstacles, using the boundary points and normals. The authors execute an ablation study to verify if their choice of representation is the best one, and indeed, it is a much better representation compared to the internal points. They compare their approach in several scenarios with different state-of-the-art methods, ranging from heuristic to neural, in 2D and 3D. The final comparison shows that their method is better than MPNet. This work does not consider robots but rigid bodies of different shapes, which makes the search space considerably smaller in comparison.

Zhou et al. [114] present a method called residual reinforcement learning. This method consists of a guided training of the reinforcement learning agent. Contrary to original reinforcement learning where the agent does trial and error to learn the policy, which increases the training time considerably and leads to problems in convergence. In this work, the authors define two policies: initial policy and residual policy. The training of the residual policy is done using the initial policy. The initial policy is the seminal MPNet [7] network, which in reinforcement learning taxonomy is an imitation learning method. The residual policy is the PPO algorithm [72]. The role of the residual policy is to compensate or to correct the initial policy action, so the final action is the combination of the initial and residual

policy action. The problem and environment setting is that the robot should pick and place a small green block between two obstacles without knocking them over. For results, the authors compare their work with the original PPO algorithm showing that the reward is higher.

Kamali et al. [115] propose an approach to use reinforcement learning (PPO) [72] to improve the teleoperation of a robot arm. The reinforcement learning method is used to map the trajectory from the human arm space to the robot arm space while avoiding collision and singularities. The scenarios considered are two: reaching and dynamic path following. In the reaching scenario, the robot has to move from one point to another while avoiding obstacles, singularities, and joint limits. The dynamic path following scenario aims to follow the human hand motion mapping the trajectory of the human hand to the robot joint space while avoiding collisions, joint limits, and singularities.

More specifically in the list below we present the Kamali et al. [115] reinforcement learning setup:

- Observations: Robot joint angles, goal trajectory;
- Action: joint angle offset, Δt (time interval);
- Episode: episode ends when the maximum number of steps has been reached;
- Rewards: distance between current joint state and goal state, collision term and joint limit term, respectively;

Kamali et al. [115] execute experiments by collecting demonstration data for both scenarios. The authors compare results with STOMP algorithm [160], and still, the results point to STOMP having a better result than the proposed algorithm.

Li et al. [5] implement changes to the original DDPG [70] algorithm to train for motion planning of a 6 DOF robot. The first change regards memory management, instead of storing every interaction as experience, the authors separate into three experience storage: sampling, collision, and success. The sampling experience storage is composed of random samples of collision and success. The authors use this approach because they claim that learning is more efficient using success data. The second change is how to tackle the problem, instead of training the agent using all the 6 robot joint data, the authors decided to separate into two sets of 3 joints. The processes by which the authors call pre-training where the system focuses on the first three axes (from the base) which mainly account for the robot reach, compared to the last three axes.

In their experiments, the authors compare a traditional approach of training all 6 joints with their approach of separating the joint groups. The results point to higher rewards and fewer steps when using the proposed method. Additionally, the training time is considerably reduced by using the authors approach. The authors used a fixed starting point and selected some endpoints, and the environment has no collision objects (other than the robot self-collision). In future works the authors intend to improve the idea by using random start and goal points and adding obstacles.

Additionally, the work also makes an ablation study to compare breaking a robot into smaller components (2 groups of joints) so the training is faster and obtains better performance in the end.

Tang et al. [17] propose using the DDPG [70] algorithm for the coordinated motion of a dual-arm space robot. The task goal is for the robots to grab a box by its handles (each arm grabs one handle) while in an environment with randomly placed collision objects. The authors treat the opposing arm as a collision object too. The agent controls the joint torques of each robot. The reward function is composed of three components: the end effector moves closer to the goal, avoids excessive torque, and another one rewards small changes in distance when an object gets very close to the target and to increase precision. The authors compare their result with RRT [38], RRT-connect [157] and RRT* [42] algorithms, obtaining better result than all. The start and goal points seem fixed, the only difference is the collision objects placed randomly in the environment.

Zhou et al. [18] develop a curriculum reinforcement learning strategy to train a motion planning agent. The curriculum consists of varying the size of the collision objects based on the success rate of the agent, using the logic that larger obstacles are harder and smaller obstacles are easier. The authors use the concept of the effective sphere to scale the object size. The authors use the PPO [72] algorithm to train the agent and the observation (features) are: the target position, the robot TCP position, and robot joint angles. The authors compare their approach with the original PPO and conclude that the curriculum approach obtains higher rewards.

The work by Ewerton et al. [116] focuses on imitation learning from human demonstrations. More specifically, this paper contribution aims to improve sub-optimal human demonstrations and adapt to changes in the environment. The authors propose a method that uses the Gaussian process model to learn the distribution of trajectories and a reinforcement learning-based method called PRO to optimize the trajectories. This acts as a self-supervised learning based on distributions obtained from the GP.

In the experiments, the authors measure their framework capabilities in different tasks, the authors compare the current algorithm with their previous work. The first experiment aims to assist a human in the teleportation of a beam in a virtual environment, the goal is to go from the start position and orientation the final position and orientation. This task is difficult for humans to estimate the 3D cues from a 2D view. Therefore in the proposed framework, the user performs the task for 10 times and then these demonstrations are used to train the model and the optimization method which then optimizes the trajectory and guides the user. The next experiment focuses on adaptation to dynamic environments. The environment has a wall with a hole in it, and then the algorithm has the starting point, location of the hole, and goal point information. Thereafter the user must collect 3 samples per environment and the algorithm can learn and

optimize trajectory. The final experiment uses the real-world and a 7 DOF robotic arm. Again, the environment information is the start position, goal position, and obstacle positions and then the users must collect demonstrations and the algorithm uses the data to learn, optimize and finally generalize to unseen scenarios. The authors use optitrack motion tracking system to detect the location of the goal and obstacles.

Liu et al. [117] introduce a reinforcement learning based framework for robot pick and place skill acquisition. The authors use a modified version of the PPO [72] algorithm they call MAPPO, which stands for Manipulation Actor PPO. The novelty lies in the design of the reward function, which takes into consideration a desired configuration of objects and the current position of an object using Euclidean distance. The actions are also discrete, i.e., arm init, move, pick, place. The object position data is obtained using an RGB-D sensor (Kinect2 camera). Additionally, the actor also outputs which object it wants to manipulate.

To evaluate their approach the authors design a set of experiments that consist in increasing levels of difficulty: move to the target, pick a cube and place it on top of another cube, sort cylinders by placing them on top of cubes of the same color, and finally the arrangement task of placing cubes on top of each other in a specific arrangement. The authors compare their results with A3C [161] and original PPO [72]. The results point to faster learning (faster algorithm convergence) and higher rewards.

Bhuiyan et al. [4] formulate a novel method for collision-free robotic motion planning. Their method uses the PPO and Actor-critic networks to train a deep reinforcement learning agent. The novelty of the work is that the authors use virtual laser sensors attached to the robot body and tip to gather real-time information about the environment. The observation vector has 13 values corresponding to the robot information: 6 joint angles (radian), 3 values describing the end effector pose, 3 values describing the difference between current position and goal position, and 1 value describing the distance between current position and goal position. The observation also contains 129 values corresponding to the virtual laser sensor measurements. The final observation state is the combination of the robot observation and laser sensors observation. The network has as input the 129 virtual laser sensor measurements and the goal position, totaling 135 as input.

In the experiments, the authors considered two types of robots: Kuka (7 DOF) and UR (6 DOF). They design different experiments for each robot. For comparative results, the authors compared their approach with several variants of the RRT algorithm: NC-RRT and RRT [38]. In all scenarios, their method was faster and had a smaller path length than the classical algorithms. Additionally, the authors observe that the path generated by their method is more consistent, whereas the path generated by sampling-based planners are more chaotic due to the random sampling nature. Figure 19 illustrates the training and inference process using the virtual sensors as observations.

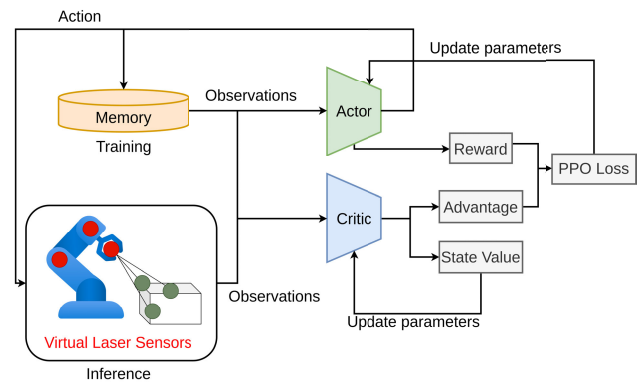


FIGURE 19. Diagram for training and inference in Bhuiyan et al. [4] approach.

Liu et al. [118] present a dual-arm focused multi-agent reinforcement learning framework called: dual-arm deep deterministic policy gradient (DADDPG) which in turn is based on the multi-agent DDPG (MADDPG) [162]. The novelty seems that the robots share observations and outputs, and additionally, the critic has access to all policies. However, the novelty of the work and what distinguishes DADDPG from MADDPG is not clear. The authors execute experiments first in a simulation environment (using Mujoco [163]) and prepare 4 tasks: reach, push, pick up, and gripping pot. After obtaining good results in the simulation environment the authors run the algorithm on a real scenario for the pick-up task. However, in the real-world the detection of the grasp point adds an additional layer of complexity to the problem. The authors use Mask-RCNN to detect the pole and computer vision to segment and obtain the grasping points. Additionally, the authors use a kinetic sensor (RGBD camera) to obtain visual and spatial information. The real robot system is implemented using ROS and even though the coordinate frames are different than the simulation, the real robot could accomplish the task.

Wong et al. [119] develop a Soft actor-critic (SAC) [164] reinforcement learning approach for motion planning and collision avoidance between two robot arms. In their approach they consider each arm as an agent and the environment contains information about both arms. The information that the network has access to deeply describes the current state of the two robots, the target, and the collision information. The network has 55 inputs, as described below:

- 1) Current information of the robot arm (start): position, orientation (quaternion), and redundant angle. Totaling 8 inputs;
- 2) Target information (goal): position, orientation (quaternion), reverse orientation (quaternion). Totaling 11 inputs;
- 3) Collision information: Collision information between each combination of two robot links. Totaling 15 inputs;
- 4) Joint information: joint limits, elbow joint position, wrist joint position. Totaling 13 inputs;

- 5) Arm reach: work reach of the arm. Totaling 1 input;
- 6) Expect angle of each arm joint at the target: joint angles when reaching target configuration. Totaling 7 input.

One of Wong et al. [119] contributions is the training scheme. The authors present three different training schemes: single-arm training (avoid self-collision), dual-arm simultaneous training (avoid collision between arms), and distributed training (improve sample efficiency). Additionally, the authors propose a progressive training method to improve convergence by relaxing the minimum position accuracy of collision and reach, which is basically the curriculum. For the results, the authors compare with an existing dual-arm motion planning method called DMAMP and also execute experiments on simulation and real scenarios. The authors execute three different experiments: completely random start and goal position, completely random start and goal position, but fixed orientation, error-prone zone experiments (use a zone where the probability of collision is high). The authors also present an ablation study to evaluate the effect of each data input on the network, and as a result the network with all inputs had the best performance with the lowest self-collision, joint limits and singularity occurrences. Figure 20 illustrates one of the training schemes to avoid collision between arms.

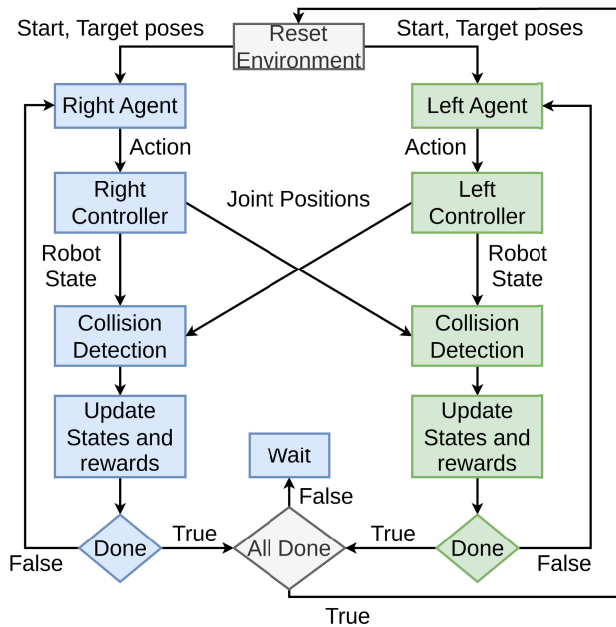


FIGURE 20. Training to avoid collision between arms in Wong et al. [119] approach.

Prianto et al. [120] propose a reinforcement learning based motion planning framework for a dual (or n arms) robotic arm using the Soft Actor-Critic method. The work considers two or more arms and treats them as a virtual single-arm, so the algorithm outputs a set of joints for both arms. The state is the current robot joint state and the action is an increment to this state, so the next state is the sum of the current state with the action. The episode continues until the robot has reached the goal state (there is a tolerance

value, so the goal state is relaxed and not exact), or the maximum number of states has been reached. At each step, the reward is constantly -1 and when the arms reach the goal state, the reward is zero. Therefore, the objective is to reach the goal state with minimum time steps. This is a sparse reward problem, so the authors decide to apply SAC [164] and HER [74] methods to overcome this limitation. The authors apply their approach to two open manipulator arms (3 DOF) and setup an environment with fixed obstacles. The authors also compare their results with the TD3 method, and also compare the path length and smoothness with TD3 [165] and PRM [39], showing that their method obtains superior results. Figure 21 illustrates the training and inference methods.

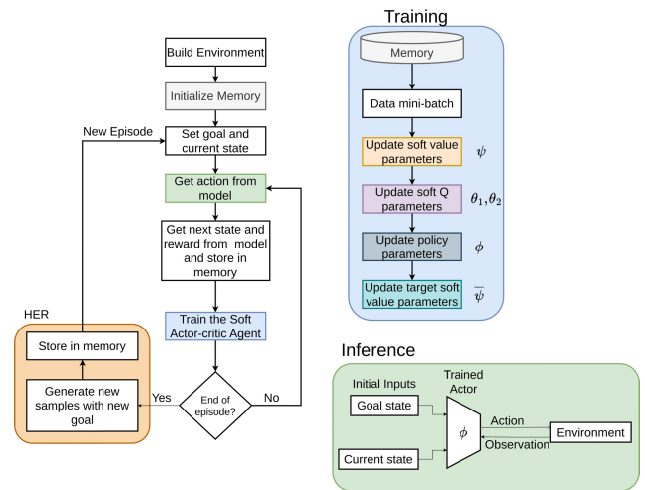


FIGURE 21. Training and inference diagrams for Prianto et al. [120] approach.

Prianto et al. [121] presents a reinforcement learning based framework to control dual robotic arms in an environment where there are periodically moving obstacles. The authors employ the SAC [164] and HER [74] methods to train the agent and also introduce information about the obstacle position to the observation. This work is very similar to the main author previous work [120] which applied SAC [164] and HER [74] for learning collision-free and optimal motion planning with fixed obstacles. Similarly, in this work the authors consider the dual-arm as a single virtual arm, so they do not need to consider complex environment modeling for each arm (since each arm might be collision to the other arm).

The main difference between this and the previous work is the addition of the periodically moving obstacle. The framework tracks the obstacle position and saves a buffer of positions of size d and this is available as the state information. One of the neural networks predicts the future location of the obstacles. The results point to a high success rate (but still lower than previous work) and rewards. The authors also do an ablation study to compare the impact of the size of the buffer to store moving obstacle positions. Which points out that more information about the moving obstacle,

the better the result. Finally, the authors compare their results with other methods, namely PRM [39] and TD3 [165].

Park et al. [122] introduce a novel method to control dual-arm robotic systems under moving obstacle scenarios. The authors use SAC and HER algorithms to solve the motion planning problem. This paper tries to improve previous work of the same research group [120], [121]. The robot arms are considered as a single entity, not separate agents. Additionally, to predict the future positions of the obstacles the authors use an LSTM [166] network that given current and previous positions of the obstacle, can predict several time-steps in the future where the obstacle might be. The obstacle movement is periodic.

For experiments, the authors use the same set of 3 DOF arms. Additionally, execute experiments in both simulation and real-world scenarios. The authors compare this approach with their previous approach. The clear difference is the use of the LSTM to predict obstacle position. Given that the results did not show a significant difference in final reward or success rate compared to the previous approach, the LSTM helps in the convergence, the agent converges to maximum reward much faster when using LSTM. Figure 22 illustrates the diagram for training and inference using the agent and LSTM.

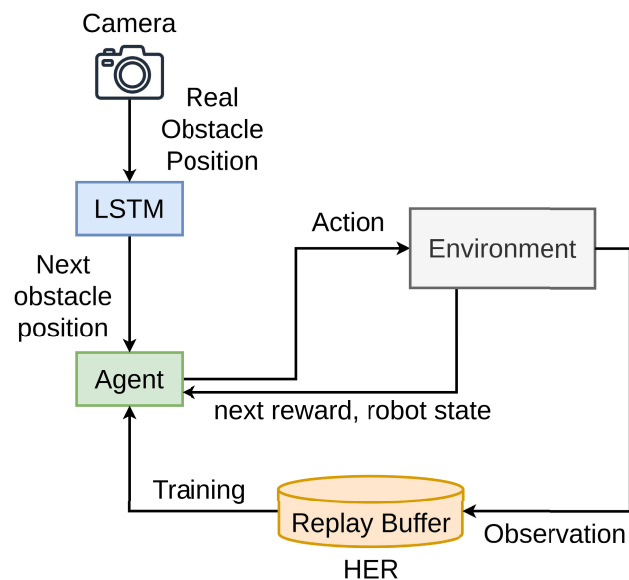


FIGURE 22. Training and inference diagram for Park et al. [122] approach.

Jia et al. [123] develop a framework for collision-free motion planning using a 6 DOF robotic arm. In their approach, they use the reinforcement learning method Q-learning [69] to learn a policy that moves the tip of the robot from start to goal while avoiding obstacles. Their approach only considers the tip of the robot for the Q-learning algorithm, so the State is defined by a set of 3D coordinates of the robot arm tip, the action is a delta over the previous state (positive and negative differences on the tip position). The reward function considers the distance between the tip and

the obstacle and also has an extra punishment to encourage detour where if the last action hits an obstacle, and the next action does the same there is an additional punishment. Additionally, the authors propose assisted learning using the distance between the robot arm and the target, to encourage it to move towards the target. After the Q-values converge they also prepare an algorithm that takes into consideration the tip position and uses the possible inverse kinematics solutions to find the joint angles to move the robot to that position.

The authors design two scenarios for the experiments: a simple and a complex scenario. Both were implemented on simulation. The main difference is how difficult it is to reach the target, in the simple scenario there is more room for the robot to move to reach the target, but in the complex scenario, there is a narrow tight path to the target. During training the authors compare the training without their proposed assisted learning method and with it. In summary, the assisted learning approach speeds up the training and the proposed framework can solve the problem.

Ashvin et al. [124] implement a goal-conditioned reinforcement learning framework to learn policies for executing general-purpose skills. The authors employ a VAE (β -VAE) [167] to learn representation of the image input. Both goals and state are obtained through the β -VAE, i.e. both goal and state are a latent representation of the raw input image. First the authors train the β -VAE by executing random policy and collecting observations. With the trained β -VAE the authors train the TD3-based algorithm, and finally to improve representation β -VAE is fine-tuned with both random policy and state observations during the TD3 training. The reward function is the Mahalanobis distance in the latent space. To further improve the sampling efficiency the authors generate new goals by sampling from the latent space. This approach is similar to HER, but it surpasses HER because HER is limited to states seen along the trajectory, while their approach might be any goal in the latent space. In the experiments, they use several simulated environments in Mujoco software [163]. First, the authors compare the representation learning aspect of their approach with other methods, namely DSAE, HER [74], ORACLE, and L&R. Next they compare different reward functions and relabeling strategies. The authors also implement their system on a real-world robot and execute the pusher and reacher tasks.

Zhang et al. [125] introduce a novel sampling-based motion planning (SBMP) method for dynamic environments using graph neural networks (GNN) in two stages (The authors use two GNNs). In the first stage (first GNN), the approach encodes the global information by having the sampled graph in the configuration space as input, and its goal is to encode the graph structure using attention mechanisms. Additionally, they incorporate the obstacle trajectories using the idea of temporal encoding and neural radiance fields (NeRF) so that the network can capture temporal patterns from high-frequency inputs. In the second stage, the approach incorporates the robot's current state, the

local graph structure, and the time-shifted obstacle trajectory to generate the policy for the robot. Figure 23 illustrates the 2 stages of the method. They use the imitation learning and DaGGER methods to train the agent with SIPP method as an oracle.

They compare their method with the original SIPP and Dijkstra-H (follow the shortest path) algorithms for experiments. The advantage of the proposed method is the reduction in collision check operations. However, regarding success rate and path time ratio, SIPP is better for randomly generated and complex problems. Additionally, the authors executed an ablation study on the encoding (global, local, and temporal), showing slight metrics improvements. This work shows the capability of GNNs to encode the sampled environments and learn safe paths. Despite considerably reducing the collision check operations, it still lags behind the classic methods regarding success rate and path length/time. The encoding slightly improves the evaluation metrics, which need considerable improvement because the authors dedicate half of their method to encoding. Also, the authors assume a periodic and known obstacle trajectory, which is not always the case.

Wahid et al. [126] addresses the problem of object navigation, which consists of moving an agent to the location of an object with a given label in a complex and unstructured environment. The authors propose a single end-to-end Distributed Soft Actor-critic architecture using Resnet + LSTM to learn the policy. This work focuses on 2D motion planning, not arm motion planning (6D or more). The authors parallelize the data collection on multiple agents in simulation environments and then combine the data to train the model. The reward formula considers collision, number of steps, the distance towards the goal, and success in reaching the goal, i.e., $R(s, a) = \text{collision}(s) + \text{step}(s) + \text{progress}(s, a) + \text{success}(s)$. They consider all the components for motion planning, such as collision, number of nodes in the path, and distance to the goal. The difference is that the agent has an egocentric view of the world in RGB and LiDAR (1D array). The authors also provide the action of the previous step and whether it was a collision or not, which speeds up convergence. Also, the authors conduct experiments with an oracle observation that is a binary mask of only the goal object, which divides the process into two steps: exploration while the object is not visible and navigation and collision avoidance when the object is visible. The LSTM model gets the concatenation of all observations as input.

For experiments, they compare different types of observations with different policies: human control, Roomba policy (move straight until collision, rotate, move again, and repeat), and topograph traversal (build a graph of the environment before navigation). The results show that despite having a success rate inferior to human control, it is better than the other automated approaches in all scenarios. They also obtain high success rates by combining RGB, LiDAR, and Oracle observations. They observe some interesting emergent behaviors, such as scanning the room by rotating and peeking

into rooms before entering, and some not-good behaviors, such as over-exploration. For future works, they plan to integrate transformers for better long-horizon planning. This work focuses on 2D planning conditioned with image and lidar data. In terms of motion planning problem complexity, it is not a very complex problem. However, exploring unknown environments guided by vision complicates the problem due to semantics.

Chen et al. [127] formulate a reinforcement learning approach for motion planning in dynamic environments. The authors use a 7 DOF panda arm in simulation and apply the DDPG and SAC algorithms to solve the problem. The authors do a simple ablation study to define which kind of observation is better, and they point out that instead of using direct joint angles position and velocity, the end effector position and velocity (fixed orientation) seem to have better convergence and accuracy. Additionally, the authors define goals and obstacles (position and velocity) relative to the end effector. They assume perfect conditions from sensors regarding obstacle detection. The possible actions are Δ in the end effector x, y, and z coordinates. The reward is the negative of the distance between the end effector and the goal combined with the closest distance between the end effector and obstacles. Their results show that SAC has a better accuracy and safe rate (measure of collision). The SAC method reached an average success rate of around 80% and could plan collision-free after 5000 training episodes. The authors plan to study a sim-to-real in the future. This method shows promising results for moving obstacles and multiple manipulators. Additionally, the study on which way to represent the environment leads to a better and faster result in learning is very important for the context of reinforcement learning and its known data efficiency problems.

Kulathunga et al. [128] evaluate how to combine classic deterministic motion planning algorithms with RL approaches. This paper compares the optimistic planning of deterministic systems (OPD) approaches of uniform look-ahead tree searching with Q-learning and PPO approaches. The authors use an OctoMap representation. In the results, the authors show that PPO performance is below the deterministic tree search and the uniform look-ahead tree search despite execution time being four times faster. Given the results and problem formulation, this work discretizes the 3D environment and searches. There is no sampling method; thus, it is not scalable because A* and similar methods are satisfactory for 2D planning but not for 6D planning.

Gu et al. [129] introduce a novel method that combines RL and a heuristic method for motion planning. The rationale behind the approach is that autonomous vehicles might have macro information but lack detailed information in some critical parts of the environment for safe and precise motion planning. Therefore, the proposed method combines a macro scale planner using the topological path search (TPS) heuristic method, followed by an RL (Q-Learning) method for micro-scale detailed planning and finally, a Trajectory

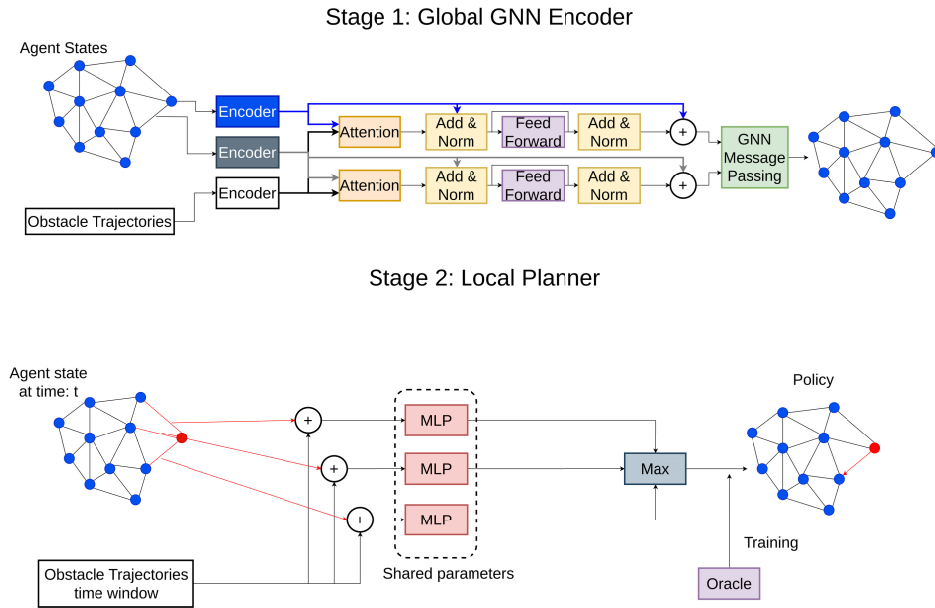


FIGURE 23. Diagram of the two stages of Zhang et al. [125] approach.

lane model to smooth the trajectory and make it feasible for the vehicle. They consider uncertainty, dynamic, safety, and smoothing constraints in formulating the RL problem. Uncertainty constraints relate to the environment and the position and shape of obstacles. Dynamic constraints relate to the vehicle model. Safety constraints relate to the minimum distance between vehicles and obstacles. Smoothing constraints relate to trajectory smoothing. The TPS method constructs a graph of the environment with paths and obstacles. Then, the RL method is applied in smaller portions to output a finer collision-free path. Finally, the TLM method smooths out the trajectory from the TPS and RL methods. Using this approach, the authors prove that it is faster than simply applying RL to solve the problem, and the method could deal with uncertain environments.

The authors compare their method with a modified A* that guarantees kinematic feasible trajectories, showing that the proposed method outputs shorter paths. Combining the strengths of data-driven approaches with the completeness and exactness of heuristic methods is always interesting. This work achieves considerable speedups by reducing the scope of the RL algorithm to smaller areas, thus simplifying and speeding up the processing.

Zhang et al. [130] present a receding horizon reinforcement learning kinodynamic motion planning (RHRL-KDP) approach that accounts for the inaccuracies on sensors and dynamic obstacles. The authors use an actor-critic architecture with time as additional input. They augment the actor with a Hamilton-Jacobi-Bellman (HJB) equation with potential field realization for the safety constraints for better adjustment to new and dynamic environments and uncertainty modeling. For experiments, they model a vehicle and its physics in Matlab and compare them with RRT*, MPCC,

and Safe RL methods in online and offline learning settings. The results show that the proposed method has a better cost, completion time, and average speed while having the lowest max acceleration in all scenarios.

Yu et al. [6] develop a Q-learning method that learns adaptive motion planning from human kinesthetic demonstrations. The method consists of building a library of basic motions from human demonstrations, and the RL agent can generate the motion for the task if the task motion is within the motions in the library or ask for additional help. First, the authors define the syntax for the task definition, and then they build a library for motion and define a motion feature similarity measure. Finally, the authors formulate this problem as an RL problem. They define the motion features as screw transformations. The task is a sequence of critical robot configurations. The motion similarity criteria compare the dual quaternion representation of the relative motion stored in the library and the task definition. The RL agent replaces an exhaustive search of task definitions in the library. Their approach is effective, yet it does not seem to generalize. Every time the agent finds a sufficiently difficult scenario, it asks for human demonstrations and its memory increases. Subsequently, speed decreases because the library grows exponentially. However, it is effective for the specific tasks it was trained to solve.

Zhou et al. [47] formulate a discrete SAC algorithm for indoor service robots motion planning in a dynamic environment. The main contributions are in the new discrete sac algorithm using an attention-based LSTM that encodes the robot obstacle features in time and an improved data replay that combines online and offline data. The state of the robot and the obstacles has an observable part (position, velocity, and radius) and a hidden part (goal position, preferred speed,

and heading angle), so the robot must plan according to the moving obstacle observation, and the obstacles move the same. The main objective of the robot is to minimize time to the goal without collision, so the reward function considers this objective. Regarding the network architecture, the authors used one LSTM for the actor and one for the critic, and they share the same attention network. The replay data combines offline and online data to ensure data diversity for training using cosine similarity. Add that data to the training if the similarity is smaller than a threshold. The authors used the ORCA simulator [168] for experiments containing two scenarios: circle and square crossing. In the experiments, the number of obstacles and replay data configurations vary. The results show that their method outperforms the state-of-the-art algorithms (learned and heuristic) and that the replay buffer configuration contributes considerably to the success rate.

Huang et al. [131] propose an RL approach for motion planning of redundant space robots. This work combines RL with null-space motion (joint movement while maintaining the end effector position) to avoid obstacles. Additionally, the authors employ curriculum learning to train the RL agent. In this approach, the robot motion consists of two components: the end-effector, the motion of the base, and links, while the maintaining end effector poses (null space). This motion decomposition is necessary because, in space, the base and the joints have a couple of relationships (the motion of one affects the other). Therefore, to compensate for this, redundant manipulators are necessary. Otherwise, the end-effector pose could not be maintained. The framework uses SAC RL to control the motion null-space components, and the motion equation generates the end effector trajectory. The RL policy and end effector trajectory results are combined to move the robot. As for training, the authors employ a curriculum strategy that comprises positioning obstacles at random (standard behavior) and at a safe distance to trigger collision avoidance motion. The authors also change the end effector pose during training if the agent is performing well to increase the data variability.

In the experiments, the authors verify that despite trained in a single obstacle, the policy generalizes to multiple obstacles. The authors also consider moving obstacles, and the agent can adapt. The agent achieved a success rate of 91.4%, the same as the current state-of-the-art algorithm (gradient projection method - GPM) but three times faster. This application setting is quite different from the standard motion planning setting. The whole environment moves relative to each other at different velocities and accelerations, and the robot base is not stationary, so the robot's motion might also change the base's motion. The authors only tested on a 2D plane, but results indicate that the framework can support higher dimensions. The authors also consider perfect sensing of the obstacle trajectory and position impossible in real-world scenarios.

Bai et al. [132] present a novel method that combines the Artificial Potential Field (APF) with reinforcement learning (SAC). The reinforcement learning method observation is

the current robot state, target position, and environment information (PointNet [93] encoder). The action space is a motion increment. The reward function is related to the goal position and obstacle collision.

The authors employ curriculum learning to train the reinforcement learning model. The authors employ the APF method when the robot reaches a certain distance to the target pose to avoid the local extremum problem and other limitations inherited from APF. They compare different instances of the SAC method, such as original SAC, SAC with HER, curriculum learning, guided trajectory with original APF, and their method (SAC + curriculum learning + APF). The results show that adding the APF considerably increases the success rate in higher resolution (0.01m). Additionally, the authors compare their method with RRT-connect, showing that despite a lower success rate, it has a lower running time and path length. Since they are using APF, their method can avoid obstacles in real-time. Finally, the authors execute sim-to-real without fine-tuning, and the manipulator can avoid static and dynamic obstacles.

Combining the global planning of the SAC method with fine detail and the dynamic response of the APF method is a promising direction that combines the strengths of both methods. Still, the authors did not achieve higher success rates when compared to heuristic methods. Therefore, this approach requires further refining. Figure 24 illustrates the method using two depth cameras for environment data acquisition.

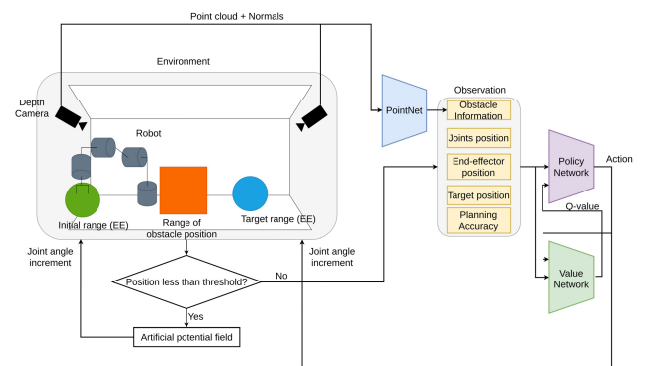


FIGURE 24. Diagram of Bai et al. [132] approach.

Luo et al. [133] develop a novel data collection process for RL. The method is named self-imitation learning by experience plus (SILP+). In this approach, the authors use RL for motion planning and a PRM-based planner to store collision-free robot configurations. The idea is to re-utilize the data obtained in unsuccessful episodes (in this case, when the agent reaches the maximum number of steps) to generate more data in that environment by planning from one collision-free node to another, and the path contains only previously visited collision-free nodes. Since the nodes are collision-free, they skip the collision-checking procedure, reducing data generation time. The paths that this PRM-based approach generates serve as an experience

replay. In this work, the authors represent the obstacles as bounding boxes. In summary, this approach balances the RL exploration capabilities and improves its sample inefficiency with imitation learning. The authors also propose a Gaussian process-guided exploration of obstacles to improve state sampling around obstacles based on existing collected data that guides the agent to the highest reward area.

They experiment with different RL strategies, such as SAC, DDPG, and Behavioral Clone (BC), and experimented with data efficiency improvement methods, such as SILP and HER. Their experiments show that their method contributes considerably to the success rate of the trained model and reduces the training time. The authors also evaluate different ways of dealing with collision during the episode: terminate the episode, and choose another random collision-free state (discard collision information), choose another random collision-free state (add collision information to experience buffer). The latter approach (add collision information to experience buffer) yields the best result. Also, the authors evaluate using the Gaussian process-guided exploration to improve training time and number of collisions during training. Finally, the authors evaluate the sim-to-real performance of their method and prove that it does not degrade.

Lawson et al. [134] implement a transformer-based RL approach for robot control. They use a PRM-guided method for robot skill discovery. This approach uses a hierarchical reinforcement learning (HRL) method that uses different policies acting at different levels of abstraction. The core concept of this paper is return-conditioned reinforcement learning, which is a reinforcement learning problem posed as supervised learning that, instead of simply returning an action, returns an action conditioned to an expected reward. The authors extend a method called decision transformers (DT), which applies transformers to conditioned RL scenarios. They use the output graph of the PRM as a goal for training RL policies in a goal-conditioned fashion. The authors also provide external information, such as camera frames or occupancy grids, to the agent. One of the points to note is that the whole trajectory is output by using transformers since decision transformers consider the RL as a sequence modeling problem. One fundamental change the authors make to the decision transformer relates to the return-to-go, which the original DT assumes to be constant.

However, the authors want to train on a multi-environment and trajectory data. Therefore, the authors try to learn the mean of the undiscounted goal-directed value function, which can learn the return-to-go by parameterizing it as a separate neural network. The authors also treat the distribution shift case, where a trained policy deviates a little at each step, causing failure in the end, by recovering from failure using PRM and then adding two new trajectories to the dataset (failure and recovered) so they can perform additional learning. They compare their method in different simulation environments, from 2D to 23D (humanoid), and test it on a real scenario

using a turtle bot. The baselines are Control transformers, behavioral cloning control transformers, and planning-guided fine-tuning control transformers. This approach shows an increase in the success rate to 26%.

C. LARGE LANGUAGE MODELS IN MOTION PLANNING

In recent years, the use of large language models (LLMs) as an assistant for writing text and code and even as an information search engine has become ubiquitous because they provide a natural language interface for retrieving data [80]. We naturally see approaches trying to use LLM knowledge for other domains, and it is no different for motion planning. Therefore, we ask the following question: Are LLMs capable of understanding the geometrical constraints necessary for motion planning and providing control commands for the embodied agents?

To address this question, we categorize the approaches into two main groups: LLMs as high-level planners and multi-modal LLMs performing high-level planning and low-level control [20], [21]. In this context, high-level control refers to the LLM guiding the robot in a human-readable manner, such as moving to the table, picking the bottle, or going to the sofa [19], [135]. This type of control aligns more closely with human thinking, in that we do not think on the details such as exact motion or muscle contractions we are going to execute, but we plan the actions at high-level. Conversely, low-level control involves the robot joint angle and motor control, which is more technical and less intuitive [11].

The TAMP field greatly benefits from LLMs' capabilities as task planners because LLMs can perform long-horizon planning. This approach uses LLMs as zero-shot task planners by doing prompt engineering, and other models or classic methods execute low-level control.

LLM3 [135] introduces a novel framework that utilizes zero-shot GPT-3 for task and motion planning. Here, the LLM understands the task and devises a high-level plan, while a motion planner executes the low-level controls. The innovation lies in the closed-loop connection between the LLM and the motion planner, enabling the latter to provide feedback upon failure, prompting the LLM to replan. Figure 25 illustrates the diagram of the LLM3 framework. For experiments, the authors propose a box-packing task in simulation and real life, where picking and placing are two possible actions. Pick receives as a parameter a box color, and place receives as a parameter a location in the 2D plane with a rotation.

The approach by Hallgarten et al. [136] applies LLM to the self-driving car scenario. In this work, the authors propose a new set of challenging driving scenarios for benchmarking new methods. The authors also compare several methods in this new set of scenarios, including their method that combines LLM with a rule-based planner. We want to emphasize the comparison between a fine-tuned LLM-based planner that outputs the future waypoints and the hybrid planner, where the LLM chooses among a discrete number of

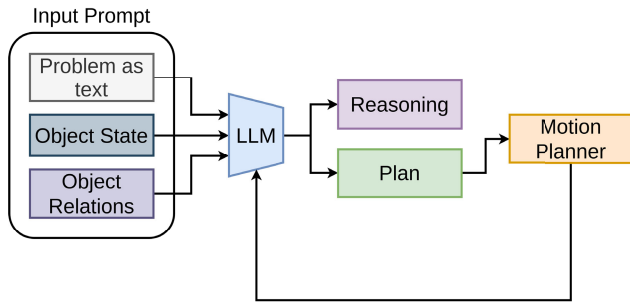


FIGURE 25. Diagram of LLM3 [135] showing components of the prompt.

actions that a rule-based planner executes. In this comparison, the hybrid planner has a considerably higher score, using the same base LLMs.

LLM-Assist [137] is an approach to apply LLMs to self-driving cars. In this work, the authors propose using the LLM to assist a rule-based method. When the rule-based method scores are below an acceptable threshold, the framework invokes the LLM to create a new trajectory (unconstrained approach) or improve the rule-based trajectory parameters (parametrized). In their experiments, the authors confirm that using the parametrized LLM approach improves the rule-based results.

Text2Motion [138] is another approach in TAMP for controlling robotic arms for pick and place tasks. Similarly to LLM3, Text2Motion uses LLMs for high-level semantic planning and pre-trained policies for low-level control skills. The LLM has access to the instruction, symbolic states, and available skills and outputs a plan and the method uses the plan to execute the specific skill policies to change the states of objects to complete the instruction. Figure 26 illustrates the Text2Motion approach.

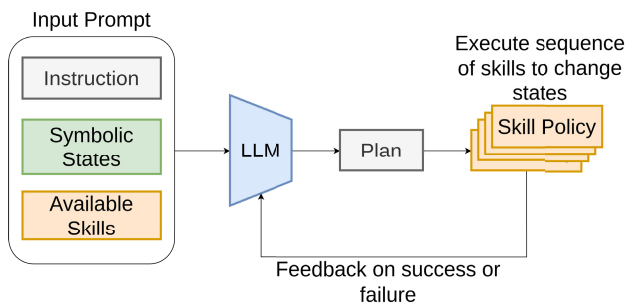


FIGURE 26. Diagram of Text2Motion [138] approach.

Ouyang et al. [139] apply a framework similar to LLM3 and Text2Motion for quadruped robot control. However, instead of using a single LLM for semantic task planning, they use three different LLMs for task planning, parameter calculation, and code generation. Finally, an RL policy executes the low-level controls for each discrete task. Figure 27 illustrates the diagram of Ouyang et al. [139] approach.

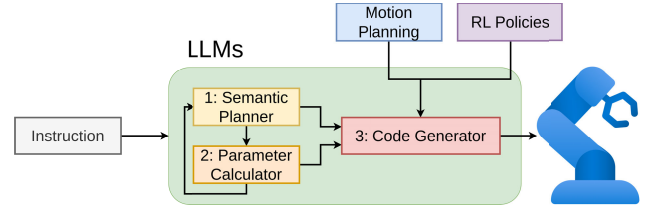


FIGURE 27. Diagram of Ouyang et al. [139] approach.

Inner monologue [140] and SayCan [141], similar to Text2Motion, is an approach in robotic manipulation applications. Both models work similarly because the LLM role is task planning, and a learned policy executes the low-level control. The difference between the two works lies in how they incorporate LLM into the approach. Inner monologue uses closed-loop language feedback during the execution of the task, which might be scene description, questions, or success detection. On the other hand, SayCan uses an open-ended approach where the authors train a model to predict the success rate of the LLM action output and combine it with the LLM score to choose the best action. Figures 28 illustrate an example of Inner monologue [140] and Figure 29 presents a diagram of SayCan [141] where the LLM and a value function work together to judge affordances of tasks based on human instructions before actually executing it.

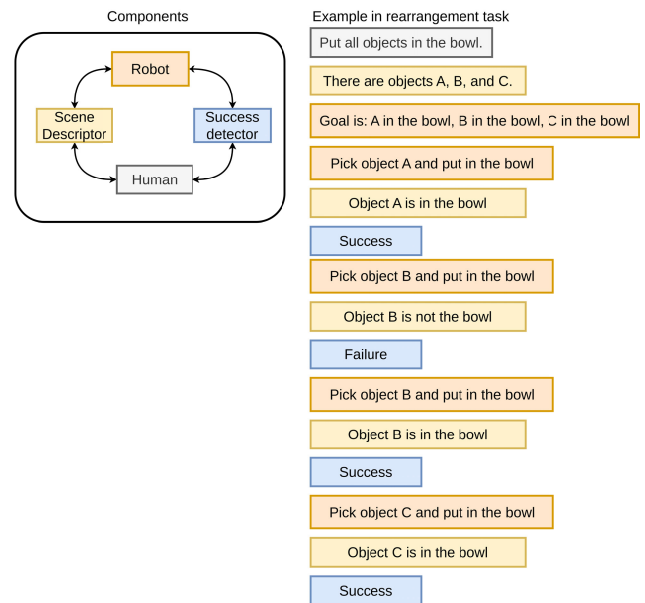


FIGURE 28. Example of Inner monologue [140] executing object rearrangement task.

Other approaches use multi-modal LLM models, which can understand textual and other data types, like images [20], [21]. These approaches might theoretically execute both high-level planning and low-level control. However, these models need fine-tuning before usage and, unlike the previous group, cannot be run zero-shot.

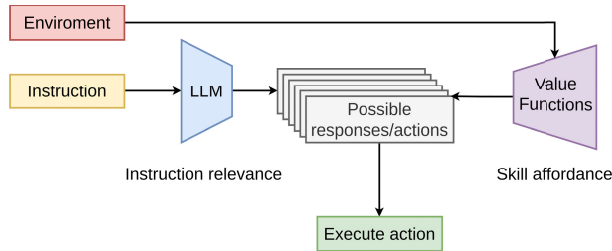


FIGURE 29. Diagram of SayCan [141] approach.

LMDrive [21] uses a multi-modal LLM with text, multi-view RGB, and lidar inputs. The authors evaluate different LLMs as the backbone of their system and verify that LLaVA [169] has the best performance. Their approach can predict the following waypoints from the vehicle using text and sensor information. The LLM is pre-trained, but the image and lidar data tokenization networks need fine-tuning.

Robot Transformer (RT) architecture is a family of models that aim to achieve language-conditioned robot arm control. RT does not necessarily use an LLM in its first iteration but incorporates components with an LLM scale size (35 million parameters) [19]. Its subsequent iterations also use LLMs. RT-1 [19] takes images and natural languages as inputs and tokenizes them to input in a final transform model that outputs a discrete set of actions. This discrete set of actions directly controls the robot arm, gripper, and base. The authors prove the model generalizes to new natural language instructions, objects, and backgrounds.

The RT-2 [20] extends RT-1 using a multi-modal LLM. The inputs and outputs are the same as the RT-1. However, they replace the RT-1 transformer network with a fine-tuned vision-language model (PALI-X and PALM-E) and jump from 35 million parameters to 55 billion and 11 billion parameters, respectively. The jump in the number of learnable parameters improves generalization for unseen objects, backgrounds, and environments.

RT-X [142] authors have proposed a partnership between university laboratories to collect data with different embodiments performing different tasks to increase the variability of the data. The RT-X effort aims to evaluate how RT-1 and RT2 performance can increase with more data and to prove if the generic model is better than a specialized one. Still, generalizing between embodiments and new tasks is an open question in the literature. RT-X is not a novel method but a study of how RT-1 and RT-2 behave with different embodiments and problems.

Chen et al. [143] propose a novel method called NLMap. This method enables agents to execute a pre-exploration of the environment to identify objects and context and then propose a series of high-level actions based on the exploration. Previous methods cannot do exploration outside of the surrounding scene. Therefore, the authors propose extending the SayCan approach by providing a language-queryable spatial semantic representation using Visual language knowledge distillation (ViLD) and Contrastive language-image pre-training (CLIP). SayCan apply learned value functions of

skills based on LLM high-level planning. However, it requires the known location of the objects in the scene. NLMap tries to overcome this limitation by introducing the exploration and spatial semantic representation to add object location knowledge to SayCan. Their consists of executing an initial exploration phase of the unstructured environment and applying ViLD and CLIP to encode the image information with an estimated image location using the 3D sensor. Next, the method extracts the human instruction's object names and uses them to query the representations generated in the exploration phase. Then, it uses this information to augment the SayCan model with the dynamic locations of objects and navigation locations. However, SayCan policies focus on specific goals and objects in mind, so NLMap cannot fully generalize them but only augment them to include a certain level of dynamical behavior and object generalization.

Yilun et al. [144] present a method for long-horizon planning using vision-language and text-to-vision models. The model takes the long horizon task and the current image observation as input and outputs a video plan with a language description of the final task. Goal-conditioned policies on video can convert these plans to robot control. Their approach has three main components: Policy, dynamic model, and heuristic function. First, there is a video language model that generates the low-level states from start to goal conditioned on the starting state and language task description. Second, they have another video language model to generate a heuristic conditioned on the starting state, task description, and goal (generated in the first step). This heuristic predicts the number of actions to reach from the start state to the goal state (They use a fine-tuned PALM-E model). Finally, a VLM searches over the space of possible abstract actions conditioned on the goal state. Their method predicts possible future actions and goals based on the initial state and task description. It uses several VLM to execute specific tasks, such as generating low-level actions, predicting how many actions are left to reach the goal, and searching the action space for the best action sequence. They compare the current state of the art in this area and prove that their work has the highest success rate compared to RT-2 and PALM-E. Still, the authors acknowledge that the image data is insufficient to capture the entire 3D state or accurately predict dynamics and physics in specific tasks. Hence they propose using larger models (better dynamics), multi-view (3D state), and reinforcement learning feedback (physics) to improve on these limitations.

VI. CONCLUSION AND FUTURE DIRECTIONS

In Section V, we explored several works in three main groups: neural planners, reinforcement learning planners, and LLM planners. We explored each paper's contribution in detail and discussed its limitations and possible improvements. During this process, we identified some gaps and open investigation paths that are common to the field. This section explores these gaps and points to possible paths for future work.

A. DATA REPRESENTATION

The first point we discussed is the most critical in most of the works we present in our survey: data representation. To use machine learning methods, one must extract features from raw data. ML methods use these features to learn patterns and thus generalize them to unseen data. Furthermore, machine learning methods do not deal well with values that have a significant variation, hence normalization and compression are standard steps in ML data processing pipelines. Therefore, how to represent the environment, robot configuration, and constraints is vital to the success of an approach.

The planning environments may be as simple as a few objects on a table or as complicated as a street with people walking around. The robot embodiment may be a simple point robot or a humanoid robot with over 40 degrees of freedom. So, the first step is to represent this data type in a shared space where the ML approaches can learn patterns effectively. All the neural approaches in this work use some data representation (e.g., CAE, VAE, Embeddings).

Furthermore, the data type also limits and affects the learning process. For example, most works use point clouds to represent 3D environments because any 3D sensing device outputs point clouds without extra processing. In contrast, Voxels and OctoMaps [170] require additional processing to convert from point clouds. Finally, fully connected neural networks are capable of processing point clouds [93]. At the same time, Voxels and octomaps require 3D convolutional neural networks for feature extraction, which are known for being slower and more resource-intensive.

Motion planning and task planning require an understanding of the environment. However, environment data representation is still an open research question, but methods such as PointNet [93], PointNet++ [150], and Point transformers [171] present compelling results that make them good candidates for point cloud feature extraction. We see this pattern in many of the works we explore here [36], [86]. Similarly, there are methods such as PVNN [172] to extract features from voxels and octomaps [173]. However, few works apply voxels to represent the environment [27], [28].

Another topic related to data representation is which type of data to use. Our survey covered works that use only the joint positions [101], others that use a simplified representation of the obstacles [85], and works that use both robot joint positions and obstacle representation [7], [86]. It is clear that, by only using robot joint information, one might not expect that the approach could handle unseen obstacle positions. Therefore, a fusion of several data types is necessary to improve the generalization capabilities of the models.

B. DATA, METRICS, AND COMPLETENESS IN AI PLANNERS

The second point is related to data generation. Data generation in robotics is expensive, that is, it is expensive to get robots and leave them running for extended periods. So,

simulation plays a vital role in any robotics work. However, there is a lack of benchmark datasets for robotic arm motion planning. In contrast, there are benchmark datasets for autonomous vehicles. Each work defines a set of test environments and chooses an embodiment to evaluate the proposed method. This approach makes it challenging to compare works since there is no common baseline in which researchers might test their approach against others.

In the context of result comparison, standard metrics such as path length, time, and path cost do not cover certain aspects of the environment. The complexity of the planning environment is never covered. The density of obstacles in an environment and the number of possible paths affect the other metrics, thus, we believe it would be interesting to consider a metric that could better reflect the environment's "difficulty."

Within the works we presented in our survey, a significant portion aims to improve on existing heuristic methods such as SBMPs. However, an equally significant portion does not show sufficient improvement to justify the cost of creating data and the additional cost of using heavy neural network architectures. Additionally, the lack of exactness in neural networks does not contribute to a reliable solution, which requires a fallback to the heuristic method in the worst cases. However, neural networks are orders of magnitude faster than existing heuristic methods and have the advantage of parallel processing by using batch inference, which are good points for collision checking and sampling tasks [7], [86].

Completeness is a central requirement in motion planning. Finding a solution when it exists and pointing out that no solution exists is paramount for the motion planning method to be minimally reliable. However, neural networks are approximators and are not as exact as other heuristic methods. Therefore, some strategies, such as using bidirectional trees [7], falling back to heuristic methods in a worst-case scenario, and setting a maximum number of attempts before returning that a solution was not found, are feasible adaptations to improve the reliability of neural planners.

C. LLM PLANNER ANALYSIS

Since LLMs are such a novel topic, and it is challenging to compare with other types of approaches in our survey, thus we analyze and discuss them separately from the other two groups. LLMs are a good fit for the task and motion planning problem, given the long-horizon sequence prediction capabilities. They execute the role of task planner without requiring additional fine-tuning, and with additional fine-tuning, they are able output low-level controls in the task and motion planning setting. Additionally, LLMs offer natural language feedback and explainability for the task they are executing [174], [175], [176].

However, LLMs require fine-tuning to the embodiment they are supposed to control to output the low-level controls. To fine-tune an LLM, one requires an astronomical amount of data. Still, the network does not generalize to different embodiments and widely different task settings. It might

generalize to different objects and scenes similar to the training examples. For example, RT-1 [19] and RT-2 [20] required several months of data collection.

Therefore, in the RT-X [142], the authors have proposed a partnership between university laboratories to collect data with different embodiments performing different tasks to increase the variability of the data. The RT-X effort aims to evaluate how RT-1 and RT-2 performance may increase with more data and to prove if the generic model is better than a specialized one. Still, generalizing between embodiments and new tasks is an open question in the literature.

The primary use cases where these approaches run in a real-world embodiment focus on manipulation, which is the problem of recognizing objects and grasping/using them, such as grasping a bottle or opening a drawer or door. Therefore, none consider collision checking or dynamic constraints, and neither prove completeness. Therefore, applying LLMs directly to motion planning and evaluating their capabilities of understanding geometric and dynamic constraints is still an open question.

LLMs are trained in an astronomical amount of data and are the largest AI models yet with billions of parameters [80]. This scale allows these models to have unseen levels of semantic reasoning, making them adaptable to retrieve the stored knowledge to perform different tasks. However, the amount of data and computational resources required to train and run such models is equally astronomical. Therefore, applying these models in real-time constraints is not feasible, and motion planning requires a certain level of real-time reaction.

The use of LLM in robotics is still in its infancy. Currently, the best tradeoff is to use LLMs as a high-level long horizon planner, and using a specialized low-level controller is the best combination. Still, this subfield of research has open questions for investigation.

D. FUTURE DIRECTIONS

Regarding future directions, we consider LLMs to be a promising direction in solving motion planning and task planning problems. However, the computational and data requirements make training costly, and the latency makes it difficult to apply in scenarios that require real-time processing. Still, there are indications that research on creating foundational LLMs for robotics is a promising direction for research in the field that merges robotics with LLMs because creating foundation models focused on robotics would enable zero-shot application to different embodiments and tasks.

Another promising avenue for future research is methods that combine heuristics with neural networks, and reinforcement learning. Despite their more limited solution scope, these methods seem to have better real-world applications. In this field, research on data representation, that is, how to encode the environment, robotics, and multi-sensor or multi-modal data to feed to the neural network, is one of the most important directions for the progress of these

kinds of approaches. Additionally, the variety and quantity of datasets for benchmarking robotic arm applications are limited. With more benchmarking datasets, one may evaluate and compare methods in various applications, scenarios, and embodiments, leading to better progress in the field.

Finally, the intersection between a high-level planner using LLMs and a low-level controller using traditional or ML methods is a middle ground that we consider another promising direction. In this approach, one can leverage the LLM reasoning capabilities without fine-tuning and with simple prompt engineering. By using traditional or ML methods for low-level control, the data requirements are reduced considerably.

REFERENCES

- [1] A. H. Qureshi and M. C. Yip, "Deeply informed neural sampling for robot motion planning," 2018, *arXiv:1809.10252*.
- [2] E. Schmerling and M. Pavone, "Kinodynamic planning," in *Encyclopedia of Robotics*, 1st ed., Cham, Switzerland: Springer, 2019.
- [3] B. Donald, P. Xavier, J. Canny, and J. Reif, "Kinodynamic motion planning," *J. ACM*, vol. 40, no. 5, pp. 1048–1066, Nov. 1993.
- [4] T. Bhuiyan, L. Kästner, Y. Hu, B. Kutschank, and J. Lambrecht, "Deep-reinforcement-learning-based path planning for industrial robots using distance sensors as observation," in *Proc. 8th Int. Conf. Control Robot. Eng. (ICCRE)*, Apr. 2023, pp. 204–210.
- [5] Z. Li, H. Ma, Y. Ding, C. Wang, and Y. Jin, "Motion planning of six-DOF arm robot based on improved DDPG algorithm," in *Proc. 39th Chin. Control Conf. (CCC)*, Jul. 2020, pp. 3954–3959.
- [6] T. Yu and Q. Chang, "User-guided motion planning with reinforcement learning for human–robot collaboration in smart manufacturing," *Expert Syst. Appl.*, vol. 209, Dec. 2022, Art. no. 118291.
- [7] A. H. Qureshi, Y. Miao, A. Simeonov, and M. C. Yip, "Motion planning networks: Bridging the gap between learning-based and classical motion planners," 2019, *arXiv:1907.06013*.
- [8] E. D. Sacerdoti, "The nonlinear nature of plans," in *Proc. Int. Joint Conf. Artif. Intell.*, Sep. 1975, pp. 206–214.
- [9] C. Reed Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, "Integrated task and motion planning," 2020, *arXiv:2010.01083*.
- [10] M. Mansouri, F. Pecora, and P. Schüller, "Combining task and motion planning: Challenges and guidelines," *Frontiers Robot. AI*, vol. 8, May 2021, Art. no. 637888.
- [11] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge, U.K.: Cambridge Univ. Press, 2017.
- [12] A. Orthey, C. Chamzas, and L. E. Kavraki, "Sampling-based motion planning: A comparative review," *Annu. Rev. Control, Robot., Auto. Syst.*, vol. 7, no. 1, pp. 285–310, Jul. 2024.
- [13] Z. Kingston, M. Moll, and L. E. Kavraki, "Sampling-based methods for motion planning with constraints," *Annu. Rev. Control, Robot., Auto. Syst.*, vol. 1, no. 1, pp. 159–185, May 2018.
- [14] H. Choset, K. M. Lynch, S. Hutchinson, G. A. Kantor, W. Burgard, L. E. Kavraki, and S. Thrun, *Principles of Robot Motion: Theory, Algorithms, and Implementations*. Cambridge, MA, USA: MIT Press, May 2005.
- [15] B. Ichter and M. Pavone, "Robot motion planning in learned latent spaces," 2018, *arXiv:1807.10366*.
- [16] B. Ichter, J. Harrison, and M. Pavone, "Learning sampling distributions for robot motion planning," 2017, *arXiv:1709.05448*.
- [17] M. Tang, X. Yue, Z. Zuo, X. Huang, Y. Liu, and N. Qi, "Coordinated motion planning of dual-arm space robot with deep reinforcement learning," in *Proc. IEEE Int. Conf. Unmanned Syst. (ICUS)*, Oct. 2019, pp. 469–473.
- [18] D. Zhou, R. Jia, and H. Yao, "Robotic arm motion planning based on curriculum reinforcement learning," in *Proc. 6th Int. Conf. Control Robot. Eng. (ICCRE)*, Apr. 2021, pp. 44–49.
- [19] A. Brohan et al., "RT-1: Robotics transformer for real-world control at scale," 2022, *arXiv:2212.06817*.
- [20] A. Brohan et al., "RT-2: Vision-language-action models transfer web knowledge to robotic control," 2023, *arXiv:2307.15818*.

- [21] H. Shao, Y. Hu, L. Wang, S. L. Waslander, Y. Liu, and H. Li, "LMDrive: Closed-loop end-to-end driving with large language models," 2023, *arXiv:2312.07488*.
- [22] J. Wang, T. Zhang, N. Ma, Z. Li, H. Ma, F. Meng, and M. Q.-H. Meng, "A survey of learning-ssbased robot motion planning," *IET Cyber-Syst. Robot.*, vol. 3, no. 4, pp. 302–314, May 2021.
- [23] T. McMahon, A. Sivaramakrishnan, E. Granados, and K. E. Bekris, "A survey on the integration of machine learning with sampling-based motion planning," *Found. Trends Robot.*, vol. 9, no. 4, pp. 266–327, 2022.
- [24] F. Noroozi, M. Daneshmand, and P. Fiorini, "Conventional, heuristic and learning-based robot motion planning: Reviewing frameworks of current practical significance," *Machines*, vol. 11, no. 7, p. 722, Jul. 2023.
- [25] R. Tedrake, *Robotic Manipulation*. Cambridge, MA, USA: MIT, 2024.
- [26] S. M. Lavalle, *Planning Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2006.
- [27] A. H. Qureshi, J. Dong, A. Choe, and M. C. Yip, "Neural manipulation planning on constraint manifolds," 2020, *arXiv:2008.03787*.
- [28] A. H. Qureshi, J. Dong, A. Baig, and M. C. Yip, "Constrained motion planning networks x," 2020, *arXiv:2010.08707*.
- [29] R. E. Fikes and N. J. Nilsson, "STRIPS: A new approach to the application of theorem proving to problem solving," *Artif. Intell.*, vol. 2, nos. 3–4, pp. 189–208, Dec. 1972.
- [30] C. Aeronautiques et al., "Pddl—The planning domain definition language," Yale Center Computat. Vis. Control, New Haven, CT, USA, Tech. Rep. CVC TR98003/DCS TR1165, 1998.
- [31] J. Hoffmann, "FF: The fast-forward planning system," *AI Mag.*, vol. 22, no. 3, p. 57, Sep. 2001.
- [32] E. Plaku and G. D. Hager, "Sampling-based motion and symbolic action planning with geometric and differential constraints," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2010, pp. 5002–5008.
- [33] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Math.*, vol. 1, no. 1, pp. 269–271, Dec. 1959.
- [34] A. H. Qureshi, A. Mousavian, C. Paxton, M. C. Yip, and D. Fox, "NeRP: Neural rearrangement planning for unknown objects," 2021, *arXiv:2106.01352*.
- [35] J. J. Johnson, A. H. Qureshi, and M. Yip, "Learning sampling dictionaries for efficient and generalizable robot motion planning with transformers," 2023, *arXiv:2306.00851*.
- [36] J. J. Johnson, A. H. Qureshi, and M. C. Yip, "Zero-shot constrained motion planning transformers using learned sampling dictionaries," 2023, *arXiv:2309.15272*.
- [37] J. J. Johnson, L. Li, F. Liu, A. H. Qureshi, and M. C. Yip, "Dynamically constrained motion planning networks for non-holonomic robots," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 6937–6943.
- [38] S. M. LaValle, "Rapidly-exploring random trees : A new tool for path planning," Res. Rep. 9811, 1998.
- [39] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Trans. Robot. Autom.*, vol. 12, no. 4, pp. 566–580, Apr. 1996.
- [40] D. Berenson, S. Srinivasa, and J. Kuffner, "Task space regions: A framework for pose-constrained manipulation planning," *Int. J. Robot. Res.*, vol. 30, no. 12, pp. 1435–1460, Mar. 2011.
- [41] D. Berenson, S. S. Srinivasa, D. Ferguson, and J. J. Kuffner, "Manipulation planning on constraint manifolds," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2009, pp. 625–632.
- [42] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," 2011, *arXiv:1105.1186*.
- [43] B. Kim, T. T. Um, C. Suh, and F. C. Park, "Tangent bundle RRT: A randomized algorithm for constrained motion planning," *Robotica*, vol. 34, no. 1, pp. 202–225, Jan. 2016.
- [44] L. Jaillet and J. M. Porta, "Path planning with loop closure constraints using an atlas-based RRT," in *Proc. Int. Symp. Robot. Res.*, 2011, pp. 1–12.
- [45] J. C. Kew, B. Ichter, M. Bandari, T.-W. E. Lee, and A. Faust, "Neural collision clearance estimator for batched motion planning," 2019, *arXiv:1910.05917*.
- [46] L. McNabb and R. S. Laramée, "How to write a visualization survey paper: A starting point," in *Proc. Eurographics (Education Papers)*, Jan. 2019, pp. 29–39.
- [47] C. Zhou, B. Huang, and P. Fränti, "A review of motion planning algorithms for intelligent robots," *J. Intell. Manuf.*, vol. 33, no. 2, pp. 387–424, Feb. 2022.
- [48] X. Xiao, B. Liu, G. Warnell, and P. Stone, "Motion planning and control for mobile robot navigation using machine learning: A survey," *Auto. Robots*, vol. 46, no. 5, pp. 569–597, Mar. 2022.
- [49] L. Dong, Z. He, C. Song, and C. Sun, "A review of mobile robot motion planning methods: From classical motion planning workflows to reinforcement learning-based architectures," *J. Syst. Eng. Electron.*, vol. 34, no. 2, pp. 439–459, Apr. 2023.
- [50] S. Teng, X. Hu, P. Deng, B. Li, Y. Li, Y. Ai, D. Yang, L. Li, Z. Xuanyuan, F. Zhu, and L. Chen, "Motion planning for autonomous driving: The state of the art and future perspectives," *IEEE Trans. Intell. Vehicles*, vol. 8, no. 6, pp. 3692–3711, Jun. 2023.
- [51] S. Aradi, "Survey of deep reinforcement learning for motion planning of autonomous vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 740–759, Feb. 2022.
- [52] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychol. Rev.*, vol. 65, no. 6, pp. 386–408, 1958.
- [53] J. Schmidhuber, "Deep learning in neural networks: An overview," 2014, *arXiv:1404.7828*.
- [54] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, "Dive into deep learning," 2021, *arXiv:2106.11342*.
- [55] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.* Red Hook, NY, USA: Curran Associates, 2012, pp. 1097–1105.
- [56] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015, *arXiv:1512.03385*.
- [57] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," 2016, *arXiv:1610.02357*.
- [58] M. Yu, C. Yu, M.-M. Naddaf-Sh, D. Upadhyay, S. Gao, and C. Fan, "Efficient motion planning for manipulators with control barrier function-induced neural controller," 2024, *arXiv:2404.01184*.
- [59] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, Jul. 2008.
- [60] A. Maćkiewicz and W. Ratajczak, "Principal components analysis (PCA)," *Comput. Geosci.*, vol. 19, no. 3, pp. 303–342, 1993.
- [61] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, *arXiv:1312.6114*.
- [62] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2014, *arXiv:1409.0473*.
- [63] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2017, *arXiv:1706.03762*.
- [64] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [65] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," 2020, *arXiv:2006.11239*.
- [66] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," 2015, *arXiv:1503.03585*.
- [67] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018.
- [68] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, May 1992.
- [69] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [70] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [71] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. 31st Int. Conf. Mach. Learn.*, Beijing, China, Jun. 2014, pp. 387–395.
- [72] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [73] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust region policy optimization," 2015, *arXiv:1502.05477*.

- [74] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba, "Hindsight experience replay," 2017, *arXiv:1707.01495*.
- [75] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction To Natural Language Processing, Computational Linguistics, and Speech Recognition*. Upper Saddle River, NJ, USA: Prentice-Hall, 2009.
- [76] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," 2013, *arXiv:1310.4546*.
- [77] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proc. EMNLP*, Jan. 2014, pp. 1532–1543.
- [78] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," 2014, *arXiv:1409.3215*.
- [79] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [80] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, San Francisco, CA, USA, 2018.
- [81] L. Harold Li, M. Yatskar, D. Yin, C.-J. Hsieh, and K.-W. Chang, "VisualBERT: A simple and performant baseline for vision and language," 2019, *arXiv:1908.03557*.
- [82] A. Radford, J. Wook Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," 2021, *arXiv:2103.00020*.
- [83] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever, "Zero-shot text-to-image generation," 2021, *arXiv:2102.12092*.
- [84] B. Ichter, E. Schmerling, T.-W. E. Lee, and A. Faust, "Learned critical probabilistic roadmaps for robotic motion planning," 2019, *arXiv:1910.03701*.
- [85] C.-M. Hung, S. Zhong, W. Goodwin, O. P. Jones, M. Engelcke, I. Havoutis, and I. Posner, "Reaching through latent space: From joint statistics to path planning in manipulation," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 5334–5341, Apr. 2022.
- [86] A. Fishman, A. Murali, C. Eppner, B. Peele, B. Boots, and D. Fox, "Motion policy networks," in *Proc. 6th Conf. Robot Learn. (CoRL)*, Jan. 2022, pp. 1–13.
- [87] J. J. Johnson, U. S. Kalra, A. Bhatia, L. Li, A. H. Qureshi, and M. C. Yip, "Motion planning transformers: A motion planning framework for mobile robots," 2021, *arXiv:2106.02791*.
- [88] L. Li, Y. Miao, A. H. Qureshi, and M. C. Yip, "MPC-MPNet: Model-predictive motion planning networks for fast, near-optimal planning under kinodynamic constraints," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4496–4503, Jul. 2021.
- [89] F. Xie, M. Dominguez-Kuhne, B. Riviere, J. Song, W. Hönig, S.-J. Chung, and Y. Yue, "Joint-space multi-robot motion planning with learned decentralized heuristics," 2023, *arXiv:2311.12385*.
- [90] C. Gaebert and U. Thomas, "Learning-based adaptive sampling for manipulator motion planning," in *Proc. IEEE 18th Int. Conf. Autom. Sci. Eng. (CASE)*, Aug. 2022, pp. 715–721.
- [91] T. Lai, W. Zhi, T. Hermans, and F. Ramos, "Parallelised diffeomorphic sampling-based motion planning," in *Proc. 5th Conf. Robot Learn.*, vol. 164, A. Faust, D. Hsu, and G. Neumann, Eds., Nov. 2020, pp. 81–90.
- [92] E. Lee and H. Yang, "UaMPNet: Uncertainty-aware motion planning network for manipulator motion planning," *IEEE Access*, vol. 12, pp. 63302–63316, 2024.
- [93] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," 2016, *arXiv:1612.00593*.
- [94] S. Chow, D. Chang, and G. A. Hollinger, "Parallelized control-aware motion planning with learned controller proxies," *IEEE Robot. Autom. Lett.*, vol. 8, no. 4, pp. 2237–2244, Apr. 2023.
- [95] Y. Lu, Y. Ma, D. Hsu, and P. Cai, "Neural randomized planning for whole body robot motion," 2024, *arXiv:2405.11317*.
- [96] G. Palomares, I. Becerra, and R. Murrieta-Cid, "Control inference neural network for motion planning with dynamical systems," *IEEE Robot. Autom. Lett.*, vol. 8, no. 12, pp. 8224–8231, Dec. 2023.
- [97] B. Wulft, P. Matsson, T. B. Schön, and M. Norrlöf, "Neural motion planning in dynamic environments*," *IFAC-PapersOnLine*, vol. 56, no. 2, pp. 10126–10131, 2023.
- [98] R. Ni and A. H. Qureshi, "NTFields: Neural time fields for physics-informed robot motion planning," 2022, *arXiv:2210.00120*.
- [99] R. Ni and A. H. Qureshi, "Progressive learning for physics-informed neural motion planning," 2023, *arXiv:2306.00616*.
- [100] R. Ni and A. H. Qureshi, "Physics-informed neural motion planning on constraint manifolds," 2024, *arXiv:2403.05765*.
- [101] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters, "Motion planning diffusion: Learning and planning of robot motions with diffusion models," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2023, pp. 1916–1923.
- [102] F. Nawaz, T. Li, N. Matni, and N. Figueroa, "Learning complex motion plans using neural ODEs with safety and stability guarantees," 2023, *arXiv:2308.00186*.
- [103] F. F. Arias, B. Ichter, A. Faust, and N. M. Amato, "Avoidance critical probabilistic roadmaps for motion planning in dynamic environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 10264–10270.
- [104] P. Kicki, P. Liu, D. Tateo, H. Bou-Ammar, K. Walas, P. Skrzypczyński, and J. Peters, "Fast kinodynamic planning on the constraint manifold with deep neural networks," *IEEE Trans. Robot.*, vol. 40, pp. 277–297, 2024.
- [105] S. Hou, M. Bdiwi, A. Rashid, S. Krusche, and S. Ihlenfeldt, "A data-driven approach for motion planning of industrial robots controlled by high-level motion commands," *Frontiers Robot. AI*, vol. 9, Jan. 2023, Art. no. 1030668.
- [106] C. Quintero-Peña, W. Thomason, Z. Kingston, A. Kyriklidis, and L. E. Kavraki, "Stochastic implicit neural signed distance functions for safe motion planning under sensing uncertainty," 2023, *arXiv:2309.16862*.
- [107] M. Kurenkov, A. Potapov, A. Savinykh, E. Yudin, E. Kruzhkov, P. Karpyshev, and D. Tsetsurukou, "NFOMP: Neural field for optimal motion planner of differential drive robots with nonholonomic constraints," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 10991–10998, Oct. 2022.
- [108] J. A. Ashley, D. J. Kennedy, and B. Xie, "Kinodynamic motion planning for robotic arms based on learned motion primitives from demonstrations," in *Proc. IEEE/ASME Int. Conf. Adv. Intell. Mechatronics (AIM)*, Jun. 2023, pp. 221–227.
- [109] Z. Huang, H. Liu, J. Wu, and C. Lv, "Differentiable integrated motion prediction and planning with learnable cost function for autonomous driving," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 11, pp. 15222–15236, Nov. 2024.
- [110] T. Osa, "Motion planning by learning the solution manifold in trajectory optimization," *Int. J. Robot. Res.*, vol. 41, no. 3, pp. 281–311, Mar. 2022.
- [111] M. Koptev, N. Figueroa, and A. Billard, "Neural joint space implicit signed distance functions for reactive robot manipulator control," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 480–487, Feb. 2023.
- [112] Y. Hong, Z. Ding, Y. Yuan, W. Chi, and L. Sun, "Obstacle avoidance learning for robot motion planning in human-robot integration environments," in *Proc. IEEE Trans. Cognit. Develop. Syst.*, Feb. 2023, vol. 15, no. 4, pp. 2169–2178.
- [113] R. Strudel, R. Garcia, J. Carpentier, J.-P. Laumond, I. Laptev, and C. Schmid, "Learning obstacle representations for neural motion planning," 2020, *arXiv:2008.11174*.
- [114] D. Zhou, R. Jia, H. Yao, and M. Xie, "Robotic arm motion planning based on residual reinforcement learning," in *Proc. 13th Int. Conf. Comput. Autom. Eng. (ICCAE)*, Mar. 2021, pp. 89–94.
- [115] K. Kamali, I. A. Bonev, and C. Desrosiers, "Real-time motion planning for robotic teleoperation using dynamic-goal deep reinforcement learning," in *Proc. 17th Conf. Comput. Robot Vis. (CRV)*, May 2020, pp. 182–189.
- [116] M. Ewerton, G. Maeda, D. Koert, Z. Kolev, M. Takahashi, and J. Peters, "Reinforcement learning of trajectory distributions: Applications in assisted teleoperation and motion planning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 4294–4300.
- [117] D. Liu, Z. Wang, B. Lu, M. Cong, H. Yu, and Q. Zou, "A reinforcement learning-based framework for robot manipulation skill acquisition," *IEEE Access*, vol. 8, pp. 108429–108437, 2020.
- [118] L. Liu, Q. Liu, Y. Song, B. Pang, X. Yuan, and Q. Xu, "A collaborative control method of dual-arm robots based on deep reinforcement learning," *Appl. Sci.*, vol. 11, no. 4, p. 1816, Feb. 2021.

- [119] C.-C. Wong, S.-Y. Chien, H.-M. Feng, and H. Aoyama, "Motion planning for dual-arm robot based on soft actor-critic," *IEEE Access*, vol. 9, pp. 26871–26885, 2021.
- [120] E. Prianto, M. Kim, J.-H. Park, J.-H. Bae, and J.-S. Kim, "Path planning for multi-arm manipulators using deep reinforcement learning: Soft actor-critic with hindsight experience replay," *Sensors*, vol. 20, no. 20, p. 5911, Oct. 2020.
- [121] E. Prianto, J.-H. Park, J.-H. Bae, and J.-S. Kim, "Deep reinforcement learning-based path planning for multi-arm manipulators with periodically moving obstacles," *Appl. Sci.*, vol. 11, no. 6, p. 2587, 2021.
- [122] K.-W. Park, M. Kim, J. Kim, and J. Park, "Path planning for multi-arm manipulators using soft actor-critic algorithm with position prediction of moving obstacles via LSTM," in *Proc. Appl. Sci.*, Sep. 2022, vol. 12, no. 19, p. 9837.
- [123] Y. Jia, Y. Li, B. Xin, and C. Chen, "Path planning with autonomous obstacle avoidance using reinforcement learning for six-axis arms," in *Proc. IEEE Int. Conf. Netw., Sens. Control (ICNSC)*, Oct. 2020, pp. 1–6.
- [124] A. Nair, V. Pong, M. Dalal, S. Bahl, S. Lin, and S. Levine, "Visual reinforcement learning with imagined goals," 2018, *arXiv:1807.04742*.
- [125] R. Zhang, C. Yu, J. Chen, C. Fan, and S. Gao, "Learning-based motion planning in dynamic environments using GNNs and temporal encoding," in *Proc. Adv. Neural Inf. Process. Syst.*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., Red Hook, NY, USA: Curran Associates, Jan. 2022, pp. 30003–30015.
- [126] A. Wahid, A. V. Stone, K. Chen, B. Ichter, and A. Toshev, "Learning object-conditioned exploration using distributed soft actor critic," in *Proc. Conf. Robot Learn.*, vol. 155, J. Kober, F. Ramos, and C. Tomlin, Eds., Nov. 2020, pp. 1684–1695.
- [127] L. Chen, Z. Jiang, L. Cheng, A. C. Knoll, and M. Zhou, "Deep reinforcement learning based trajectory planning under uncertain constraints," *Frontiers Neurorobotics*, vol. 16, May 2022, Art. no. 883562.
- [128] G. Kulathunga, "A reinforcement learning based path planning approach in 3D environment," *Proc. Comput. Sci.*, vol. 212, pp. 152–160, Jun. 2022.
- [129] S. Gu, G. Chen, L. Zhang, J. Hou, Y. Hu, and A. Knoll, "Constrained reinforcement learning for vehicle motion planning with topological reachability analysis," *Robotics*, vol. 11, no. 4, p. 81, Aug. 2022.
- [130] X. Zhang, Y. Jiang, Y. Lu, and X. Xu, "Receding-horizon reinforcement learning approach for kinodynamic motion planning of autonomous vehicles," *IEEE Trans. Intell. Vehicles*, vol. 7, no. 3, pp. 556–568, Sep. 2022.
- [131] Z. Huang, G. Chen, Y. Shen, R. Wang, C. Liu, and L. Zhang, "An obstacle-avoidance motion planning method for redundant space robot via reinforcement learning," *Actuators*, vol. 12, no. 2, p. 69, 2023.
- [132] C. Bai, J. Zhang, J. Guo, and C. P. Yue, "Adaptive hybrid optimization learning-based accurate motion planning of multi-joint arm," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 9, pp. 5440–5451, Sep. 2023.
- [133] S. Luo and L. Schomaker, "Reinforcement learning in robotic motion planning by combined experience-based planning and self-imitation learning," 2023, *arXiv:2306.06754*.
- [134] D. Lawson and A. H. Qureshi, "Control transformer: Robot navigation in unknown environments through PRM-guided return-conditioned sequence modeling," 2022, *arXiv:2211.06407*.
- [135] S. Wang, M. Han, Z. Jiao, Z. Zhang, Y. Nian Wu, S.-C. Zhu, and H. Liu, "LLM3: Large language model-based task and motion planning with motion failure reasoning," 2024, *arXiv:2403.11552*.
- [136] M. Hallgarten, J. Zapata, M. Stoll, K. Renz, and A. Zell, "Can vehicle motion planning generalize to realistic long-tail scenarios?" 2024, *arXiv:2404.07569*.
- [137] S. P. Sharan, F. Pittaluga, B. G. V. Kumar, and M. Chandraker, "LLM-assist: Enhancing closed-loop planning with language-based reasoning," 2023, *arXiv:2401.00125*.
- [138] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg, "Text2Motion: From natural language instructions to feasible plans," *Auto. Robots*, vol. 47, no. 8, pp. 1345–1365, Nov. 2023.
- [139] Y. Ouyang, J. Li, Y. Li, Z. Li, C. Yu, K. Sreenath, and Y. Wu, "Long-horizon locomotion and manipulation on a quadrupedal robot with large language models," 2024, *arXiv:2404.05291*.
- [140] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," 2022, *arXiv:2207.05608*.
- [141] M. Ahn et al., "Do as I can, not as I say: Grounding language in robotic affordances," 2022, *arXiv:2204.01691*.
- [142] A. O'Neill et al., "Open X-embodiment: Robotic learning datasets and RT-X models," 2023, *arXiv:2310.08864*.
- [143] B. Chen, F. Xia, B. Ichter, K. Rao, K. Gopalakrishnan, M. S. Ryoo, A. Stone, and D. Kappler, "Open-vocabulary queryable scene representations for real world planning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 11509–11522.
- [144] Y. Du, M. Yang, P. Florence, F. Xia, A. Wahid, B. Ichter, P. Sermanet, T. Yu, P. Abbeel, J. B. Tenenbaum, L. Kaelbling, A. Zeng, and J. Tompson, "Video language planning," 2023, *arXiv:2310.10625*.
- [145] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Informed RRT: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic," 2014, *arXiv:1404.2334*.
- [146] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (BIT): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," 2014, *arXiv:1405.5848*.
- [147] L. Janson, E. Schmerling, A. Clark, and M. Pavone, "Fast marching tree: A fast marching sampling-based method for optimal motion planning in many dimensions," 2013, *arXiv:1306.3532*.
- [148] Y. Li, Z. Littlefield, and K. E. Bekris, "Asymptotically optimal sampling-based kinodynamic planning," 2014, *arXiv:1407.2896*.
- [149] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2009, pp. 489–494.
- [150] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep hierarchical feature learning on point sets in a metric space," 2017, *arXiv:1706.02413*.
- [151] M. P. Strub and J. D. Gammell, "Adaptively informed trees (AIT): Fast asymptotically optimal path planning through adaptive heuristics," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2020, pp. 3191–3198.
- [152] K. Van Wyk, M. Xie, A. Li, M. A. Rana, B. Babich, B. Peele, Q. Wan, I. Akinola, B. Sundaralingam, D. Fox, B. Boots, and N. D. Ratliff, "Geometric fabrics: Generalizing classical mechanics to capture the physics of behavior," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 3202–3209, Apr. 2022.
- [153] M. Bhardwaj, B. Sundaralingam, A. Mousavian, N. Ratliff, D. Fox, F. Ramos, and B. Boots, "STORM: An integrated framework for fast joint-space model-predictive control for reactive manipulation," in *Proc. Conf. Robot Learn.*, Jan. 2021, pp. 750–759.
- [154] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robot. Autom. Mag.*, vol. 4, no. 1, pp. 23–33, Mar. 1997.
- [155] C. Rösmann, F. Hoffmann, and T. Bertram, "Kinodynamic trajectory optimization and control for car-like robots," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2017, pp. 5681–5686.
- [156] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.
- [157] J. J. Kuffner and S. M. LaValle, "RRT-connect: An efficient approach to single-query path planning," in *Proc. ICRA. Millennium Conf. IEEE Int. Conf. Robot. Automat. Symposia Proc.*, vol. 2, Dec. 2000, pp. 995–1001.
- [158] S. M. Khansari-Zadeh and A. Billard, "Learning stable nonlinear dynamical systems with Gaussian mixture models," *IEEE Trans. Robot.*, vol. 27, no. 5, pp. 943–957, Oct. 2011.
- [159] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," 2020, *arXiv:2003.08934*.
- [160] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2011, pp. 4569–4574.
- [161] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," 2016, *arXiv:1602.01783*.
- [162] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," 2017, *arXiv:1706.02275*.

- [163] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2012, pp. 5026–5033.
- [164] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” 2018, *arXiv:1801.01290*.
- [165] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” 2018, *arXiv:1802.09477*.
- [166] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [167] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, “Beta-VAE: Learning basic visual concepts with a constrained variational framework,” in *Proc. Int. Conf. Learn. Represent.*, Apr. 2017, pp. 1–12.
- [168] J. van den Berg, M. Lin, and D. Manocha, “Reciprocal velocity obstacles for real-time multi-agent navigation,” in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2008, pp. 1928–1935.
- [169] H. Liu, C. Li, Q. Wu, and Y. Jae Lee, “Visual instruction tuning,” 2023, *arXiv:2304.08485*.
- [170] Y. Liu, N. E. Pears, P. Rosin, and P. Huber, *3D Imaging, Analysis and Applications*, 2nd ed., Cham, Switzerland: Springer, Sep. 2020.
- [171] H. Zhao, L. Jiang, J. Jia, P. Torr, and V. Koltun, “Point transformer,” 2020, *arXiv:2012.09164*.
- [172] Z. Liu, H. Tang, Y. Lin, and S. Han, “Point-voxel CNN for efficient 3D deep learning,” in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2019, pp. 1–11.
- [173] P.-S. Wang, “OctFormer: Octree-based transformers for 3D point clouds,” *ACM Trans. Graph.*, vol. 42, no. 4, pp. 1–11, Jul. 2023.
- [174] Z. Xu, Y. Zhang, E. Xie, Z. Zhao, Y. Guo, K.-Y. K. Wong, Z. Li, and H. Zhao, “DriveGPT4: Interpretable end-to-end autonomous driving via large language model,” 2023, *arXiv:2310.01412*.
- [175] Y. Lin, C. Li, M. Ding, M. Tomizuka, W. Zhan, and M. Althoff, “DrPlanner: Diagnosis and repair of motion planners for automated vehicles using large language models,” 2024, *arXiv:2403.07470*.
- [176] L. Chen, O. Sinavski, J. Hünemann, A. Karnsund, A. J. Willmott, D. Birch, D. Maund, and J. Shotton, “Driving with LLMs: Fusing object-level vector modality for explainable autonomous driving,” 2023, *arXiv:2310.01957*.



GABRIEL PEIXOTO DE CARVALHO received the B.S. degree in computer engineering from the Technology Institute, Federal University of Pará, Belém, Brazil, in 2016, and the M.S. degree in computer science from the Center of Mathematics Computing and Cognition, Federal University of ABC, Santo André, Brazil, in 2019. Currently, he is pursuing the Ph.D. degree in engineering with Osaka University, Osaka, Japan. Since February 2020, he has been with Connected Robotics Inc. as a Robotics AI Research Engineer. His current research interests include machine learning algorithms, computer vision, and robotics.



TETSUYA SAWANOBORI received the B.S. degree in mathematical engineering from the Faculty of Engineering, The University of Tokyo, Tokyo, Japan, in 2005, and the M.S. degree in informatics from the Graduate School of Informatics, Kyoto University, Kyoto, Japan, in 2008. After being an Independent Robot Controller Engineer, he founded Connected Robotics, in 2014 and pivoted the company's focus to the food industry and robotics, in 2017. He has developed robots to improve productivity in food factories and successfully implemented them in several factories. He continues to lead the company to innovate the food industry.



TAKATO HORII (Member, IEEE) received the M.E. and Ph.D. degrees in engineering from Osaka University, Osaka, Japan, in 2013 and 2018, respectively. He was a Project Researcher with The University of Electro-Communications, from 2017 to 2019. He was then an Assistant Professor with Osaka University, from 2019 to 2020, and has been an Associate Professor with Osaka University, since 2020. His current research interests include computational modeling of human cognitive functions, such as emotion and creativity, and machine learning algorithms.

...